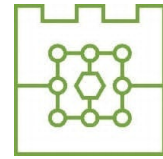




Politechnika Krakowska
im. Tadeusza Kościuszki
Wydział Informatyki i Matematyki



Michał Bąk

Numer albumu: 144753

Porównanie metod uczenia przez wzmacnianie w Ultimate Texas Hold'em z niepełną informacją: wpływ reprezentacji stanu i funkcji nagrody na jakość strategii

Comparison of reinforcement learning methods in Ultimate Texas Hold'em under imperfect information: the impact of state representation and reward function on strategy quality

Praca dyplomowa magisterska
na kierunku Informatyka
specjalność Data Science

Promotor:

dr inż. Anna Plichta

Promotor pomocniczy:

mgr inż. Piotr Biskup

Kraków, 2026

SPIS TREŚCI

Wykaz ważniejszych oznaczeń i skrótów	5
1 Wstęp	7
1.1 Cel pracy	7
1.2 Zakres pracy	7
1.3 Problem badawczy	8
1.4 Struktura pracy	8
2 Podstawy teoretyczne	11
2.1 Poker jako gra z niepełną informacją	11
2.2 Texas Hold'em	11
2.3 Ranking układów pokerowych	11
2.4 Ultimate Texas Hold'em	12
2.4.1 Charakterystyka gry	12
2.4.2 Przebieg rozdania	12
2.4.3 Zakłady i decyzje gracza	13
2.4.4 Rozstrzyganie rozdania	13
2.4.5 Zakłady poboczne	13
2.5 Wartość oczekiwana i przewaga kasyna	14
2.6 Strategie bazowe i ocena jakości strategii	15
2.7 Uczenie przez wzmacnianie	15
2.7.1 Podstawowe pojęcia	16
2.8 Modelowanie UTH jako procesu decyzyjnego	16
3 Projekt i implementacja środowiska Ultimate Texas Hold'em	19
3.1 Założenia projektowe środowiska	19
3.2 Architektura rozwiązania	20
3.3 Model kart i sterowanie talią	22
3.3.1 Reprezentacja karty	22
3.3.2 Standardowa talia	22
3.3.3 Abstrakcja źródła kart i talia testowa	23
3.4 Ocena układów pokerowych	23
3.4.1 Reprezentacja wartości ręki	23
3.4.2 Ocena układu pięciokartowego	24
3.4.3 Wybór najlepszej ręki	25
3.5 Stan wewnętrzny i obserwacja agenta	25
3.5.1 Pełny stan rozdania	25
3.5.2 Obserwacja agenta	27
3.5.3 Projekcja stanu na obserwację	27
3.6 Maszyna stanów i przestrzeń akcji	28

3.7	Porównanie układów i rozliczanie zakładów	29
3.7.1	Przebieg porównania układów	29
3.7.2	Kwalifikacja krupiera	30
3.7.3	Model rozliczenia zakładów	31
3.7.4	Rozliczenie zakładu Blind	31
3.7.5	Rozliczenie Ante, Play i spasowania	32
3.8	Funkcja nagrody i wynik epizodu	32
3.9	Interfejs silnika	32
3.9.1	Tworzenie silnika i rozpoczęcie epizodu	33
3.9.2	Wykonanie decyzji	33
3.9.3	Właściwości publiczne	34
3.10	Rejestrowanie przebiegu i walidacja środowiska	34
3.10.1	Rejestrowanie historii i jej rekordy	34
3.10.2	Poziomy testowania	35
3.11	Ograniczenia środowiska	36
3.12	Podsumowanie	36
4	Agenci bazowi i infrastruktura eksperymentalna	39
4.1	Wspólny interfejs agenta	39
4.2	Prowadzenie pojedynczego epizodu	39
4.3	Agent losowy	39
4.4	Agent regułowy	40
4.4.1	Reguły decyzyjne	40
4.4.2	Implementacja strategii regułowej	41
4.5	Symulacja wielu epizodów i jej powtarzalność	41
4.6	Metryki oceny	42
4.7	Porównanie agentów bazowych	42
4.8	Zapis wyników eksperymentu	43
4.9	Walidacja infrastruktury eksperymentalnej	43
4.10	Podsumowanie	44
5	Metody uczenia przez wzmacnianie	45
5.1	Reprezentacja stanu	45
5.1.1	RAW: reprezentacja surowa	45
5.1.2	FEATURES: reprezentacja z cechami domenowymi	46
5.2	Funkcja nagrody	47
5.3	Deep Q-Network	48
5.3.1	Sieć Q i maskowanie akcji	48
5.3.2	Eksploracja epsilon-greedy	49
5.3.3	Bufor doświadczeń	50
5.3.4	Sieć docelowa i cel Bellmana	50
5.3.5	Strata i optymalizacja	51
5.4	Policy Gradient - REINFORCE	51
5.4.1	Sieć polityki i wybór akcji	52
5.4.2	Uczenie metodą REINFORCE	53
5.4.3	Reprodukowalność i diagnostyka	54
5.5	Porównanie podejść	54
6	Metodyka eksperymentów	57

6.1	Macierz eksperymentalna	57
6.2	Powtarzalność eksperymentów	58
6.3	Budżet treningowy	58
6.4	Walidacja okresowa	59
6.5	Rozszerzona analiza treningu	59
6.6	Ewaluacja końcowa	59
6.7	Porównania sparowane	60
7	Wyniki eksperymentów	63
7.1	Wyniki końcowej ewaluacji	63
7.2	Wpływ reprezentacji stanu	65
7.3	Wpływ funkcji nagrody	67
7.4	Wpływ algorytmu	68
7.5	Przebieg procesu uczenia	70
7.6	Wpływ zwiększenia budżetu treningowego	71
7.7	Stabilność procesu uczenia	74
7.8	Analiza zachowania wyuczonych polityk	75
7.9	Dyskusja wyników	79
	Podsumowanie i dalszy rozwój pracy	83

WYKAZ WAŻNIEJSZYCH OZNACZEŃ I SKRÓTÓW

UTH *Ultimate Texas Hold'em*: badana odmiana pokera kasynowego

RL *reinforcement learning*: uczenie przez wzmacnianie

DQN *Deep Q-Network*: algorytm uczenia przez wzmacnianie oparty na aproksymacji wartości akcji

REINFORCE algorytm uczenia przez wzmacnianie z rodziny policy gradient

EV *expected value*: wartość oczekiwana, tu średni zysk netto na jedno rozdanie

CI *confidence interval*: przedział ufności

SD *standard deviation*: odchylenie standardowe

SE *standard error*: błąd standardowy

POMDP *partially observable Markov decision process*: częściowo obserwowalny proces decyzyjny Markowa

ReLU *rectified linear unit*: funkcja aktywacji sieci neuronowej

RAW wariant reprezentacji stanu oparty wyłącznie na surowym kodowaniu kart

FEATURES wariant reprezentacji stanu rozszerzony o ręcznie zaprojektowane cechy pokerowe

$Q(s, a)$ funkcja wartości akcji a w stanie s

$V^\pi(s)$ funkcja wartości stanu s dla polityki π

γ współczynnik dyskontowania nagród

ε parametr strategii eksploracji epsilon-greedy

1. WSTĘP

Gry karciane od dawna stanowią obszar zastosowania metod sztucznej inteligencji, ponieważ łączą w sobie elementy losowości oraz podejmowania decyzji przy niepełnej informacji. W klasycznych problemach uczenia maszynowego model uczy się na podstawie gotowego zbioru danych, a w grach agent musi samodzielnie podejmować decyzje, obserwować ich konsekwencje i poprawiać swoją strategię.

Szczególnie interesującym przypadkiem są gry pokerowe, w których wynik pojedynczego rozdania nie zależy jedynie od posiadanych kart, ale również od decyzji podjętych w ramach kolejnych etapów gry. Warte uwagi jest tu Ultimate Texas Hold'em, kasynowa odmiana pokera bazująca na zasadach Texas Hold'em, w której gracz rywalizuje nie z innymi graczami, lecz z krupierem. Odmiana ta ma ograniczoną liczbę możliwych akcji, jasno określone reguły wypłat oraz sekwencyjny przebieg rozdania. Te cechy czynią ją odpowiednim problemem do rozważenia w ramach uczenia przez wzmacnianie.

Uczenie przez wzmacnianie (ang. *reinforcement learning*, RL) pozwala analizować sytuacje, w których agent uczy się strategii poprzez interakcję ze środowiskiem. W przypadku Ultimate Texas Hold'em środowiskiem może być symulator gry, obserwacją: informacje dostępne graczowi w bieżącym etapie rozdania, akcją: decyzja gracza, a nagrodą: końcowy wynik finansowy rozdania. Takie podejście umożliwia badanie, w jaki sposób różne reprezentacje stanu oraz różne definicje funkcji nagrody wpływają na jakość wyuczonej strategii.

1.1. Cel pracy

Głównym celem tej pracy jest ocena efektywności wybranych algorytmów uczenia przez wzmacnianie do podejmowania decyzji w Ultimate Texas Hold'em. Zbadano również wpływ sposobu reprezentacji obserwacji oraz konstrukcji funkcji nagrody na przebieg uczenia i jakość otrzymywanej strategii.

Za cel praktyczny obrano zaprojektowanie i implementację kompletnego procesu badawczego obejmującego środowisko gry, agentów bazowych i uczących się, mechanizm prowadzenia treningu oraz powtarzalną ocenę uzyskanych strategii. Porównanie zawiera zarówno różnice pomiędzy metodami uczenia, jak i ich odniesienie do strategii bazowych, w tym agenta losowego i agenta regułowego.

1.2. Zakres pracy

Zakres pracy obejmuje zaprojektowanie i implementację środowiska odwzorowującego podstawowy wariant Ultimate Texas Hold'em, w którym uwzględniono podstawowe elementy rozgrywki: karty gracza i krupiera, karty wspólne, obowiązkowe zakłady Ante i Blind oraz decyzyjny zakład Play. W proponowanej wersji środowiska pominięto

zakłady poboczne, takie jak Trips oraz Progressive, ponieważ nie są one bezpośrednio związane z głównym procesem decyzyjnym agenta.

W ramach pracy przygotowano różne warianty numerycznej reprezentacji obserwacji agenta, utworzonej na podstawie dostępnej części stanu gry. Reprezentacja obserwacji obejmuje między innymi informacje o kartach gracza, kartach wspólnych, aktualnym etapie rozdania oraz dostępnych akcjach. Poddano analizie również wpływ sposobu przekształcania finansowego wyniku rozdania w sygnał nagrody.

Zakres realizacji pracy obejmuje:

1. Implementacja środowiska odwzorowującego podstawowy wariant UTH.
2. Implementacja mechanizmu oceny układów, przebiegu rozdania i rozliczania zakładów.
3. Przygotowanie agentów bazowych (ang. *baseline agents*) stanowiących punkty odniesienia dla metod uczących się.
4. Implementacja wybranych metod uczenia przez wzmacnianie.
5. Przygotowanie i porównanie różnych reprezentacji obserwacji.
6. Przygotowanie i porównanie wybranych funkcji nagrody.
7. Opracowanie powtarzalnej procedury treningu i oceny agentów.
8. Analiza jakości strategii, przebiegu uczenia oraz wyników uzyskanych względem pozostałych metod i strategii bazowych.

1.3. Problem badawczy

Problem badawczy dotyczy wpływu wyboru metody uczenia przez wzmacnianie, reprezentacji obserwacji oraz funkcji nagrody na przebieg uczenia oraz jakość strategii uzyskiwanej w Ultimate Texas Hold'em.

Analizie poddano także to, które konfiguracje pozwalają osiągnąć najwyższy średni wynik netto i najmniejszą oczekiwaną stratę, jak stabilny jest proces uczenia oraz czy uzyskane strategie przewyższają przyjęte strategie bazowe. Agenci bazowi służą jako punkty odniesienia do oceny metod uczenia przez wzmacnianie.

1.4. Struktura pracy

Struktura pracy składa się z kilku rozdziałów. W rozdziale pierwszym przedstawiono cel, zakres oraz problem badawczy pracy. W rozdziale drugim zawarto podstawy teoretyczne dotyczące pokera, Texas Hold'em, Ultimate Texas Hold'em, wartości oczekiwanej, przewagi kasyna oraz uczenia przez wzmacnianie.

W kolejnych rozdziałach opisano projekt środowiska symulacyjnego, sposób reprezentacji stanu gry, funkcję nagrody oraz zastosowane metody uczenia przez wzmocnienie. Następnie przedstawiono eksperymenty, uzyskane wyniki oraz analizę porównawczą agentów. W ostatnim rozdziale zawarto podsumowanie pracy, wnioski oraz możliwe kierunki dalszego rozwoju.

2. PODSTAWY TEORETYCZNE

2.1. *Poker jako gra z niepełną informacją*

Poker jest zbiorem gier karcianych, w których wynik zależy zarówno od losowego rozdania kart, jak i od decyzji podejmowanych przez graczy. W przeciwieństwie do gier z pełną informacją (np. szachy) uczestnicy rozgrywki nie znają wszystkich elementów stanu gry. Ukryte pozostają przede wszystkim karty przeciwników, co sprawia, że decyzje podejmowane są w warunkach niepełnej informacji. Poker z niepełną informacją jest przy tym uznanym testem porównawczym (ang. *benchmark*) w sztucznej inteligencji, czego przykładem jest rozwiązanie uproszczonego wariantu heads-up limit Texas Hold'em [1].

Właśnie z uwagi na tę niepełną informację poker stanowi ciekawy problem badawczy. W pokerze pojedyncza decyzja gracza nie powinna być oceniana wyłącznie na podstawie uzyskanego wyniku na koniec rozdania. Istotniejsze jest, jak dana decyzja wpływa na wartość oczekiwaną (ang. *expected value*, EV) w dłuższym horyzoncie czasowym, przy wielu rozdaniach.

2.2. *Texas Hold'em*

Odmiana pokera Texas Hold'em jest jedną z najpopularniejszych na świecie. W tej odmianie każdy z graczy otrzymuje po dwie zakryte karty własne (ang. *hole cards*), znane tylko temu graczowi, natomiast na stole etapami pojawia się ostatecznie pięć kart wspólnych (ang. *community cards*). Gracz, wykorzystując karty wspólne oraz swoje karty własne, musi utworzyć możliwie najlepszy układ pięciokartowy.

Sama rozgrywka składa się z kilku etapów: etapu przed flopem (ang. *preflop*), flopa (ang. *flop*), czyli odkrycia trzech pierwszych kart wspólnych, turna (ang. *turn*), czyli czwartej karty wspólnej, rivera (ang. *river*), czyli odkrycia piątej i ostatniej karty wspólnej, oraz showdownu (ang. *showdown*), czyli końcowego porównania układów. Warto dodać, że pomiędzy odkrywaniem kolejnych kart wspólnych dokonuje się palenia kart (ang. *burning*), czyli odkładania wierzchniej karty z talii na bok. Po rozdaniu kart własnych oraz po odkryciu flopa, turna i rivera odbywa się runda licytacji, w której gracze mogą stawiać zakłady, podbijać je lub spasować.

2.3. *Ranking układów pokerowych*

Zarówno w Texas Hold'em, jak i w Ultimate Texas Hold'em końcowy wynik rozdania zależy od siły najlepszego układu pięciokartowego utworzonego z dostępnych kart (pięciu kart wspólnych oraz dwóch własnych). Układy pokerowe mają ustaloną hierarchię, która pozwala jednoznacznie porównać ręce graczy lub krupiera w odmianie UTH. Przykładowy ranking układów od najsilniejszego do najsłabszego przedstawiono w tabeli 2.1 [2].

Najsilniejszym układem jest poker królewski, natomiast najsłabszym układem jest wysoka karta. Jeśli gracz i krupier posiadają układ tej samej kategorii (np. obaj mają parę), o zwycięstwie decydują wartości kart składających się na dany układ oraz karty dodatkowe (ang. *kickers*).

Tabela 2.1. Ranking układów pokerowych od najsilniejszego do najsłabszego

Układ	Opis
Poker królewski	A, K, Q, J, 10 w tym samym kolorze
Poker	Pięć kolejnych kart w tym samym kolorze
Kareta	Cztery karty tej samej wartości
Full	Trójka oraz para
Kolor	Pięć kart w tym samym kolorze
Strit	Pięć kolejnych kart
Trójka	Trzy karty tej samej wartości
Dwie pary	Dwa różne układy par
Para	Dwie karty tej samej wartości
Wysoka karta	Brak silniejszego układu

2.4. *Ultimate Texas Hold'em*

2.4.1. Charakterystyka gry

Ultimate Texas Hold'em jest kasynową odmianą pokera opartą na zasadach Texas Hold'em. Różnica polega na tym, że gracz rywalizuje z krupierem, który reprezentuje kasyno, a nie z innymi graczami, co oznacza, że konieczność blefowania (ang. *bluffing*) jest tu zbędna. Istotną cechą Ultimate Texas Hold'em jest również ograniczona liczba momentów decyzyjnych: gracz może wykonać zakład Play (ang. *Play bet*) tylko raz w trakcie rozdania, a im wcześniej podejmie tę decyzję, tym wyższy zakład może postawić, kosztem mniejszej liczby dostępnych informacji [2–4]. Szczegółowe wysokości zakładu Play na poszczególnych etapach przedstawia tabela 2.2.

2.4.2. Przebieg rozdania

Rozdanie rozpoczyna się od postawienia przez gracza obowiązkowych zakładów Ante (ang. *Ante bet*) oraz Blind (ang. *Blind bet*). Oba te zakłady stawia się w tej samej wysokości. Opcjonalnie gracz może również postawić zakład poboczny (ang. *side bet*), na przykład Trips, opisany szerzej w dalszej części tego rozdziału [2].

Kiedy zakłady Blind oraz Ante zostaną postawione, krupier rozdaje graczowi i sobie po dwie karty własne. Karty krupiera pozostają przy tym dla gracza zakryte. Teraz, w fazie preflop, po ujzeniu swoich dwóch kart, gracz stoi przed pierwszą decyzją: może zaczekać (ang. *check*) lub wykonać zakład Play. Czekanie powoduje przejście do kolejnego etapu gry, jakim jest flop: następuje wtedy palenie jednej karty z talii i odkrycie kolejnych trzech kart wspólnych, a gracz znów podejmuje decyzję, czy chce zaczekać, czy wykonać zakład Play. Jeżeli gracz czeka również po flopie, następuje palenie karty i odkrywane są dwie ostatnie karty wspólne (turn i river), a gracz musi wybrać pomiędzy

spasowaniem a wykonaniem zakładu Play. Wysokości zakładu Play na każdym z tych etapów przedstawiono w tabeli 2.2 [2, 3].

2.4.3. Zakłady i decyzje gracza

Zakłady Blind i Ante są obowiązkowe, natomiast zakład Play jest opcjonalny, zależny od decyzji gracza i możliwy do postawienia raz w trakcie rozdania. To właśnie zakład Play, z punktu widzenia modelowania problemu jako środowiska uczenia przez wzmocnianie, jest najważniejszy, ponieważ jego wysokość i moment wykonania wynikają z decyzji agenta. Ante i Blind są traktowane jako początkowy koszt udziału w rozdaniu, natomiast zakład Play odzwierciedla wybór strategii w aktualnym stanie gry.

Tabela 2.2. Dostępne decyzje gracza w Ultimate Texas Hold'em

Etap	Dostępne decyzje	Wysokość zakładu Play
Preflop	czekanie lub zakład Play	3x albo 4x Ante
Flop	czekanie lub zakład Play	2x Ante
Po odkryciu całego stołu	spasowanie lub zakład Play	1x Ante

2.4.4. Rozstrzyganie rozdania

Po dokonaniu decyzji przez gracza o wykonaniu zakładu Play krupier odsłania swoje dwie karty własne. Gracz oraz krupier tworzą najlepszy możliwy układ pięciokartowy z siedmiu dostępnych kart. Następnie układy są porównywane zgodnie z hierarchią układów pokerowych.

Krupier kwalifikuje się do gry (tj. do wypłaty zakładu Ante), jeżeli posiada co najmniej parę. Jeżeli krupier się nie kwalifikuje, zakład Ante jest zwracany graczowi. Zakład Play jest rozliczany na podstawie bezpośredniego porównania układu gracza i krupiera. W przypadku zwycięstwa gracza zakład Play wypłacany jest w stosunku 1:1, w przypadku przegranej gracz traci zakład Play, a w przypadku remisu zakład jest zwracany [2, 3].

Zakład Blind ma odrębną logikę rozliczania i nie jest wypłacany za każdy zwycięski układ gracza. Jeżeli gracz pokona krupiera układem o wartości co najmniej strita, Blind wypłacany jest zgodnie z tabelą wypłat przedstawioną w tabeli 2.3. Jeżeli gracz wygra rozdanie układem słabszym niż strit, zakład Blind jest zwracany. W przypadku przegranej gracza zakład Blind jest tracony, a w przypadku remisu zwracany [2].

2.4.5. Zakłady poboczne

W wielu wariantach Ultimate Texas Hold'em dostępne są również zakłady poboczne, takie jak Trips i Progressive. Zakład Trips jest opcjonalny i wypłacany na podstawie końcowego układu gracza, niezależnie od tego, czy gracz pokonał krupiera. Najczęściej wygrywa wtedy, gdy końcowy układ gracza ma wartość co najmniej trójki. Zakłady poboczne mogą różnić się w zależności od kasyna i stosowanej tabeli wypłat [2].

Głównym celem pracy jest modelowanie sekwencyjnego procesu decyzyjnego do-

Tabela 2.3. Tabela wypłat zakładu Blind przyjęta w modelu środowiska

Układ gracza	Wypłata
Poker królewski	500:1
Poker	50:1
Kareta	10:1
Full	3:1
Kolor	3:2
Strit	1:1
Pozostałe układy	Zwrot

tyczącego zakładu Play. Ponieważ zakłady poboczne są opcjonalne, a wysokość ich wypłat zależy od konkretnego kasyna, postanowiono ich nie uwzględniać w modelu środowiska.

2.5. Wartość oczekiwana i przewaga kasyna

Wartość oczekiwana (ang. *expected value*, EV) określa średni wynik finansowy decyzji lub strategii przy założeniu dużej liczby powtórzeń gry [5]. W kontekście Ultimate Texas Hold'em pojedyncze rozdanie może zakończyć się zarówno zyskiem, jak i stratą, jednak skuteczność strategii należy oceniać na podstawie średniego wyniku uzyskiwanego w wielu rozdaniach.

Przewaga kasyna (ang. *house edge*) jest matematyczną miarą określającą oczekiwany zysk kasyna względem zakładu gracza. Nie oznacza to jednak, że kasyno wygrywa każde pojedyncze rozdanie, lecz że przy dużej liczbie rozegranych rozdań średni wynik będzie przesunął się na korzyść kasyna. Oczywiście, gracz może osiągać krótkoterminowe zyski, jednak w długim okresie wynik gry powinien zbliżać się do wartości oczekiwanej wynikającej z zasad gry [6].

W Ultimate Texas Hold'em przewaga kasyna podawana jest względem początkowego zakładu Ante. Metryki przyjęte w pracy opierają się na wyniku netto względem Ante. Nawet niewielka procentowa przewaga kasyna prowadzi do istotnej oczekiwanej straty w długim horyzoncie czasowym.

W tabeli 2.4 przedstawiono przykładowe wartości przewagi kasyna dla wybranych gier kasynowych, w tym Ultimate Texas Hold'em [7].

Tabela 2.4. Przykładowe wartości przewagi kasyna dla wybranych gier kasynowych

Gra	Przewaga kasyna
Blackjack	0,28%
Ultimate Texas Hold'em	2,19%
Ruletka europejska	2,70%
Ruletka amerykańska	5,26%
Automaty do gier	2–15%

Tabela 2.4 pokazuje, że przewaga kasyna jest cechą konstrukcyjną gier hazardowych, ale jej poziom istotnie różni się pomiędzy grami, a podane wartości zależą przy

tym od wariantu zasad gry.

Podane wartości nie są jednak w pełni porównywalne między sobą, ponieważ różnią się bazą odniesienia. Dla Ultimate Texas Hold'em przewaga 2,19% jest liczona względem samego zakładu Ante, podczas gdy średnia całkowita stawka gracza (Ante, Blind i Play łącznie) wynosi ponad cztery jednostki Ante. Miarą lepiej porównywalną między grami różniącymi się liczbą i wysokością zakładów jest element ryzyka (ang. *element of risk*), czyli przewaga kasyna liczona względem średniej całkowitej stawki. Dla optymalnej strategii Ultimate Texas Hold'em element ryzyka wynosi 0,526%, znacznie mniej niż przewaga 2,185% liczona względem samego Ante [3]. Wyniki analizowane w tej pracy wyrażono, jak już wspomniano, względem zakładu Ante, dlatego przy porównaniach z innymi grami warto mieć na uwadze tę różnicę bazy odniesienia.

Warto zaznaczyć, że celem niniejszej pracy nie jest wykazanie, że agent może trwale uzyskać dodatnią wartość oczekiwaną w grze fundamentalnie zaprojektowanej na korzyść kasyna, lecz sprawdzenie, które metody uczenia przez wzmacnianie są w stanie nauczyć się strategii minimalizującej tę przewagę.

2.6. Strategie bazowe i ocena jakości strategii

Wprowadzono dwie strategie bazowe: agenta losowego (ang. *random agent*) i agenta regułowego (ang. *rule-based agent*). Obaj agenci stanowią punkty odniesienia, pozwalające ocenić jakość strategii wyuczonej przez agenta uczącego się przez wzmacnianie. Istotne jest porównanie agentów uczących się pomiędzy sobą, ale również względem strategii bazowych. Agent losowy wybiera jedną z legalnych akcji bez uwzględniania siły kart, stanowi prosty punkt odniesienia i umożliwia wstępną kontrolę działania infrastruktury eksperymentalnej. Agent regułowy wykorzystuje z góry określone heurystyki i reprezentuje bardziej wymagający punkt odniesienia.

Samo przewyższenie agenta losowego nie stanowi wystarczającego dowodu uzyskania strategii dobrej jakości, natomiast porównanie z agentem regułowym pozwala ocenić, czy uczenie prowadzi do decyzji konkurencyjnych wobec jawnie zaprojektowanych zasad.

2.7. Uczenie przez wzmacnianie

Uczenie przez wzmacnianie (ang. *reinforcement learning*, RL) jest paradygmatem uczenia maszynowego, w którym agent uczy się podejmowania decyzji poprzez interakcję ze środowiskiem. W każdym kroku agent obserwuje aktualny stan (ang. *state*) środowiska, wybiera akcję (ang. *action*), a następnie otrzymuje nagrodę (ang. *reward*) oraz przechodzi do kolejnego stanu. Celem agenta jest wyuczenie strategii, określanej również jako polityka decyzyjna (ang. *policy*), która maksymalizuje sumę nagród w długim horyzoncie czasowym [8].

Przed flopem gracz zna jedynie swoje dwie karty, ale może wykonać najwyższy

zakład Play. Po flopie posiada więcej informacji, lecz maksymalny zakład jest mniejszy. Po odkryciu wszystkich kart wspólnych zna już pełny board, ale może wykonać wyłączenie zakład 1x Ante albo spasować. Agent musi więc nauczyć się kompromisu pomiędzy ryzykiem, ilością informacji oraz potencjalną wypłatą.

2.7.1. Podstawowe pojęcia

Zachowanie agenta opisuje polityka, która może być deterministyczna, przypisując stanowi jedną akcję, albo stochastyczna, określając rozkład prawdopodobieństwa wyboru akcji. Jakość decyzji ocenia się na podstawie zwrotu epizodycznego, czyli sumy nagród uzyskanych do końca epizodu:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}, \quad (2.1)$$

gdzie $\gamma \in [0, 1]$ jest współczynnikiem dyskontowania, a T oznacza koniec epizodu. Współczynnik γ reguluje wagę nagród odległych w czasie względem nagród natychmiastowych [8].

Oczekiwany zwrot z danego stanu opisuje funkcja wartości stanu $V^\pi(s)$, natomiast oczekiwany zwrot po wykonaniu akcji w danym stanie opisuje funkcja wartości akcji $Q^\pi(s, a)$. Te funkcje pozwalają porównywać decyzje i stanowią podstawę w wielu metodach uczenia [8].

W trakcie uczenia agent musi równoważyć eksplorację (ang. *exploration*), czyli wypróbowywanie nowych akcji w celu zdobycia informacji, oraz eksploatację (ang. *exploitation*), czyli wybór akcji uznanych dotąd za najlepsze. Niewłaściwy balans między nimi prowadzi do niestabilnego przebiegu uczenia.

Funkcje wartości i polityki można reprezentować w postaci tablicowej, gdy przestrzeń stanów jest niewielka, albo za pomocą aproksymacji funkcji, na przykład sieci neuronowej, gdy przestrzeń stanów jest duża. Przestrzeń stanów Ultimate Texas Hold'em jest zbyt duża na podejście tablicowe, dlatego funkcja wartości akcji jest aproksymowana siecią neuronową, co opisano szczegółowo w rozdziale dotyczącym zastosowanej metody uczenia [8].

Dwa elementy konfiguracji procesu uczenia mają niebagatelne znaczenie: sposób reprezentacji obserwacji, decydujący o tym, jakie informacje trafiają na wejście agenta, oraz konstrukcja funkcji nagrody, przekształcająca finansowy wynik rozdania w sygnał uczący. Na analizie tych dwóch elementów skupiono się w niniejszej pracy.

2.8. Modelowanie UTH jako procesu decyzyjnego

Kończąc niniejszy rozdział, warto podkreślić, że w Ultimate Texas Hold'em agent nie zna pełnego stanu środowiska, a jedynie jego obserwację. Agent nie wie, jakie karty ma krupier, ani jakie karty są następne w talii. Problem niniejszej pracy można więc trakto-

wać jako częściowo obserwowalny proces decyzyjny Markowa (ang. *partially observable Markov decision process*, POMDP) [9]. To rozróżnienie obserwacji od pełnego stanu jest istotne dla zachowania niepełnej informacji, która jest charakterystyczna dla pokera. Odwzorowano to w projekcie środowiska opisanym w kolejnym rozdziale [8].

3. PROJEKT I IMPLEMENTACJA ŚRODOWISKA ULTIMATE TEXAS HOLD'EM

W niniejszym rozdziale przedstawiono projekt oraz implementację środowiska Ultimate Texas Hold'em wykorzystywanego w dalszej części pracy do uczenia i oceny agentów. Wcześniej opisane zasady gry odwzorowano w postaci silnika odpowiedzialnego za zarządzanie całym przebiegiem rozdania. Silnik udostępnia agentowi legalne akcje do wykonania oraz rozlicza wynik finansowy rozdania.

Implementację podzielono na niezależne moduły odpowiedzialne między innymi za reprezentację kart, ocenę układów, sterowanie przebiegiem rozdania, walidację akcji oraz rozliczanie zakładów. Kodowanie obserwacji, odwzorowanie akcji i konstrukcja nagrody są poza warstwą realizującą zasady gry, co umożliwia porównywanie różnych reprezentacji stanu i funkcji nagrody.

3.1. Założenia projektowe środowiska

Podstawową jednostką interakcji ze środowiskiem jest jedno kompletne rozdanie (epizod). Rozdanie rozpoczyna się od utworzenia talii i przekazania graczowi oraz krupierowi po dwóch kart własnych, a kończy się spasowaniem gracza albo automatycznym rozstrzygnięciem układów.

Agent może podjąć decyzje wyłącznie w trzech etapach: przed flopem, po odkryciu flopa oraz po odkryciu wszystkich pięciu kart wspólnych. W zależności od etapu może czekać, wykonać zakład Play o odpowiednim mnożniku albo spasować. Po wykonaniu zakładu Play przez gracza środowisko automatycznie odkrywa pozostałe karty, ocenia ręce gracza i krupiera, i przechodzi do stanu końcowego.

Wartości zakładów wyrażono w jednostkach względnych, tzn. jedna jednostka odpowiada wartości zakładu Ante. Zakłady Ante i Blind mają zatem wartość jednej jednostki, natomiast wartość zakładu Play zależy od momentu jego wykonania i może wynosić od jednej do czterech jednostek. Przyjęto takie podejście, aby uniezależnić środowisko od konkretnej waluty i wysokości stawki.

Silnik zwraca szczegółowe rozliczenie poszczególnych zakładów oraz łączny wynik netto rozdania, będący podstawą nagrody terminalnej. Rozdzielono mechanizm rozliczania gry od funkcji nagrody. Uzasadnienie tej decyzji przedstawiono w podrozdziale 3.8.

Zadbano również o powtarzalność eksperymentów, przez co talia kart może być tasowana przy użyciu ziarna generatora liczb pseudolosowych. Dzięki temu możliwe jest odtworzenie całej sekwencji rozdań. W testach jednostkowych zastosowano dodatkowo talię deterministyczną, która pozwala określić kolejność dobieranych kart i zweryfikować konkretne przypadki przebiegu oraz rozliczenia rozdania.

3.2. Architektura rozwiązania

Środowisko Ultimate Texas Hold'em zaprojektowano jako zestaw niezależnych modułów domenowych. Każdy moduł odpowiada za jeden określony obszar, taki jak reprezentacja kart, ocena układów, sterowanie przebiegiem rozdania albo rozliczanie zakładów. Centralnym elementem rozwiązania jest klasa `UTHGame`, która odpowiada za koordynowanie działania pozostałych komponentów, ale nie implementuje bezpośrednio wszystkich reguł gry.

Przyjęto taki podział, aby ograniczyć zależności pomiędzy poszczególnymi częściami systemu. Model kart i ewaluacja układów są dzięki temu testowane niezależnie od zasad Ultimate Texas Hold'em. Analogicznie mechanizm rozliczania zakładów nie odpowiada za odkrywanie kart ani za walidację dozwolonych akcji. Dzięki temu zmiana jednego elementu, na przykład reprezentacji obserwacji agenta, nie wymaga modyfikowania sposobu oceny układów lub obliczania wypłat.

Najniższą warstwę rozwiązania stanowią moduły `cards` i `hands`. Pierwszy z nich definiuje karty oraz talię, natomiast drugi odpowiada za ocenę i porównywanie układów pokerowych.

Moduł `uth` zawiera modele stanu, reguły legalności akcji, mechanizm rozdawania kart, sposób rozliczania zakładów oraz logikę sterującą kolejnymi fazami rozdania. Klasa `UTHGame` pełni rolę menedżera tej warstwy. Na podstawie aktualnego stanu i wybranej akcji wywołuje ona odpowiednie operacje, aktualizuje stan rozdania oraz zwraca rezultat danego kroku.

Agent nie komunikuje się bezpośrednio z talią, ewaluatorem ani pełnym stanem wewnętrznym gry. Otrzymuje obiekt `UTHObservation`, a swoją decyzję przekazuje do metody `step`. Wynikiem wykonania akcji jest obiekt `StepResult`, zawierający kolejną obserwację oraz, w przypadku zakończenia rozdania, jego wynik i szczegółowe rozliczenie. Takie podejście pozwala rozgraniczyć logikę środowiska od implementacji agenta.

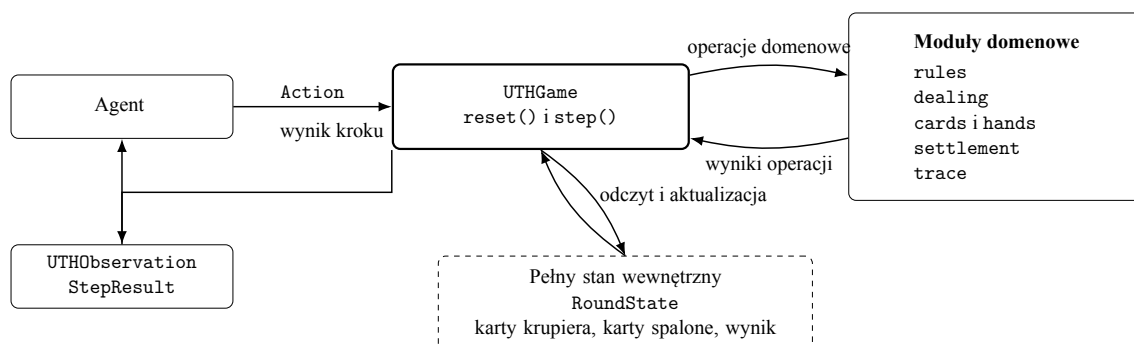
W tabeli 3.1 przedstawiono najważniejsze moduły rozwiązania i zakres ich odpowiedzialności.

Co istotne, silnik domenowy nie zależy od konkretnej biblioteki uczenia przez wzmacnianie. Integrację z takimi bibliotekami wydzielono do osobnej warstwy korzystającej z publicznego interfejsu `UTHGame` i przekształcającej obiekty domenowe na reprezentacje numeryczne. Pozwala to zachować jedną implementację zasad gry dla wszystkich badanych reprezentacji obserwacji, funkcji nagrody i metod uczenia.

Ogólny przepływ informacji pomiędzy agentem, klasą sterującą i pozostałymi elementami silnika przedstawiono na rysunku 3.1. Agent komunikuje się wyłącznie z publicznym interfejsem klasy `UTHGame`. Pełny stan rozdania oraz moduły realizujące reguły gry pozostają ukryte za tą warstwą.

Tabela 3.1. Podział odpowiedzialności pomiędzy modułami środowiska UTH

Moduł	Odpowiedzialność
cards	Reprezentacja pojedynczej karty, jej rangi, koloru oraz talii składającej się z 52 unikalnych kart.
hands	Ocena pięciokartowych układów, wybór najlepszego układu z pięciu, sześciu lub siedmiu kart oraz porównywanie wartości rąk.
uth.models	Definicja pełnego stanu rozdania, obserwacji agenta oraz wyniku pojedynczego kroku środowiska.
uth.rules	Określanie zbioru legalnych akcji dla każdej fazy rozdania.
uth.dealing	Realizacja kolejności rozdawania kart, odkrywania flopa, turna i rivera oraz obsługa kart spalonych.
uth.card_source	Abstrakcja źródła kart oraz deterministyczna talia wykorzystywana w testach.
uth.settlement	Określanie wyniku rozdania, kwalifikacji krupiera oraz rozliczanie zakładów Ante, Blind i Play.
uth.game	Koordinacja przebiegu rozdania, walidacja akcji, przechodzenie pomiędzy fazami oraz tworzenie wyników kolejnych kroków.
uth.trace	Opcjonalne rejestrowanie obserwacji, decyzji agenta i końcowego stanu rozdania na potrzeby diagnostyki.



Rys. 3.1. Architektura i przepływ informacji w środowisku UTH

3.3. Model kart i sterowanie talią

Reprezentacja kart i mechanizm sterowania talią stanowią podstawową warstwę środowiska. Elementy te zaprojektowano niezależnie od zasad Ultimate Texas Hold'em, dzięki czemu mogą być wykorzystywane także przez inne warianty pokera, o czym wspomniano w części pracy poświęconej możliwym kierunkom dalszego rozwoju.

3.3.1. Reprezentacja karty

Pojedynczą kartę zaimplementowano jako niemodyfikowalny obiekt `Card`, zawierający dwa pola: rangę oraz kolor. Rangi zakodowano za pomocą typu wyliczeniowego `Rank`, natomiast kolory za pomocą typu `Suit`. Przez zastosowanie typów wyliczeniowych ograniczono możliwość utworzenia karty zawierającej niepoprawną wartość, a przez podejście do karty jako niemutowalnego obiektu zapewniono bezpieczne umieszczanie jej w krotkach i zbiorach, a także wykorzystania porównania liczebności zbioru do wykrywania zduplikowanych kart.

Rangi kart przyjmują wartości liczbowe od 2 do 14. Karty od dwójki do dziesiątki zachowują swoje naturalne wartości, natomiast walet, dama, król i as otrzymują odpowiednio wartości 11, 12, 13 i 14. Przyjęto taką konwencję, ponieważ umożliwia ona bezpośrednie porównywanie rang podczas oceny układów.

Warto dodać, że as jest kartą najwyższą, natomiast szczególny przypadek strita od asa do piątki obsługiwany jest przez ewaluator układów. Zbiór kolorów obejmuje trefle, karo, kiery i piki.

3.3.2. Standardowa talia

Klasa `Deck` reprezentuje standardową talię. Podczas jej tworzenia generowane są wszystkie kombinacje trzynastu rang i czterech kolorów. Liczba kart w pełnej talii wynosi zatem

$$|\mathcal{D}| = 13 \cdot 4 = 52. \quad (3.1)$$

Ponieważ każda kombinacja rangi i koloru występuje dokładnie raz, wygenerowana talia zawiera 52 unikalne karty.

Podstawową operacją jest metoda `draw()`, która pobiera jedną kartę z wierzchu talii i następnie ją usuwa z kolekcji. Jest również operacja służąca do pobrania wielu kart `draw_many()`. Próba pobrania karty z pustej talii lub większej liczby kart niż liczba dostępnych kart pozostałych w talii powoduje zgłoszenie wyjątku. Takie zachowanie umożliwia szybkie wykrycie niepoprawnego przebiegu rozdania.

Domyślnie talia jest tasowana bezpośrednio po utworzeniu. Zadbano o możliwość utworzenia talii w konkretnej kolejności początkowej, co bywa użyteczne podczas testowania samego modelu kart.

Aby zapewnić powtarzalność eksperymentów, w klasie Deck zastosowano możliwość przekazania ziarna generatora liczb pseudolosowych. Utworzenie dwóch talii z tym samym ziarnem prowadzi do uzyskania tej samej kolejności kart. Na poziomie klasy UTHGame zastosowano dodatkowo generator nadrzędny, który przy każdym rozpoczęciu nowego rozdania generuje osobne ziarno przekazywane do nowej talii. Dzięki temu kolejne rozdania w obrębie jednego eksperymentu różnią się od siebie, ale cała ich sekwencja może zostać odtworzona przez ponowne uruchomienie środowiska z tym samym ziarnem początkowym. Jeżeli dwa eksperymenty zostaną uruchomione z tym samym ziarnem środowiska i zachowają tę samą kolejność wywołań, otrzymają identyczne talie, co ogranicza wpływ losowego doboru kart na ocenę różnic pomiędzy badanymi strategiami.

3.3.3. Abstrakcja źródła kart i talia testowa

Silnik rozdania nie zależy bezpośrednio od klasy Deck. Zamiast tego korzysta z abstrakcji `CardSource`, która wymaga jedynie udostępnienia operacji `draw()`. Każdy obiekt spełniający ten kontrakt może zostać użyty jako źródło kolejnych kart, dzięki czemu moduł odpowiedzialny za rozdawanie nie musi rozpoznawać, czy korzysta z przetasowanej talii, czy ze źródła deterministycznego.

Dzięki deterministycznemu źródłu możliwe było przygotowanie powtarzalnych testów obejmujących określone scenariusze rozgrywki, kwalifikacji krupiera i rozliczenia zakładów. Do testowania konkretnych scenariuszy wprowadzono klasę `FixedDeck`, która jest jawnie przekazaną sekwencją kart. Przed zapisaniem sekwencji sprawdzane jest, czy wszystkie jej elementy są poprawnymi obiektami `Card` oraz czy każda karta jest unikalna, dzięki czemu błąd w danych testowych nie może doprowadzić do rozdania zawierającego dwie identyczne karty.

3.4. Ocena układów pokerowych

Zadaniem modułu oceny układów pokerowych jest określenie wartości dowolnego poprawnego układu pięciokartowego oraz wybranie najlepszej możliwej ręki z pięciu, sześciu lub siedmiu dostępnych kart. W finalnej fazie, podczas porównania układów, ewaluator jest wywoływany dla gracza i krupiera zawsze na siedmiu kartach. Pozostałe dwa warianty wykorzystuje natomiast agent regułowy opisany w rozdziale poświęconym agentom bazowym.

3.4.1. Reprezentacja wartości ręki

Do reprezentacji kategorii układów pokerowych zaimplementowano typ wyliczeniowy `HandRank`. Kolejnym kategoriom przypisuje rosnące wartości liczbowe od 0 do 8, począwszy od wysokiej karty, a kończąc na pokerze. Dzięki temu porównanie kategorii odbywa się wprost przez porównanie odpowiadających im wartości całkowitych.

Sama kategoria nie wystarcza jednak do rozstrzygnięcia wszystkich przypadków

(np. gracz i krupier mogą mieć układy tej samej kategorii, lecz różniące się rangą kart tworzących dany układ albo kartami dodatkowymi). Z tego względu pełną wartość ręki zaimplementowano jako obiekt `HandValue`, zawierający kategorię układu `rank` oraz uporządkowaną krotkę wartości rozstrzygających remis `tiebreak`.

Klucz porównawczy ręki można zapisać jako

$$K(h) = (r(h), t_1(h), t_2(h), \dots, t_n(h)), \quad (3.2)$$

gdzie $r(h)$ oznacza wartość kategorii układu, natomiast $t_1(h), \dots, t_n(h)$ są kolejnymi wartościami rozstrzygającymi remis. Klucze porównywane są leksykograficznie. W pierwszej kolejności porównywana jest kategoria układu, a dopiero w przypadku jej równości kolejne elementy krotki `tiebreak`.

Dla lepszego zobrazowania, wartość pary asów z królem, dziesiątką i dziewiątką jako kartami dodatkowymi może zostać zapisana w postaci

$$K(h) = (\text{ONE_PAIR}, 14, 13, 10, 9). \quad (3.3)$$

Jeżeli druga ręka również zawiera parę asów, porównanie przechodzi kolejno do króla, dziesiątki i dziewiątki. Kolory kart nie uczestniczą w rozstrzyganiu remisów, zgodnie ze standardowymi zasadami pokera.

Obiekt `HandValue` sprawdza zgodność długości krotki `tiebreak` z kategorią układu oraz waliduje, czy wszystkie jej elementy są liczbami całkowitymi odpowiadającymi poprawnym rangom kart. W ten sposób ograniczono możliwość utworzenia wartości ręki, która nie mogłaby powstać w prawidłowym rozdaniu.

3.4.2. Ocena układu pięciokartowego

Funkcja `evaluate_five_card_hand()` przyjmuje pięć unikalnych kart. W pierwszym kroku funkcja oblicza liczbę wystąpień każdej rangi. Na tej podstawie możliwe jest wykrycie par, dwóch par, trójki, fulla oraz karety. Niezależnie sprawdzane są dwa warunki: czy wszystkie karty mają ten sam kolor oraz czy pięć różnych rang tworzy ciąg kolejnych wartości.

Kategorie sprawdzane są od najsilniejszej do najsłabszej. Jeżeli spełnione są jednocześnie warunki strita i koloru, zwracany jest poker. W przeciwnym razie funkcja sprawdza kolejno kareta, full, kolor, strit, trójka, dwie pary, jedna para oraz wysoka karta.

Wartość `tiebreak` jest budowana bezpośrednio podczas rozpoznawania kategorii. Wynikiem funkcji nie jest jedynie nazwa układu, lecz kompletna wartość umożliwiająca porównanie z dowolną inną poprawną ręką. Szczególnym przypadkiem jest strit A-2-3-4-5, w którym as pełni rolę najniższej karty, ewaluator zwraca wtedy strita o najwyższej karcie równej 5, bez zmiany wartości asa w pozostałych kategoriach.

3.4.3. Wybór najlepszej ręki

Podczas porównania układów gracz i krupier dysponują łącznie siedmioma kartami. Rękę pokerową tworzy dokładnie pięć kart, dlatego funkcja `evaluate_best_hand()` generuje wszystkie możliwe pięciokartowe podzbiory dostępnych kart, ocenia każdy z nich, a następnie wybiera układ o największej wartości `HandValue`. Generowanie każdego możliwego podzbioru nie jest optymalnym rozwiązaniem, jednak liczba kombinacji jest niewielka i stała, co potraktowano jako kompromis pomiędzy prostotą implementacji a wydajnością.

Liczba sprawdzanych kombinacji dla n dostępnych kart wynosi

$$N(n) = \binom{n}{5}. \quad (3.4)$$

Dla liczby kart obsługiwanych przez ewaluator jest tyle kombinacji:

$$\binom{5}{5} = 1, \quad \binom{6}{5} = 6, \quad \binom{7}{5} = 21. \quad (3.5)$$

W przypadku standardowego porównania układów w UTH konieczne jest zatem sprawdzenie 21 kombinacji dla gracza oraz 21 kombinacji dla krupiera.

Przed wygenerowaniem kombinacji sprawdzane jest, czy przekazano od pięciu do siedmiu kart, czy wszystkie elementy są kartami oraz czy żadna karta nie występuje więcej niż raz.

Wynikiem operacji jest obiekt `EvaluatedHand`, który przechowuje zarówno wartość `HandValue`, jak i dokładnie pięć kart tworzących wybrany układ. Jest to wygodny sposób prezentacji wyniku porównania układów oraz diagnostyki ewaluatora.

Dla najwyższej kategorii, jaką jest poker królewski, nie utworzono osobnej kategorii `HandRank`. Rozwinięto ją jako poker z asem jako najwyższą kartą.

3.5. Stan wewnętrzny i obserwacja agenta

Ponieważ Ultimate Texas Hold'em jest grą z niepełną informacją, odwzorowanie tej właściwości wymagało rozdzielenia pełnego stanu środowiska od informacji udostępnianych agentowi.

W implementacji zastosowano dwa odrębne modele danych: `RoundState` przedstawiający pełny stan wewnętrzny rozdania oraz `UTHObservation` reprezentujący obserwację dostępną agentowi.

Agent nie powinien i nie otrzymuje odwołania do obiektu `RoundState`. Każda obserwacja jest tworzona na kanwie pełnego stanu przez osobną funkcję projekcji, która wyciąga ze stanu wyłącznie informacje dozwolone z perspektywy gracza.

3.5.1. Pełny stan rozdania

Obiekt `RoundState` zawiera wszystkie dane na temat rozdania. Są to:

- ♠ aktualną fazę gry,
- ♠ dwie karty własne gracza,
- ♠ dwie ukryte karty krupiera,
- ♠ odkryte karty wspólne,
- ♠ karty spalone,
- ♠ mnożnik wykonanego zakładu Play,
- ♠ końcowy wynik rozdania,
- ♠ szczegółowe rozliczenie zakładów,
- ♠ rezultat porównania układów.

Naturalnie nie wszystkie pola są aktywne jednocześnie. W stanach pośrednich wynik rozdania, rozliczenie, rezultat porównania układów oraz mnożnik zakładu Play pozostają nieokreślone. Pola te otrzymują wartości dopiero po przejściu do stanu terminalnego.

Dla przykładu liczba kart wspólnych i spalonych jest zależna od aktualnej fazy. Tę zależność przedstawiono w tabeli 3.2.

Tabela 3.2. Liczba kart przechowywanych w stanie w poszczególnych fazach

Faza	Karty wspólne	Karty spalone	Znaczenie
PREFLOP	0	0	Rozdano wyłącznie karty własne gracza i krupiera.
FLOP	3	1	Spalono jedną kartę i odkryto flop.
RIVER	5	2	Odkryto komplet kart wspólnych.
TERMINAL	5	2	Rozdanie zostało zakończone spasowaniem albo porównaniem układów.

Obiekt stanu zaimplementowano jako niemodyfikowalny, co wymusza utworzenie nowej instancji `RoundState` przy każdym przejściu środowiska (zamiast modyfikowania istniejącego obiektu). Ogranicza to ryzyko, że obiekt stanu może potencjalnie zostać niewłaściwie zmodyfikowany, a także ułatwia tworzenie deterministycznych testów.

Podczas tworzenia stanu sprawdzane są między innymi:

- ♠ poprawny typ fazy,
- ♠ obecność dokładnie dwóch kart gracza i dwóch kart krupiera,
- ♠ liczba kart wspólnych właściwa dla danej fazy,
- ♠ liczba kart spalonych właściwa dla danej fazy,

- ♠ brak zduplikowanych kart w całym stanie,
- ♠ zgodność pól końcowych z fazą rozdania.

Sprawdzone jest również, aby stan niekończący nie zawierał wyniku ani rozliczenia, stan zakończony spasowaniem nie zawierał mnożnika Play ani rezultatu porównania układów, natomiast stan zakończony porównaniem układów zawierał zarówno mnożnik wykonanego zakładu, jak i szczegółowe informacje o ocenionych układach gracza i krupiera oraz wyniku ich porównania.

3.5.2. Obserwacja agenta

Obiekt `UTHObservation` zawiera wyłącznie te dane, które mogą zostać wykorzystane przez agenta podczas podejmowania decyzji. Są to:

- ♠ aktualna faza gry,
- ♠ dwie karty własne gracza,
- ♠ dotychczas odkryte karty wspólne,
- ♠ zbiór legalnych akcji,
- ♠ mnożnik zakładu Play w obserwacji terminalnej.

W ramach tego obiektu umieszczono zbiór legalnych akcji, które agent może podjąć w danej fazie, dzięki czemu z agenta zdjęta jest konieczność odtwarzania zasad gry na podstawie fazy, co ułatwia implementację agentów bazowych.

Obserwacja `UTHObservation`, podobnie jak pełny stan, jest obiektem niemodyfikowalnym. Sprawdzana jest liczba widocznych kart, brak duplikatów oraz zgodność zbioru legalnych akcji z aktualną fazą gry.

3.5.3. Projekcja stanu na obserwację

Obiekt obserwacji dla agenta jest tworzony przez funkcję projekcji.

$$O_t = \phi(S_t), \quad (3.6)$$

gdzie S_t jest pełnym stanem środowiska w chwili t , natomiast O_t oznacza obserwację udostępnioną agentowi.

Dla przyjętego środowiska funkcja projekcji ma postać

$$\phi(S_t) = \left(p_t, C_t^{player}, C_t^{community}, A_t^{legal}, m_t \right), \quad (3.7)$$

gdzie p_t oznacza aktualną fazę gry, C_t^{player} karty własne gracza, $C_t^{community}$ odkryte karty wspólne, A_t^{legal} zbiór legalnych akcji, a m_t mnożnik zakładu Play, jeżeli został już wykonany.

Karty krupiera i karty spalone należą do S_t , natomiast kolejność kart w talii jest przechowywana przez wewnętrzne źródło kart. Żadna z tych informacji nie jest elementem O_t , dokładny zakres pól obu obiektów przedstawiono już w podrozdziale 3.5.1 i 3.5.2.

Ważne jest, że wynik rozdania, rozliczenie i szczegóły porównania układów są po zakończeniu epizodu zwracane przez obiekt `StepResult`, a nie przez samą obserwację. Z racji, że te dane pojawiają się dopiero po wykonaniu ostatniej decyzji, nie mogą wpłynąć na wybór akcji w bieżącym rozdaniu.

3.6. Maszyna stanów i przestrzeń akcji

Przebieg pojedynczego rozdania Ultimate Texas Hold'em odwzorowano jako maszynę stanów. Przejście do następnej fazy jest wyznaczone przez bieżącą fazę i akcję, z kolei zawartość stanu, tym samym obserwacji kolejnego stanu zależy od kart zwróconych przez źródło. Bieżąca faza określa zakres informacji widocznych dla agenta, zbiór legalnych akcji oraz operacje wykonywane przez środowisko po otrzymaniu decyzji.

Przebieg rozdania można przedstawić jako sekwencję faz:

$$\mathcal{P} = \{\text{PREFLOP}, \text{FLOP}, \text{RIVER}, \text{TERMINAL}\}. \quad (3.8)$$

Faza `TERMINAL` jest stanem absorbującym, tzn. nie ma dla niej żadnych legalnych akcji, a rozdanie wchodzi w nią wyłącznie w wyniku spasowania albo wykonania zakładu `Play` zakończonego porównaniem układów.

Metoda `reset()`, wywoływana na początku każdego rozdania, rozdaje po dwie karty graczowi i krupierowi w kolejności P_1, D_1, P_2, D_2 , gdzie P to gracz a D to krupier, tworzy stan `PREFLOP` i zwraca pierwszą obserwację. Karty wspólne i spalone nie są jeszcze dobierane. Dzieje się to w odpowiedniej fazie. Każde poprawne wykonanie metody `step()` przesuwa rozdanie do kolejnej fazy albo do stanu terminalnego. Agent nie może pozostać w tej samej fazie po wykonaniu akcji.

Pełna przestrzeń akcji składa się z sześciu decyzji domenowych:

$$\mathcal{A} = \{\text{CHECK}, \text{BET_4X}, \text{BET_3X}, \text{BET_2X}, \text{BET_1X}, \text{FOLD}\}, \quad (3.9)$$

Maszyna stanów z podziałem na fazy i zbiory legalnych akcji wygląda następująco:

$$\mathcal{A}_{\text{legal}}(p) = \begin{cases} \{\text{CHECK}, \text{BET_3X}, \text{BET_4X}\}, & p = \text{PREFLOP}, \\ \{\text{CHECK}, \text{BET_2X}\}, & p = \text{FLOP}, \\ \{\text{BET_1X}, \text{FOLD}\}, & p = \text{RIVER}, \\ \emptyset, & p = \text{TERMINAL}. \end{cases} \quad (3.10)$$

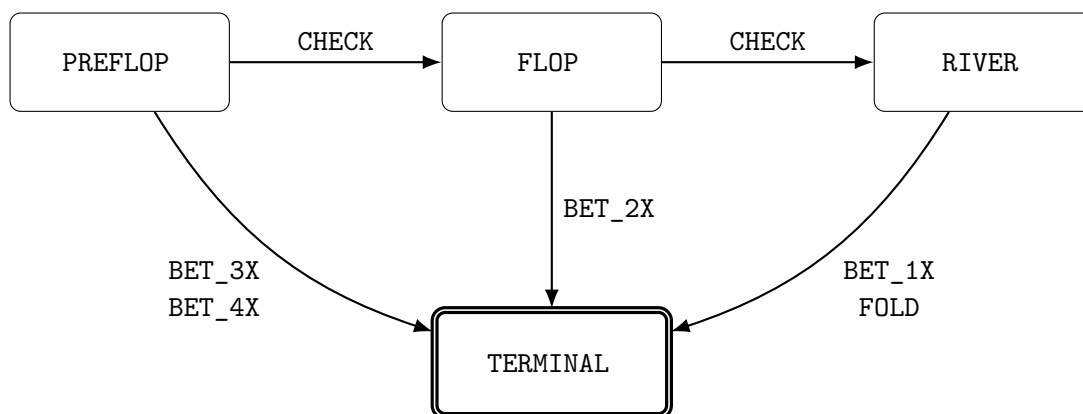
Akcja `CHECK` odkłada decyzję o zakładzie `Play` i przesuwa rozdanie do następnej fazy, spalając kartę oraz odkrywając flop albo turn i river. Każda akcja typu `BET` ustala

mnożnik Play wynikający z fazy: $4\times$ lub $3\times$ przed flopem, $2\times$ po flopie i $1\times$ na riverze.

Po zakładzie Play środowisko odkrywa pozostałe karty wspólne, przeprowadza porównanie układów i przechodzi do stanu terminalnego. Akcja FOLD kończy rozdanie na riverze bez porównania układów.

Struktura maszyny stanów gwarantuje, że zakład Play może zostać wykonany najwyżej raz w trakcie rozdania: każda akcja typu BET prowadzi bezpośrednio do stanu terminalnego, a akcja CHECK jedynie przesuwa decyzję do późniejszej fazy.

Przebieg rozdania zilustrowano schematem przedstawionym na rysunku 3.2.



Rys. 3.2. Maszyna stanów pojedynczego rozdania UTH

3.7. Porównanie układów i rozliczanie zakładów

Porównanie układów (ang. *showdown*) jest fazą końcową każdego rozdania, w którym gracz wykonał zakład Play. Środowisko odkrywa wszystkie brakujące karty wspólne, ocenia najlepszą pięciokartową rękę gracza i krupiera, a następnie porównuje otrzymane wartości. W przypadku kiedy gracz spasuje, nie dochodzi do porównania układów.

3.7.1. Przebieg porównania układów

Podczas porównania układów gracz i krupier dysponują siedmioma kartami. Ewaluator jest odpowiedzialny za wybór najlepszego pięciokartowego układu osobno dla gracza i krupiera. Wynik tej operacji jest przechowywany w obiekcie `ShowdownResult`, który zawiera:

- ♠ najlepszą rękę gracza,
- ♠ najlepszą rękę krupiera,
- ♠ wynik porównania z perspektywy gracza,
- ♠ informację o kwalifikacji krupiera.

Zbiór wszystkich możliwych wyników porównania układów przedstawiono następująco:

$$\mathcal{O}_{showdown} = \{\text{PLAYER_WIN}, \text{DEALER_WIN}, \text{PUSH}\}. \quad (3.11)$$

Konkretny rezultat porównania rąk gracza i krupiera jest wyznaczany przez funkcję O : dla wartości rąk H_p oznaczającej rękę gracza oraz H_d oznaczającej rękę krupiera, wynik porównania układów można zdefiniować jako:

$$O(H_p, H_d) = \begin{cases} \text{PLAYER_WIN}, & H_p > H_d, \\ \text{DEALER_WIN}, & H_p < H_d, \\ \text{PUSH}, & H_p = H_d. \end{cases} \quad (3.12)$$

Wydawać by się mogło, że w momencie kiedy krupier się nie kwalifikuje, nie ma sensu porównywać rąk obu stron. Można jednak rozważyć przykład, w którym każda ze stron ma jedynie wysokie karty (najniższy możliwy układ), ale najwyższa karta krupiera jest wyższa od najwyższej karty gracza. W takim przypadku krupier wygrywa rozdanie, mimo że się nie kwalifikuje. Dlatego porównanie ręki gracza i krupiera jest wykonywane nawet w sytuacji, kiedy krupier się nie kwalifikuje.

3.7.2. Kwalifikacja krupiera

W Ultimate Texas Hold'em krupier kwalifikuje się do wypłaty zakładu Ante, jeżeli posiada co najmniej jedną parę, taki warunek można zapisać w formie równania:

$$Q(H_d) = \begin{cases} 1, & \text{rank}(H_d) \geq \text{ONE_PAIR}, \\ 0, & \text{rank}(H_d) = \text{HIGH_CARD}. \end{cases} \quad (3.13)$$

gdzie $\text{rank}(H_d)$ oznacza rangę układu krupiera.

Samo zjawisko kwalifikacji krupiera nie zmienia wyniku porównania rąk, jednakże wpływa na sposób rozliczenia zakładu Ante, ponieważ:

- ♠ jeżeli krupier kwalifikuje się, zakład Ante wygrywa lub przegrywa zgodnie z wynikiem porównania układów,
- ♠ jeżeli krupier nie kwalifikuje się, zakład Ante jest zwracany niezależnie od tego, która strona ma silniejszą rękę.

Zakład Play jest rozliczany niezależnie od kwalifikacji krupiera, w wyniku bezpośredniego porównania rąk. Z kolei zakład Blind jest rozliczany według układu gracza (w przypadku jego zwycięstwa), przy remisie zwracany, a przy przegranej gracza jest przegrywany.

Wszystkie wyżej opisane zależności przedstawiono dla ułatwienia w tabeli 3.3.

Tabela 3.3. Rozliczenie zakładów po porównaniu układów

Wynik	Kwalifikacja	Ante	Blind	Play
Wygrana gracza	tak	wygrana 1:1	według tabeli wypłat	wygrana 1:1
Wygrana gracza	nie	zwrot	według tabeli wypłat	wygrana 1:1
Remis	dowolna	zwrot	zwrot	zwrot
Wygrana krupiera	tak	przegrana	przegrana	przegrana
Wygrana krupiera	nie	zwrot	przegrana	przegrana

3.7.3. Model rozliczenia zakładów

Zaprojektowano obiekt `WagerSettlement`, który reprezentuje rozliczenie pojedynczego zakładu. Składa się on z `stake`, czyli początkowej wartości postawionego zakładu, oraz `net_profit`, czyli zysku albo straty netto.

Dla pojedynczego zakładu o stawce s i zysku netto n całkowity zwrot brutto wynosi $g = s + n$, jeśli $g = s$ oznacza to zwrot stawki, w przypadku $g = 0$ jej pełną utratę, a stawka $s = 0$ zakład niepostawiony.

Pełne rozliczenie rozdania reprezentuje obiekt `Settlement`, zawierający osobne wyniki dla Ante, Blind oraz Play. Łączny wynik netto S_{net} jest sumą zysków netto wszystkich trzech zakładów, można go zapisać w postaci równania:

$$S_{net} = n_{Ante} + n_{Blind} + n_{Play}. \quad (3.14)$$

Do reprezentacji wartości finansowych wykorzystano typ `Fraction`, ponieważ ten typ pozwala na przechowywanie wypłaty takiej jak $3/2$ (bez stosowania przybliżeń zmienneoprzecinkowych), dzięki czemu unika się błędów zaokrągleń.

3.7.4. Rozliczenie zakładu Blind

Zakład Blind ma stałą stawkę równą jednej jednostce Ante. W sytuacji kiedy wygrywa gracz, zysk z racji tego zakładu jest uzależniony od tego, jak wysoki układ uda się graczowi osiągnąć. Dla układów słabszych od strita Blind jest zwracany. Dla strita lub silniejszego układu stosowana jest tabela wypłat przedstawiona w tabeli 3.4.

Tabela 3.4. Tabela wypłat zakładu Blind

Układ gracza	Wypłata	Zysk netto
Wysoka karta	zwrot	0
Jedna para	zwrot	0
Dwie pary	zwrot	0
Trójka	zwrot	0
Strit	1:1	1
Kolor	3:2	$\frac{3}{2}$
Full	3:1	3
Kareta	10:1	10
Poker	50:1	50
Poker królewski	500:1	500

Zakład Blind jest stosowany wyłącznie wtedy, gdy gracz wygrał porównanie ukła-

dów. Przy remisie zakład jest zwracany, natomiast przy wygranej krupiera pełna stawka Blind zostaje utracona.

3.7.5. Rozliczenie Ante, Play i spasowania

Zakład Ante przy remisie jest zwracany. Jeżeli krupier kwalifikuje się, Ante wygrywa albo przegrywa zgodnie z wynikiem porównania układów. W przypadku braku kwalifikacji zakład jest zwracany.

Zakład Play jest wypłacany 1:1, niezależnie od jego mnożnika. Dla zobrazowania można posłużyć się przykładem: gracz postawił wygrany zakład Play $4\times$, co daje zysk netto równy czterem jednostkom. Przy porażce tracona jest pełna wartość zakładu, natomiast przy remisie cała stawka zostaje zwrócona.

Jeżeli gracz spasuje na riverze, nie dochodzi do porównania układów. Ante i Blind zostają przegrane, natomiast Play nie został postawiony.

Rozliczenie spasowania ma zatem postać

$$S_{fold} = (n_{Ante} = -1, n_{Blind} = -1, n_{Play} = 0) . \quad (3.15)$$

Wtedy łączna stawka wynosi dwie jednostki, a wynik netto jest równy -2 .

W celu rozróżnienia zakładu niepostawionego od zakładu postawionego, ale zwróconego, w rozliczeniu Play zapisywana jest stawka równa zero.

3.8. Funkcja nagrody i wynik epizodu

Aby móc sprawnie zmieniać warianty funkcji nagrody wykorzystywanej przez algorytmy uczenia przez wzmacnianie, oddzielono mechanizm rozliczania zakładów od konstrukcji sygnału nagrody. Silnik domenowy wyznacza dokładny wynik finansowy rozdania zgodny z zasadami Ultimate Texas Hold'em. Natomiast konstruowanie sygnału nagrody na jego podstawie należy do warstwy integrującej silnik z agentem. Dzięki temu możliwe jest proste porównywanie różnych funkcji nagrody, bez ingerencji w zasady gry.

3.9. Interfejs silnika

Silnik domenowy zaimplementowano w taki sposób, aby udostępniał niewielki interfejs publiczny, za pomocą którego można rozpocząć rozdanie, pobrać obserwację, wykonać decyzję oraz odczytać wynik zakończonego epizodu. Interfejs nie zależy od konkretnego algorytmu uczenia, co czyni go dość uniwersalnym rozwiązaniem do wykorzystania przy różnych eksperymentach.

Fundamentalnym elementem jest klasa `UTHGame`. Pojedyncza instancja tej klasy przechowuje stan aktualnego rozdania, źródło kart oraz opcjonalną historię decyzji. Po zakończeniu rozdania ta sama instancja może zostać wykorzystana ponownie przez wywołanie metody `reset()`.

3.9.1. Tworzenie silnika i rozpoczęcie epizodu

Konstruktor klasy `UTHGame` przyjmuje dwa opcjonalne parametry: `seed` oraz `record_history`.

Ziarno (ang. *seed*) określa powtarzalny strumień kart w talii generowanych dla kolejnych rozdawań, nie ustala jednak pojedynczej, niezmienniej talii dla całej instancji. Rejestrowanie historii jest domyślnie wyłączone, służy ono do zbierania dodatkowych obiektów diagnostycznych.

Każde nowe rozdanie rozpoczyna się właśnie wywołaniem metody `reset()`, które zwraca pierwszą obserwację w fazie `PREFLOP`. W standardowym użyciu środowisko samodzielnie tworzy przetasowaną talię. Metoda umożliwia również przekazanie własnego obiektu implementującego interfejs `CardSource`, na przykład deterministycznej talii `FixedDeck`.

Przez rozpoczęcie rozdania rozumie się wybór lub utworzenie źródła kart, rozdanie po dwie karty graczowi i krupierowi, utworzenie stanu `PREFLOP`, wyzerowanie historii bieżącego rozdania oraz utworzenie pierwszej obserwacji. Reset jest wykonywany transakcyjnie, silnik najpierw spróbuje pobrać wszystkie karty potrzebne do utworzenia stanu początkowego, a dopiero po pozytywnym zakończeniu tego etapu zapisuje nowy stan i źródło kart. Jeżeli przekazane źródło nie może dostarczyć wymaganych kart, poprzedni stan silnika nie zostaje zastąpiony stanem częściowo utworzonego rozdania.

3.9.2. Wykonanie decyzji

Metodę `step()` zaprojektowano tak, aby przyjmowała jedną wartość typu `Action`, wykonała jej odpowiadające przejście i zwracała obiekt `StepResult`, zawierający:

- ♠ obserwację po wykonaniu akcji,
- ♠ wynik rozdania (jeśli osiągnięto stan terminalny),
- ♠ rozliczenie zakładów w stanie terminalnym,
- ♠ szczegóły porównania układów (jeśli gracz nie spasował).

Wynik metody `step()` kroku pośredniego zawiera jedynie kolejną obserwację. Pola dotyczące wyniku i rozliczenia pozostają wtedy nieokreślone. W kroku terminalnym obiekt zawiera informacje niezbędne do obliczenia nagrody i zarejestrowania wyniku eksperymentu.

Właściwość `terminated` obiektu `StepResult` informuje, czy wykonana akcja zakończyła rozdanie. Jej wartość jest wyznaczana na podstawie fazy, która wchodzi w skład obserwacji.

Choć zbiór legalnych akcji jest wyznaczany przez środowisko i umieszczany w obserwacji, metoda `step()` nie ufa bezkrytycznie przekazanej decyzji. Każda akcja jest wa-

lidowana ponownie przed jakąkolwiek zmianą stanu. Walidacja następuje przed dobieraniem kart, więc odrzucona decyzja nie zużywa kart z talii, nie zmienia fazy i nie jest dodawana do historii.

3.9.3. Właściwości publiczne

Poza metodami `reset()` i `step()` silnik udostępnia właściwości przedstawione w tabeli 3.5.

Tabela 3.5. Publiczny interfejs klasy `UTHGame`

Element	Typ wyniku	Odpowiedzialność
<code>reset()</code>	<code>UTHObservation</code>	Rozpoczyna nowe rozdanie i zwraca obserwację preflop.
<code>step(action)</code>	<code>StepResult</code>	Wykonuje legalną akcję i zwraca wynik przejścia.
<code>observation</code>	<code>UTHObservation</code>	Zwraca aktualne informacje widoczne dla agenta.
<code>state</code>	<code>RoundState</code>	Zwraca pełny stan wewnętrzny, przeznaczony głównie do testów i diagnostyki.
<code>is_started</code>	<code>bool</code>	Informuje, czy rozpoczęto co najmniej jedno rozdanie.
<code>history_enabled</code>	<code>bool</code>	Informuje, czy rejestrowanie historii jest aktywne.
<code>trace</code>	<code>RoundTrace</code> lub <code>None</code>	Zwraca zapis bieżącego rozdania, jeżeli historia jest włączona.

Właściwość `state` nie jest częścią interfejsu przeznaczonego do podejmowania decyzji przez agenta. Udostępniono ją jedynie do celów testowych i diagnostycznych. Agent otrzymuje wartość właściwości `observation` albo obserwację zwróconą przez `reset()` lub `step()`.

3.10. Rejestrowanie przebiegu i walidacja środowiska

Bardzo ważnym elementem projektu środowiska jest pokrycie jego implementacji testami, ponieważ potencjalne niedopatrzenia na etapie implementacji, skrajne przypadki, błędy w logice bezpośrednio rzutują na wyniki eksperymentów, co może doprowadzić do nieefektywnego uczenia się agentów. Aby zapobiec takiemu scenariuszowi, poświęcono dużą uwagę testom jednostkowym i integracyjnym, wprowadzono opcjonalny mechanizm rejestrowania przebiegu rozdania oraz wzbogacono testy o weryfikację zarówno całych komponentów, jak i ścieżek rozgrywki.

3.10.1. Rejestrowanie historii i jej rekordy

Rejestrowanie historii jest aktywowane parametrem `record_history` przekazywanym podczas tworzenia obiektu `UTHGame`. Domyślnie mechanizm ten pozostaje wyłączony, aby podczas uczenia nie generować dodatkowych obiektów diagnostycznych, które nie są potrzebne agentowi i zajmują pamięć oraz czas. Po jego włączeniu każde poprawnie rozpoczęte rozdanie otrzymuje dodatni, rosnący identyfikator `round_id`, a lista

decyzji jest zerowana przy każdym poprawnym wywołaniu metody `reset()`. Właściwość `trace` zwraca obiekt `RoundTrace`. Jeżeli historia jest wyłączona albo nie rozpoczęto jeszcze rozdania, jej wartością jest `None`. Mechanizm ten nie stanowi pamięci agenta i nie jest częścią obserwacji. Wykorzystywany jest do odtwarzania przebiegu wybranych rozdań w celu weryfikacji zgodności decyzji agenta z legalną przestrzenią akcji.

Niemutowalny obiekt `DecisionRecord` przechowuje pojedynczą decyzję agenta: obserwację dostępną agentowi przed wykonaniem decyzji oraz wybraną akcję. Argumentem za zapisywaniem obserwacji sprzed przejścia jest chęć zachowania informacji, dla jakiej obserwacji agent podjął konkretną decyzję. Zapisywanie obserwacji po wykonaniu akcji byłoby niejednoznaczne, ponieważ taki stan zawierałby informacje już odkrytych kart, które mogłyby być skutkiem wykonanej akcji. Podczas tworzenia rekordu sprawdzane jest, czy akcja należy do zbioru legalnego dla tej obserwacji.

Cały przebieg rozdania opisuje niemodyfikowalny obiekt `RoundTrace`, który zawiera identyfikator rozdania, uporządkowaną krotkę rekordów decyzji oraz aktualny pełny stan `RoundState`. Obserwacje w rekordach zawierają wyłącznie informacje dostępne agentowi, natomiast końcowy stan jest pełnym obiektem diagnostycznym, z tego powodu `RoundTrace` nie jest przekazywany agentowi, lecz służy testom.

Zapis historii jest transakcyjny. Metoda `step()` zapamiętuje bieżącą obserwację, wykonuje walidację i przejście, a rekord dodaje dopiero po utworzeniu poprawnego kolejnego stanu. Jeżeli walidacja lub odkrywanie kart zakończy się wyjątkiem, rekord nie powstaje, więc historia nie zawiera akcji nielegalnych, decyzji po zakończeniu rozdania ani niedokończonych przejść. Podobny cel przyświecał metodzie `reset()`, która zapisuje nowy stan dopiero po rozdaniu wszystkich czterech kart początkowych.

3.10.2. Poziomy testowania

Testy są kluczowe, dlatego uporządkowano je według odpowiedzialności komponentów. Testy jednostkowe sprawdzają w izolacji model kart i talię, ewaluator, modele stanu, rozdawanie, legalność akcji, porównanie układów oraz rozliczenia. Testy integracyjne z kolei wykonują kompletne rozdania obejmujące wszystkie ścieżki maszyny stanów, od `reset()` do stanu terminalnego, wraz z rejestrowaniem historii.

Testy weryfikują również między innymi brak powtarzających się kart, zgodność liczby kart wspólnych i spalonych z fazą, brak danych terminalnych w stanie pośrednim, brak porównania układów po spasowaniu, wykonanie co najwyżej jednego zakładu `Play`, brak kart krupiera i kart spalonych w obserwacji oraz brak zmiany stanu po odrzuconej akcji. Walidacja obejmuje także przypadki brzegowe i próby utworzenia stanów naruszających reguły domenowe.

Do realizacji testów wykorzystano bibliotekę `pytest`. Podczas prac nad silnikiem analizowano na bieżąco zarówno pokrycie instrukcji, jak i pokrycie rozgałęzień. Pozwoliło to wykryć nieprzetestowane ścieżki walidacji oraz przypadki brzegowe ewaluatora.

Kod źródłowy środowiska, agentów i skryptów eksperymentalnych udostępniono w repozytorium <https://github.com/Meikelele/expert-poker-player>.

3.11. Ograniczenia środowiska

Choć zaprojektowany i zaimplementowany model środowiska Ultimate Texas Hold'em jest wystarczający do przeprowadzenia badań nad strategiami podejmowania decyzji, to nie odwzorowuje on wszystkich aspektów gry w kasynie.

Prawdą jest, że zakłady Trips i Progressive nie wprowadzają dodatkowego momentu decyzyjnego, ponieważ ich wynik nie zależy od decyzji związanej z akcją Play. Niemniej zakłady poboczne wpływają na rozkład końcowych wypłat i utrudniałyby interpretację jakości strategii, a Progressive wymagałby dodatkowo modelu puli utrzymywanej pomiędzy rozdaniami, co naruszałoby niezależność epizodów. Oba zakłady poboczne można zaimplementować jako rozszerzenie warstwy rozliczeń, co może stanowić przedmiot dalszych badań.

Silnik nie modeluje również salda gracza, sesji, ryzyka bankructwa ani zarządzania kapitałem, a wartości zakładów wyraża w jednostkach Ante, dzięki czemu strategię można porównywać niezależnie od nominału.

Środowisko zaprojektowano tak, że reprezentuje ono pojedynczego gracza wobec krupiera i nie symuluje pozostałych miejsc przy stole, a zakryte karty innych graczy i tak byłyby nieznane, więc rozkład kart widocznych dla gracza odpowiadałby losowaniu bez zwracania ze standardowej talii.

Obserwacja jest obiektem domenowym zawierającym karty, fazę i zbiór legalnych akcji. W tej postaci jest czytelna i wygodna w testach, lecz nie może być bezpośrednio podana modelom uczenia maszynowego. Wynika to z tego, że kodowanie `UTHObservation` do reprezentacji numerycznej pozostaje poza zakresem klasy `UTHGame`, ponieważ porównanie reprezentacji stanu jest jednym z elementów badawczych pracy. Wspólny silnik jest wygodny, bo pozwala zestawiać różne reprezentacje na identycznych rozdaniach i regułach.

3.12. Podsumowanie

W niniejszym rozdziale przedstawiono projekt oraz implementację pojedynczego rozdania Ultimate Texas Hold'em. Silnik podzielono na niezależne moduły odpowiedzialne za reprezentację kart, ocenę układów, rozdawanie, przechowywanie stanu, walidację akcji, porównanie układów oraz rozliczanie zakładów, a przebieg rozdania odwzorowano jako maszynę stanów obejmującą fazy PREFLOP, FLOP, RIVER i TERMINAL, w której każda akcja BET kończy część decyzyjną i wymusza co najwyżej jeden zakład Play.

Ważnym elementem projektu jest rozdzielenie pełnego stanu `RoundState` od obserwacji `UTHObservation`, mianowicie agent otrzymuje wyłącznie własne karty, odkryte karty wspólne, fazę i legalne akcje, podczas gdy karty krupiera, karty spalone i kolej-

ność talii pozostają wewnętrzne. Porównanie układów korzysta z ogólnego ewaluatora wybierającego najlepsze pięć kart z siedmiu, a rozliczenia Ante, Blind i Play są przechowywane dokładnie za pomocą typu `Fraction`. Deterministyczne źródło `FixedDeck` oraz opcjonalny mechanizm historii zapewniają powtarzalność i możliwość analizy przebiegu rozdań.

Na tym etapie powstał silnik gry, który zapewnia podstawę do implementacji agentów bazowych, infrastruktury eksperymentalnej oraz metod uczenia przez wzmacnianie analizowanych w dalszej części pracy.

4. AGENCI BAZOWI I INFRASTRUKTURA EKSPERYMENTALNA

W tym rozdziale przedstawiono projekt agentów bazowych oraz infrastruktury służącej do uruchamiania i oceny strategii w środowisku Ultimate Texas Hold'em. Rolę agentów bazowych jako punktów odniesienia dla metod uczenia przez wzmacnianie opisano w rozdziale 2.

Zaprojektowano i zaimplementowano wspólny kontrakt agenta, mechanizm pojedynczego epizodu i symulację wielu rozdań, agenta losowego oraz agenta regułowego. Infrastrukturę uzupełniono o mechanizm zbierania metryk i zapis wyników eksperymentów. Wszystkie te elementy korzystają z publicznego interfejsu środowiska, opisanego w rozdziale 3.

4.1. Wspólny interfejs agenta

Zdefiniowano wspólny kontrakt Agent, który określa, w jaki sposób agent komunikuje się ze środowiskiem. Kontrakt ten udostępnia metodę `select_action()`, która przyjmuje obiekt `UTHObservation` i zwraca podjętą przez agenta decyzję typu `Action`.

Wykorzystanie protokołu do konstrukcji kontraktu pozwala zdefiniować agenta w sposób elastyczny. Klasa agenta nie musi dziedziczyć po wspólnej klasie bazowej, lecz musi udostępniać metodę o wymaganej sygnaturze. Dzięki temu ten sam mechanizm prowadzenia rozgrywki można wykorzystać dla agenta losowego, regułowego i uczącego się przez wzmacnianie bez znajomości ich wewnętrznej implementacji.

4.2. Prowadzenie pojedynczego epizodu

W celu sprawnego przeprowadzenia eksperymentu zaimplementowano osobny moduł, nazwany runnerem. Zdefiniowano w nim funkcję `play_round()`, która łączy agenta z silnikiem `UTHGame`. Funkcja ta rozpoczyna rozdanie, przekazuje bieżącą obserwację agentowi, wykonuje wybraną akcję i powtarza ten proces do osiągnięcia stanu terminalnego.

Każda decyzja zaakceptowana przez środowisko zostaje zapisana w kolejności wykonania. Po zakończeniu rozdania funkcja zwraca obiekt `EpisodeResult`, który zawiera sekwencję akcji, wynik rozdania oraz pełne rozliczenie zakładów Ante, Blind i Play.

Model `EpisodeResult` udostępnia również wynik netto, łączną wartość postawionych zakładów, liczbę decyzji i mnożnik zakładu Play. Stanowi w ten sposób lekką reprezentację zakończonego epizodu.

4.3. Agent losowy

Najprostszą strategię bazową stanowi agent losowy. Nie analizuje on wartości kart ani prawdopodobieństwa wygranej, a dla każdej obserwacji wybiera jedną z legalnych ak-

cji z jednakowym prawdopodobieństwem. W obserwacji `UTHObservation` agent dostaje zbiór legalnych akcji do wykonania. Akcje te są porządkowane zgodnie z kolejnością wartości enuma `Action`, przez co wynik generatora pseudolosowego nie zależy od kolejności iterowania po zbiorze.

Środowisko i agent posiadają niezależne generatory pseudolosowe. Generator środowiska odpowiada za kolejność kart, natomiast generator agenta odpowiada wyłącznie za wybór akcji. Rozdzielenie źródeł losowości daje możliwość zmiany strategii agenta, na przykład jego ziarna albo całej implementacji, bez zmiany przebiegu rozdań. Dzięki temu, jak pokazano w sekcji 4.7, można porównywać agenta losowego i agenta regułowego na dokładnie tym samym harmonogramie talii.

Testy obejmują zgodność ze wspólnym protokołem, wybieranie wyłącznie legalnych akcji, powtarzalność decyzji dla tego samego ziarna oraz obsługę niepoprawnych danych wejściowych. Jest również test potwierdzający, że agent nie podejmuje decyzji dla obserwacji terminalnej. Testy samego runnera opisano w sekcji 4.9.

4.4. Agent regułowy

Agent losowy jest podstawowym punktem odniesienia, lecz nie korzysta w ogóle z wiedzy o grze. Dlatego w celu uzyskania lepszego punktu odniesienia zdefiniowano agenta regułowego, który w przeciwieństwie do agenta losowego podejmuje decyzje na podstawie kart widocznych w obserwacji oraz aktualnej fazy rozdania. Przyjęte reguły oparto na uproszczonej strategii Ultimate Texas Hold'em opublikowanej przez Wizard of Odds [3].

4.4.1. Reguły decyzyjne

Zanim zostaną odkryte karty wspólne, agent podejmuje decyzję o zakładzie Play w wysokości czterokrotności Ante dla układów początkowych przedstawionych w tabeli 4.1.

Tabela 4.1. Układy początkowe uprawniające do zakładu Play $4\times$ przed flopem

Układ początkowy	Warunek
Para	od trójek wzwyż
As	z dowolną kartą towarzyszącą
Król	z dowolną kartą w tym samym kolorze
Król	z kartą pięć lub wyższą w różnych kolorach
Dama	z kartą sześć lub wyższą w tym samym kolorze
Dama	z kartą osiem lub wyższą w różnych kolorach
Walet	z kartą osiem lub wyższą w tym samym kolorze
Walet	z dziesiątką w różnych kolorach

Dla pary dwójek oraz wszystkich pozostałych układów agent wykonuje akcję CHECK. Ta strategia nie wykorzystuje zakładu `BET_3X`.

Jeżeli agent nie wykonał zakładu przed flopem, po odkryciu trzech kart wspólnych wybiera `BET_2X`, gdy posiada dwie pary lub silniejszy układ, ukrytą parę z wyjątkiem

pary dwójek albo cztery karty do koloru z własną kartą tego koloru o randze co najmniej dziesięć.

Przez ukrytą parę rozumie się parę wykorzystującą co najmniej jedną z dwóch prywatnych kart agenta. Jeżeli żaden z tych warunków nie jest spełniony, agent wykonuje akcję CHECK.

W ostatnim punkcie decyzyjnym agent wykonuje zakład BET_1X, jeżeli posiada ukrytą parę lub silniejszy układ. Dla słabszych układów wykorzystywana jest reguła oparta na liczbie kart mogących poprawić układ krupiera (ang. *outs*) do układu pokonującego gracza. Jeżeli liczba takich kart jest mniejsza niż 21, agent wykonuje BET_1X. W przeciwnym przypadku wybiera FOLD [3].

Jest to strategia deterministyczna, czyli dla tej samej obserwacji agent zawsze zwraca tę samą akcję. Nie jest to jednak strategia optymalna, ponieważ opiera się na uproszczonych heurystykach szerzej opisanych przez Wizard of Odds, a nie na pełnym przeszukaniu przestrzeni decyzji.

4.4.2. Implementacja strategii regułowej

W celu implementacji agenta regułowego zdefiniowano klasę RuleBasedAgent, która implementuje interfejs agenta pokerowego. Reguły decyzyjne oddzielono od klasy agenta, co pozwala na ich łatwe modyfikowanie w razie potrzeby.

Za ocenę obserwacji w poszczególnych fazach rozdania odpowiadają funkcje `should_raise_preflop()`, `should_raise_flop()` oraz `should_raise_river()`.

Elementem wspomagającym decyzję na riverze jest funkcja `count_dealer_outs()`. Tworzy ona zbiór kart niewidocznych dla gracza, a następnie każdą z nich analizuje jako pojedynczą potencjalną kartę prywatną krupiera. Jeżeli dołączenie danej karty do kart wspólnych prowadzi do układu silniejszego od najlepszego układu agenta, karta zostaje zaklasyfikowana jako out.

4.5. Symulacja wielu epizodów i jej powtarzalność

Aby sprawnie przeprowadzać eksperymenty, składające się z wielu rozdań, zdefiniowano obiekty `SimulationConfig` i `SimulationResult`.

Konfiguracja symulacji przechowuje uporządkowaną sekwencję ziaren wykorzystywanych do tworzenia talii. Liczba ziaren określa jednocześnie liczbę rozgrywanych epizodów.

Funkcja `run_simulation()` dla każdego ziarna tworzy osobną talię oraz nową instancję środowiska `UTHGame`. Następnie wykorzystuje wspomnianą funkcję `play_round()` do rozegrania pełnego epizodu. Wyniki wszystkich rozdań są przechowywane w obiekcie `SimulationResult`.

W taki sposób oddzielono konfigurację eksperymentu od implementacji konkretnego agenta. Ten sam obiekt `SimulationConfig` może zostać wykorzystany do oceny

wielu różnych strategii.

Przy projektowaniu infrastruktury eksperymentalnej przyjęto cel łatwego odtwarzania przeprowadzonych symulacji. Dla każdego epizodu zapisywane jest ziarno talii, dzięki czemu ponowne utworzenie talii z tym samym ziarnem daje tę samą kolejność kart. Jest to istotne, ponieważ przy porównywaniu agentów między sobą można mieć pewność, że różnice w wynikach wynikają z ich strategii, a nie z losowego układu kart.

4.6. Metryki oceny

Do porównania skuteczności agentów między sobą potrzebne są metryki umożliwiające ocenę jakości strategii. Miary zdefiniowane poniżej nie są specyficzne dla agentów bazowych, ponieważ wykorzystywane są również do oceny agentów uczących się w kolejnych rozdziałach pracy. Za podstawową miarę jakości strategii uznano wynik netto wyrażony w jednostkach zakładu Ante. I tak dla symulacji składającej się z N epizodów empiryczną wartość oczekiwaną oszacowano jako średni wynik netto na jedno rozdanie:

$$\widehat{EV} = \frac{1}{N} \sum_{i=1}^N r_i, \quad (4.1)$$

gdzie r_i oznacza wynik netto i -tego epizodu wyrażony w jednostkach Ante.

Oprócz średniego wyniku obliczany jest również całkowity wynik netto oraz średnia i całkowita wartość postawionych zakładów. Zmienność wyników oszacowano za pomocą odchylenia standardowego próby:

$$s = \sqrt{\frac{\sum_{i=1}^N (r_i - \bar{r})^2}{N - 1}}. \quad (4.2)$$

Na podstawie odchylenia standardowego próby wyznaczono błąd standardowy średniej:

$$SE = \frac{s}{\sqrt{N}}. \quad (4.3)$$

Poza tymi miarami, do oceny jakości strategii dołączane są również końcowe wyniki rozdań oraz częstotliwość wykonywania poszczególnych akcji [10].

4.7. Porównanie agentów bazowych

W celu zweryfikowania procesu eksperymentalnego, porównano agenta losowego i agenta regułowego. Każdy z tych agentów rozegrał 10 000 rozdań. Ziarno generatora budującego harmonogram ziaren talii wynosiło 20260815, natomiast generator decyzji agenta losowego zainicjalizowano ziarnem 20260816. Sam obiekt UTHGame nie otrzymuje własnego ziarna, powtarzalność zapewnia wyłącznie jawnie przekazana talia Deck (seed). Uzyskane podstawowe metryki przedstawiono w tabeli 4.2.

Wynika z tego, że agent losowy uzyskał średni wynik $-0,46435$ jednostki Ante na

Tabela 4.2. Wyniki agentów bazowych dla 10 000 rozdań

Agent	$\widehat{EV}$	s	SE	Śr. stawka	Wynik netto
RandomAgent	-0,46435	4,46422	0,04464	4,7395	-4643,5
RuleBasedAgent	0,03865	4,08130	0,04081	4,1905	386,5

rozdanie, a agent regułowy osiągnął w tej samej próbie średni wynik 0,03865, czyli wynik wyższy o około 0,503 jednostki Ante na rozdanie. Jednocześnie średnia wartość środków stawianych przez agenta regułowego była niższa.

Pomimo tego, że agent regułowy uzyskał dodatni wynik w tej konkretnej próbie, nie stanowi to potwierdzenia dodatniej rzeczywistej wartości oczekiwanej strategii. Błąd standardowy jego średniego wyniku wyniósł około 0,04081, czyli wartość zbliżoną do samego oszacowania $\widehat{EV}$. Ten wynik eksperymentu służy jedynie jako punkt odniesienia dla późniejszej oceny agentów uczących się.

W celu zobrazowania wyników końcowych agentów bazowych przedstawiono w tabeli 4.3 rozkład wyników rozdań.

Tabela 4.3. Rozkład wyników agentów bazowych dla 10 000 rozdań

Agent	Wygrana	Wygrana krupiera	Spasowanie	Remis
RandomAgent	45,01%	42,75%	8,49%	3,75%
RuleBasedAgent	47,92%	31,75%	16,79%	3,54%

Agent regułowy częściej kończył rozdania spasowaniem, lecz znacznie rzadziej doprowadzał do rozstrzygnięcia zakończonego wygraną krupiera.

4.8. Zapis wyników eksperymentu

Wyniki eksperymentu zapisano w dwóch formatach: JSON i CSV. Plik JSON zawiera konfigurację eksperymentu oraz zagregowane metryki obu agentów. Plik CSV przechowuje dane poszczególnych epizodów, w tym indeks rozdania, ziarno talii, nazwę agenta, wynik netto, całkowitą stawkę, wynik końcowy i sekwencję wykonanych akcji.

Taki podział na dwa osobne pliki pozwala zestawiać wyniki obu agentów dla tych samych ziaren talii oraz dosyć łatwo tworzyć wizualizacje.

4.9. Walidacja infrastruktury eksperymentalnej

Testami pokryto zarówno scenariusze pojedynczych epizodów, jak i symulacje wielu rozdań. Na poziomie pojedynczego epizodu testy runnera objęły wszystkie legalne ścieżki zakończenia rozdania. Na poziomie symulacji sprawdzono zgodność liczby epizodów z konfiguracją, powtarzalność wyników dla tych samych ziaren oraz wykorzystanie tych samych talii przez porównywanych agentów.

Testy agregacji metryk weryfikują obliczanie wyniku całkowitego, średniego wyniku, wartości postawionych zakładów, odchylenia standardowego, błędu standardowego oraz liczników wyników i akcji. Osobne testy pokrywają zapis wyników eksperymentu.

4.10. Podsumowanie

Zdefiniowano dwa punkty odniesienia dla dalszych eksperymentów. `RandomAgent` reprezentuje strategię pozbawioną wiedzy o wartości kart, natomiast `RuleBasedAgent` wykorzystuje deterministyczny zestaw reguł.

Przygotowano infrastrukturę, która umożliwia prowadzenie powtarzalnych symulacji, agregowanie wyników i zapisywanie danych poszczególnych epizodów. Ta infrastruktura stanowi podstawę do porównywania agentów wykorzystujących metody uczenia przez wzmacnianie z ustalonymi strategiami bazowymi w kolejnych etapach pracy.

5. METODY UCZENIA PRZEZ WZMACNIANIE

W tym rozdziale przedstawiono elementy uczenia przez wzmocnienie, wykorzystywane bezpośrednio w eksperymentach. Opisano w nim, w jaki sposób przekształcane są obserwacje do reprezentacji numerycznej oraz definiowany jest sygnał nagrody.

Dalej w rozdziale opisano najpierw dwa elementy wspólne dla obu zastosowanych algorytmów, czyli sposób kodowania stanu oraz funkcję nagrody, a następnie same algorytmy, mianowicie Deep Q-Network oraz Policy Gradient oparty na metodzie REINFORCE. Rozdział kończy się krótkim zestawieniem różnic pomiędzy tymi dwoma podejściami.

5.1. Reprezentacja stanu

Stan definiowany jest zgodnie z przyjętą w uczeniu przez wzmocnienie konwencją, jako opis aktualnej sytuacji agenta w środowisku [11].

W pracy zastosowano dwa warianty funkcji stanu, nazwane RAW i FEATURES. Obie funkcje powstają wyłącznie na podstawie obiektu `UTHObservation`, a więc informacji dostępnych graczowi.

5.1.1. RAW: reprezentacja surowa

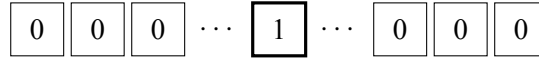
Pierwszy wariant, określony jako RAW, stanowi surowy wektor reprezentujący karty. Każda karta jest reprezentowana jako wektor one-hot (ang. *one-hot*) [12] o długości 52.

Kodowanie jednej karty jako one-hot, a nie jako pojedynczej wartości porządkowej, wynika z tego, co właściwie jest kodowane. Sama ranga karty ma charakter porządkowy i w tej postaci wykorzystywana jest w cechach domenowych FEATURES opisanych w podrozdziale 5.1.2. Tutaj jednak kodowana jest cała karta, czyli konkretna kombinacja rangi i koloru, a kolor nie ma w pokerze żadnej naturalnej hierarchii. Przypisanie 52 kartom pojedynczych, kolejnych numerów mogłoby sugerować sieci nieistniejącą zależność, na przykład że dwie karty o bliskich numerach wykazują nieuzasadnioną bliskość. Kodowanie one-hot tego problemu nie ma, ponieważ każda karta dostaje własną, niezależną pozycję w wektorze.

Kodowanie one-hot polega na tym, że każda wartość ze zbioru możliwych wartości reprezentowana jest jako wektor samych zer z pojedynczą jedynką na pozycji odpowiadającej tej konkretnej wartości [12]. A zatem każda karta to 52 liczby, z czego 51 to zera. Jedynek stoi na pozycji przypisanej dokładnie tej karcie (np. as pik), jak przedstawiono na rysunku 5.1. Agentowi dostarczana jest w ten sposób informacja o obecności karty. Nieodkryte karty to wektory zerowe.

Do reprezentacji stanu dołączono zakodowaną aktualną fazę rozdania oraz maskę

52 elementy (jedna karta)



jedynka na pozycji
konkretnej karty (np. As pik)

Rys. 5.1. Kodowanie one-hot pojedynczej karty

sześciu możliwych akcji, dzięki której sieć neuronowa ma bezpośredni dostęp do informacji o tym, które akcje są w danym momencie możliwe do wykonania. Łączny wymiar RAW wynosi

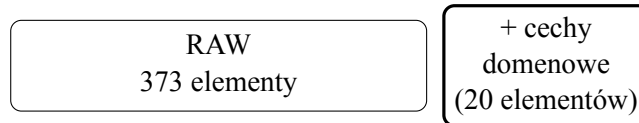
$$d_A = 7 \cdot 52 + 3 + 6 = 373. \quad (5.1)$$

Przyjęto założenie, że RAW nie będzie zawierał informacji o sile układu pokrowego. Model musi więc samodzielnie nauczyć się korelacji pomiędzy kartami a ich wartością w pokerze.

Kolejność kart w wektorze RAW odpowiada kolejności, w jakiej trafiają one do obiektu `UTHObservation`, a nie ich rangom. Kolejność ta nie jest kanonizowana, więc ta sama para kart własnych może zostać zakodowana na dwa różne sposoby w zależności od tego, która z kart została rozdana jako pierwsza.

5.1.2. FEATURES: reprezentacja z cechami domenowymi

Drugi wariant, nazwany FEATURES, jest rozszerzeniem surowego wariantu opisanego w podrozdziale 5.1.1 o 20 dodatkowych cech domenowych, przedstawionym na rysunku 5.2.



Rys. 5.2. FEATURES jako RAW rozszerzone o 20 cech domenowych

Oprócz informacji o kartach, fazie rozdania i dostępnych akcjach, do reprezentacji tego wariantu dołączono informacje o właściwościach kart prywatnych, o kategorii najlepszego widocznego układu oraz o strukturze kart, między innymi o parach, zgodności kolorów i możliwości utworzenia strita. Wszystkie 20 cech domenowych wraz z zakresem wartości i normalizacją przedstawiono w tabeli 5.1.

Wymiar drugiego wariantu wynosi

$$d_B = 373 + 20 = 393. \quad (5.2)$$

Tabela 5.1. 20 cech domenowych reprezentacji FEATURES

Cecha	Zakres	Norm.
Ranga silniejszej karty własnej	$[0, 1]$	tak
Ranga słabszej karty własnej	$[0, 1]$	tak
Para kieszeniowa	$\{0, 1\}$	–
Zgodność kolorów kart własnych	$\{0, 1\}$	–
Odstęp rang kart własnych	$[0, 1]$	tak
Kategoria układu: wysoka karta	$\{0, 1\}$	–
Kategoria układu: para	$\{0, 1\}$	–
Kategoria układu: dwie pary	$\{0, 1\}$	–
Kategoria układu: trójka	$\{0, 1\}$	–
Kategoria układu: strit	$\{0, 1\}$	–
Kategoria układu: kolor	$\{0, 1\}$	–
Kategoria układu: full	$\{0, 1\}$	–
Kategoria układu: kareta	$\{0, 1\}$	–
Kategoria układu: poker	$\{0, 1\}$	–
Liczba par wśród widocznych kart	$\{0, \frac{1}{3}, \frac{2}{3}, 1\}$	tak
Liczba trójek wśród widocznych kart	$\{0, \frac{1}{2}, 1\}$	tak
Liczba karet wśród widocznych kart	$\{0, 1\}$	tak
Postęp koloru	$[0, 1]$	tak
Postęp strita	$[0, 1]$	tak
Sparowany stół	$\{0, 1\}$	–

Porównanie obu reprezentacji pozwala ocenić, czy przekazanie modelowi cech pokerowych ułatwia proces uczenia w porównaniu z reprezentacją opartą głównie na surowym kodowaniu kart.

5.2. Funkcja nagrody

Funkcja nagrody rozumiana jest jako liczbowa ocena jakości decyzji, zwracana agentowi przez środowisko po wykonaniu przez niego działania w danym stanie [13].

W obu wariantach zastosowano nagrodę terminalną, przyznawaną dopiero na końcu rozdania. Kroki niekończące rozdania zwracają wartość zero, natomiast po zakończeniu epizodu nagroda wynika bezpośrednio z końcowego rozliczenia finansowego.

Oznacza to, że pojedyncza decyzja podjęta na wczesnym etapie (na przykład przed flopem) nie otrzymuje żadnej bezpośredniej informacji zwrotnej. Algorytm musi zatem rozwiązać problem przypisania zasługi (ang. *credit assignment problem*), czyli ocenić, w jakim stopniu ta wczesna decyzja przyczyniła się do ostatecznego wyniku [14]. Ważną rolę pełni tu współczynnik dyskontowania γ , który omówiono przy opisie obu zastosowanych algorytmów.

Pierwszy wariant to nagroda oparta na zysku netto, oznaczana dalej jako R_A . Wykorzystuje ona zysk netto gracza wyrażony w jednostkach stawki Ante:

$$R_A = \begin{cases} 0, & \text{dla kroku nieterminalnego,} \\ P_{\text{net}}, & \text{dla kroku terminalnego.} \end{cases} \quad (5.3)$$

gdzie P_{net} oznacza zysk netto gracza, czyli sumę wypłat z zakładów Ante, Blind i Play pomniejszoną o wniesione stawki.

Drugi wariant, nagroda skalowana (R_B), to liniowe przeskalowanie nagrody opartej na zysku netto tak, aby standardowa maksymalna nagroda wynosiła jeden:

$$R_B = \frac{R_A}{6}. \quad (5.4)$$

Wartość sześć odpowiada maksymalnej standardowej sumie stawek Ante, Blind i Play w pojedynczym rozdaniu.

Warto podkreślić, że zastosowane przeskalowanie nie jest obcinaniem nagród (ang. *reward clipping*). W przeciwieństwie do clippingu, który sztywno ogranicza wartości nagród do stałego przedziału (np. $[-1, 1]$), liniowe skalowanie zachowuje pełne proporcje sygnału. Dzięki temu przy bardzo wysokich wypłatach z zakładu Blind nagroda R_B nadal może przekroczyć wartość jeden, nie tracąc informacji o skali wygranej [15].

Dodatnie liniowe przeskalowanie nagrody zachowuje optymalną politykę, choć może zmieniać właściwości numeryczne samego procesu optymalizacji [16, 17].

Porównanie obu podejść umożliwia zbadanie wpływu samej skali nagrody na stabilność i szybkość procesu uczenia.

5.3. Deep Q-Network

Pierwszym algorytmem uczenia przez wzmocnianie zastosowanym w tej pracy jest Deep Q-Network, w skrócie DQN. Jest to jeden z najbardziej znanych algorytmów tego typu, więc zanim opisane zostaną szczegóły implementacji, warto krótko wyjaśnić, na czym w ogóle polega.

DQN uczy się oceniać, jak dobra jest dana akcja w danym stanie gry, przypisując jej liczbę zwaną wartością Q , zgodnie z klasycznym algorytmem Q-learning [18, 19]. Im wyższa wartość Q danej akcji, tym lepszej decyzji ona według agenta odpowiada. W klasycznym podejściu tablicowym taką wartość przechowuje się osobno dla każdej możliwej pary stanu i akcji, w osobnej komórce tabeli. W Ultimate Texas Hold'em liczba samych kombinacji kart widocznych graczowi sięga dziesiątek milionów już po flopie, a po odkryciu kolejnych kart wspólnych jest jeszcze większa, więc liczba możliwych stanów jest bardzo duża i takie podejście tablicowe byłoby niewykonalne. Zamiast niej DQN wykorzystuje sieć neuronową (ang. *neural network*), która nie zapamiętuje wartości Q dla każdego stanu z osobna, lecz uczy się je szacować na podstawie widocznych regularności, zgodnie z tym, co opisano w rozdziale 2.

5.3.1. Sieć Q i maskowanie akcji

Do przybliżania wartości Q zaimplementowano sieć QNetwork. Przyjmuje ona na wejściu jeden z dwóch wariantów reprezentacji stanu, RAW albo FEATURES, opisanych w podrozdziale 5.1, a na wyjściu zwraca sześć liczb, po jednej dla każdej z sześciu moż-

liwych akcji naraz. Dzięki temu, żeby ocenić wszystkie dostępne w danym momencie decyzje, wystarczy jedno przejście sieci, a nie sześć osobnych. W środku umieszczono dwie warstwy ukryte po 256 neuronów z funkcją aktywacji ReLU [20]:

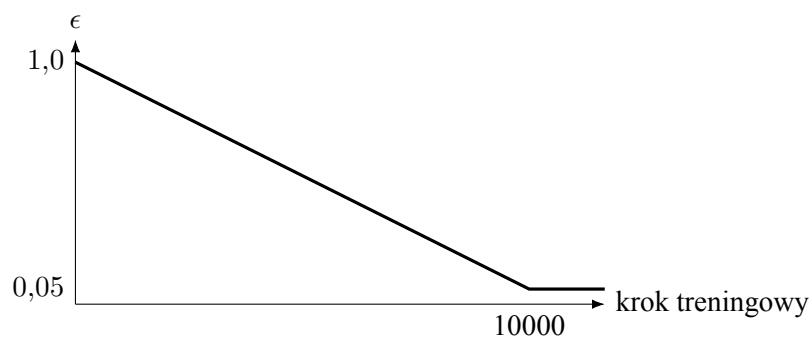
$$d_{\text{state}} \rightarrow 256 \rightarrow 256 \rightarrow 6, \quad d_{\text{state}} \in \{373, 393\}. \quad (5.5)$$

Nie każda z tych sześciu akcji jest jednak legalna w każdej fazie rozdania, na przykład spasowanie jest dostępne dopiero po odkryciu wszystkich kart wspólnych. Zanim agent podejmie decyzję, maskowane są więc wartości Q odpowiadające akcjom niedozwolonym w bieżącej fazie, dzięki czemu nigdy nie może on wybrać nielegalnej akcji. To samo maskowanie stosowane jest również przy wyznaczaniu celu dla następnego stanu, co opisano w podrozdziale o sieci docelowej i celu Bellmana.

5.3.2. Eksploracja epsilon-greedy

Sieć Q uczy się wyłącznie na podstawie akcji, które agent faktycznie wykonał, więc żeby poznać wartość jakiejś akcji, musi ją najpierw wypróbować. Gdyby agentowi pozwolono zawsze wybierać akcję, którą aktualnie ocenia jako najlepszą, mógłby on utknąć przy pierwszej, niekoniecznie najlepszej ocenie i nigdy nie sprawdzić alternatywy, która mogłaby dać lepszy wynik. Z tego powodu podczas treningu zastosowano powszechnie stosowaną w uczeniu przez wzmacnianie strategię epsilon-greedy [21].

Strategia ta polega na tym, że z pewnym prawdopodobieństwem, oznaczanym grecką literą epsilon, agent wybiera zupełnie losową legalną akcję, a w pozostałych przypadkach wybiera akcję, którą aktualnie ocenia jako najlepszą. Na początku treningu epsilon ustawiany jest na 1,0, więc agent na starcie eksploruje w zasadzie losowo, a w miarę postępu treningu wartość epsilon liniowo maleje, aż do 0,05 po 10000 kroków, zgodnie z rysunkiem 5.3. Agent z czasem więc coraz częściej korzysta z tego, czego się już nauczył, a coraz rzadziej próbuje czegoś nowego.



Rys. 5.3. Liniowy spadek wartości epsilon w trakcie treningu

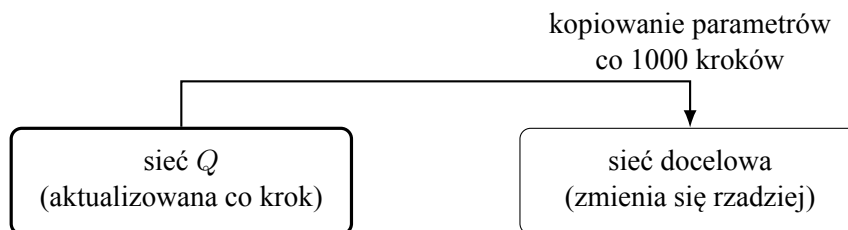
5.3.3. Bufor doświadczeń

Kolejne przejścia pomiędzy stanami nie są wykorzystywane do aktualizacji sieci od razu, tylko zapisywane w buforze doświadczeń (ang. *experience replay*), zaimplementowanym jako klasa `ReplayBuffer` przechowująca obiekty `Transition` [22, 23].

Gdyby sieć była uczona bezpośrednio na kolejnych przejściach z jednego rozdania, tak jak są rozgrywane, kilka aktualizacji z rzędu dotyczyłoby wciąż tych samych dwóch kart gracza i tego samego, stopniowo odkrywanego układu kart wspólnych. Sieć zaczęłaby więc dopasowywać się do wąskiego wycinka gry, zamiast uczyć się zależności ogólnej dla wszystkich możliwych rozdania. Bufor doświadczeń rozwiązuje ten problem. Po pierwsze, przejście trafia najpierw do bufora, a nie od razu do sieci. Po drugie, do każdej aktualizacji losowana jest porcja przejść pochodzących z wielu różnych, niepowiązanych ze sobą rozdania, dzięki czemu pojedyncza aktualizacja opiera się na zróżnicowanej próbce, a nie na fragmencie jednego rozdania.

5.3.4. Sieć docelowa i cel Bellmana

Oprócz sieci Q , która jest aktualizowana na bieżąco, utrzymywana jest druga, osobna sieć o identycznej architekturze, zwana siecią docelową (ang. *target network*). Jej parametry są synchronizowane z siecią aktualizowaną tylko okresowo, a nie na bieżąco. Wynika to z tego, że bez tego rozróżnienia cel, do którego dąży trening, zmieniałby się przy każdej pojedynczej aktualizacji razem z samą siecią, co przypominałoby próbę trafienia do celu, który sam co chwilę się przesuwa, co utrudniałoby zbieżność metody. Zależność między tymi dwiema sieciami przedstawiono na rysunku 5.4.



Rys. 5.4. Sieć Q i sieć docelowa

Cel aktualizacji wyznaczany jest zgodnie z równaniem Bellmana [24], jednym z najbardziej podstawowych równań w uczeniu przez wzmacnianie. Mówi ono, że wartość podjęcia danej akcji w danym stanie to suma nagrody otrzymanej natychmiast oraz zdyskontowanej wartości najlepszej możliwej akcji w stanie następnym. Dla przejścia nie-terminalnego cel Bellmana ma więc postać

$$y_t = r_t + \gamma \max_{a' \in A(s_{t+1})} Q_{\theta^-}(s_{t+1}, a'), \quad (5.6)$$

gdzie $A(s_{t+1})$ to zbiór akcji dozwolonych w stanie s_{t+1} , a θ^- to parametry sieci docelowej. natomiast dla przejścia terminalnego, czyli takiego, po którym rozdanie się kończy,

nie istnieje już żaden stan następny, więc cel ogranicza się do samej otrzymanej nagrody

$$y_t = r_t. \quad (5.7)$$

Współczynnik γ w tym równaniu, zwany współczynnikiem dyskontowania, określa, jak bardzo agent ma cenić nagrody odległe w czasie względem nagrody otrzymanej natychmiast [25, 26]. Wartość bliska zeru sprawiłaby, że agent liczyłby się właściwie tylko z najbliższą nagrodą, natomiast wartość bliska jedynce sprawia, że nagrody z dalszej przyszłości mają dla niego niemal taką samą wagę jak nagroda natychmiastowa. W przeprowadzonych eksperymentach przyjęto $\gamma = 0,99$, czyli wartość bliską jedynce, ponieważ samo rozdanie może obejmować kilka decyzji, a celem jest, żeby agent brał pod uwagę cały jego przebieg, a nie tylko najbliższy krok.

Funkcja `compute_bellman_targets()` implementowana jest zgodnie z powyższymi równaniami, maskując przy tym akcje niedozwolone w stanie s_{t+1} w dokładnie taki sam sposób jak przy wyborze akcji przez agenta.

5.3.5. Strata i optymalizacja

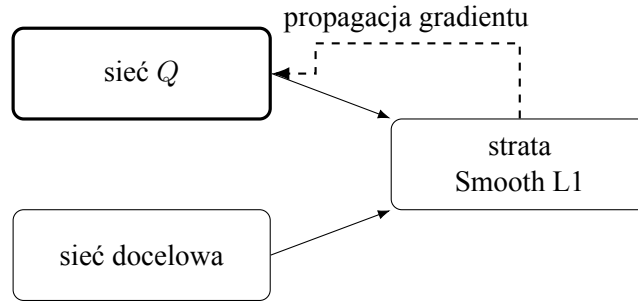
Sieć Q jest uczona poprzez minimalizację funkcji straty (ang. *loss function*), czyli funkcji, która mierzy, jak bardzo bieżąca ocena sieci różni się od wartości oczekiwanej, i którą podczas treningu dąży się zminimalizować [27]. W tym przypadku jest to funkcja Smooth L1, liczona pomiędzy $Q_\theta(s_t, a_t)$ a wyznaczoną wcześniej wartością docelową y_t , za pomocą klasy `DQNOptimizer`. W porównaniu ze zwykłym błędem średniokwadratowym Smooth L1 karze duże odchylenia łagodniej, co w tym przypadku ma uzasadnienie, ponieważ jak pokazuje tabela 2.3, pojedyncze rozdanie może zakończyć się wypłatą zakładu Blind w stosunku aż 500:1, a taki rzadki, skrajnie wysoki wynik nie powinien jednorazowo zdominować całej aktualizacji sieci [28, 29].

Do aktualizacji parametrów na podstawie wyznaczonej straty wykorzystywany jest optymalizator Adam. Jest to popularny optymalizator, który dla każdego parametru sieci osobno dostosowuje wielkość kroku aktualizacji na podstawie historii jego poprzednich gradientów, dzięki czemu zwykle zbiega szybciej i stabilniej niż podstawowy spadek gradientu ze stałym krokiem [30].

Sieć Q i sieć docelowa uczestniczą więc w każdej aktualizacji razem, ale w różny sposób. Obie wyznaczają wartości potrzebne do policzenia straty, jednak parametry aktualizuje wyłącznie propagacja gradientu przez sieć Q . Parametry sieci docelowej, jak opisano wcześniej, zmienia jedynie okresowe kopiowanie z rysunku 5.4, nigdy propagacja gradientu. Tę różnicę przedstawiono w celu wizualizacji na rysunku 5.5.

5.4. Policy Gradient - REINFORCE

Drugim algorytmem uczenia przez wzmocnienie, zaimplementowanym w tej pracy, jest Policy Gradient oparty na metodzie REINFORCE [8, 31]. W przeciwieństwie



Rys. 5.5. Przepływ gradientu podczas aktualizacji sieci Q

do DQN model nie estymuje wartości poszczególnych akcji. Sieć neuronowa bezpośrednio definiuje politykę, czyli rozkład prawdopodobieństwa akcji dla otrzymanego stanu.

5.4.1. Sieć polityki i wybór akcji

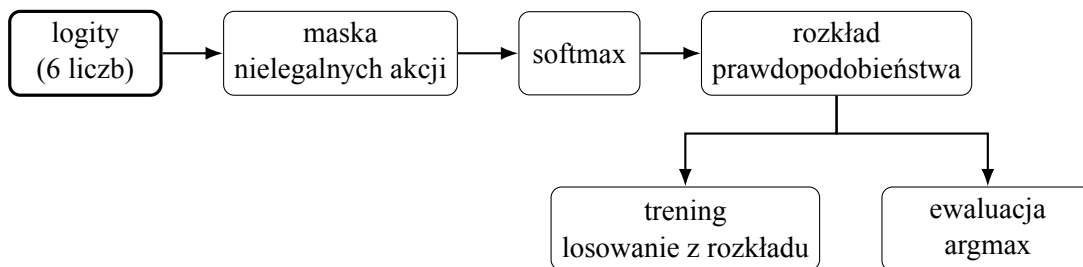
Sieć polityki PolicyNetwork przyjmuje wektor utworzony przez wybrany enkoder stanu. Zatem liczba wejść wynosi albo 373 dla RAW, albo 393 dla FEATURES. Zastosowano dwie warstwy ukryte po 256 neuronów z funkcją aktywacji ReLU oraz warstwę wyjściową o sześciu neuronach odpowiadających wszystkim akcjom.

$$d_{\text{state}} \rightarrow 256 \rightarrow 256 \rightarrow 6, \quad d_{\text{state}} \in \{373, 393\}. \quad (5.8)$$

gdzie d_{state} to wymiar wektora stanu, a 6 to liczba akcji.

Sieć zwraca surowe wartości, czyli logity. Przed utworzeniem rozkładu prawdopodobieństwa maskowane są akcje niedozwolone w aktualnej fazie rozdania. Dla legalnych akcji prawdopodobieństwa wyznaczone są za pomocą funkcji softmax [32].

Zaimplementowano klasę PolicyGradientAgent, która podczas treningu losuje akcję zgodnie z otrzymanym rozkładem, a nie zawsze wybiera akcję najbardziej prawdopodobną. Dzięki temu agent od czasu do czasu sprawdza również mniej oczywiste decyzje, co pozwala na naturalną eksplorację bez potrzeby dodatkowego mechanizmu losowania, takiego jak epsilon-greedy w DQN. Podczas ewaluacji istotna jest natomiast jak najlepsza, powtarzalna decyzja, dlatego wykorzystywana jest polityka deterministyczna, a wybierana jest legalna akcja o największym prawdopodobieństwie, czyli argument maksimum (ang. *argmax*) rozkładu. Cały ten przepływ przedstawiono na rysunku 5.6.



Rys. 5.6. Wybór akcji w Policy Gradient

5.4.2. Uczenie metodą REINFORCE

Jedno rozdanie Ultimate Texas Hold'em traktowane jest jako jeden epizod, a dla każdej decyzji zapisywany jest zakodowany stan, wybrana akcja, maska akcji legalnych oraz otrzymana nagroda w obiekcie PolicyStep, z których po zakończeniu epizodu składowana jest kompletna Trajectory. Dla każdego kroku wyznaczany jest wtedy zwrot

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k. \quad (5.9)$$

gdzie G_t to zwrot, czyli suma nagród liczona od kroku t do końca epizodu, T to liczba kroków w epizodzie, r_k to nagroda otrzymana w kroku k , a γ to współczynnik dyskontowania nagród, omówiony już w opisie DQN. Wykładnik $k - t$ rośnie wraz z odległością kroku k od kroku t , więc nagrody odległe w czasie są dyskontowane silniej niż nagrody bliskie.

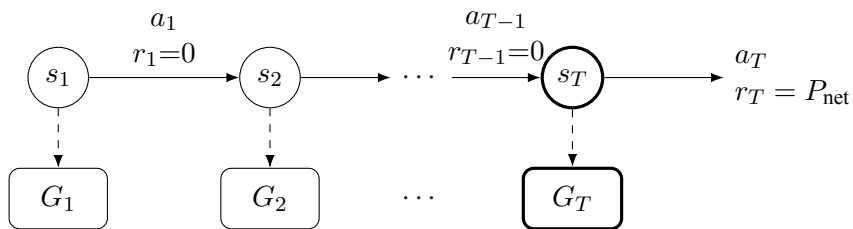
W eksperymentach przyjęto $\gamma = 1$, ponieważ optymalizowanym celem jest wynik finansowy całego rozdania, bez preferowania nagrody otrzymanej we wcześniejszym punkcie decyzji.

Parametry polityki są aktualizowane za pomocą klasy ReinforceOptimizer, zgodnie z klasycznym estymatorem REINFORCE [8]. Dla porcji zawierającej B kompletnych epizodów minimalizowana funkcja straty ma postać

$$L(\theta) = -\frac{1}{B} \sum_{i=1}^B \sum_t G_{i,t} \log \pi_{\theta}(a_{i,t} \mid s_{i,t}). \quad (5.10)$$

gdzie $G_{i,t}$ to zwrot w kroku t epizodu i , $a_{i,t}$ i $s_{i,t}$ to odpowiednio akcja i stan w tym kroku, a $\pi_{\theta}(a_{i,t} \mid s_{i,t})$ to prawdopodobieństwo wybrania tej akcji przez aktualną politykę.

Intuicyjnie ta aktualizacja zwiększa prawdopodobieństwo akcji, które doprowadziły do wysokiego zwrotu G_t , oraz zmniejsza prawdopodobieństwo akcji, po których zwrot był niski lub ujemny. Logarytm prawdopodobieństwa wyznacza kierunek zmiany parametrów, natomiast wartość i znak G_t decydują o sile tej zmiany.



$$G_1 = G_2 = \dots = G_T = r_T$$

(bo $\gamma = 1$, a $r_t = 0$ dla $t < T$)

Rys. 5.7. Zwrot G_t w epizodzie REINFORCE

Na rysunku s_t oznacza stan w kroku t , a_t podjętą w nim akcję, a r_t otrzymaną

nagrodę. Ponieważ $r_t = 0$ dla każdego kroku poza ostatnim, a $\gamma = 1$, każdy zwrot G_t sprowadza się do tej samej wartości, czyli końcowego wyniku netto rozdania $r_T = P_{\text{net}}$.

Zwroty obliczane są niezależnie dla każdego epizodu, a parametry sieci pozostają niezmiennicze podczas zbierania całej porcji. Dopiero po zebraniu 32 rozdań wykonywany jest jeden krok optymalizatora Adam. Takie podejście powinno ograniczyć wariancję aktualizacji w porównaniu z wykonywaniem osobnego kroku optymalizacji po każdym rozdaniu.

W domyślnej konfiguracji `PolicyGradientConfig` zastosowano learning rate równy 0,001, rozmiar porcji równy 32 oraz warstwy ukryte (256, 256). Nie stosuje się sieci wartości, punktu odniesienia (ang. *baseline*), regularyzacji entropii ani normalizacji zwrotów. REINFORCE pozostaje w ten sposób względnie prostą metodą Policy Gradient, którą można bezpośrednio porównać z DQN.

5.4.3. Reprodukowalność i diagnostyka

Z jednego ziarna wyprowadzane są osobne, niezależne ziarna dla sieci, środowiska i losowania akcji. Dzięki temu można później powtórzyć dokładnie ten sam trening, bez obawy, że różne źródła losowości się ze sobą pomieszą.

Podczas treningu zapisywany jest między innymi wynik każdego epizodu, liczba wykonanych kroków, wartość funkcji straty oraz norma gradientu. Dla ustalonego zestawu przykładowych stanów śledzone jest też, jak zmienia się rozkład prawdopodobieństwa akcji i jego entropia. Dane te są zbierane, ponieważ pomagają ocenić przebieg uczenia i wychwycić moment, w którym polityka zbyt szybko zawęża się do jednej, wciąż tej samej akcji.

Wyuczona sieć zapisywana jest jako punkt kontrolny (checkpoint) razem z konfiguracją treningu, wariantem reprezentacji stanu i rodzajem funkcji nagrody, dzięki czemu można ją później jednoznacznie odtworzyć i wykorzystać podczas ewaluacji.

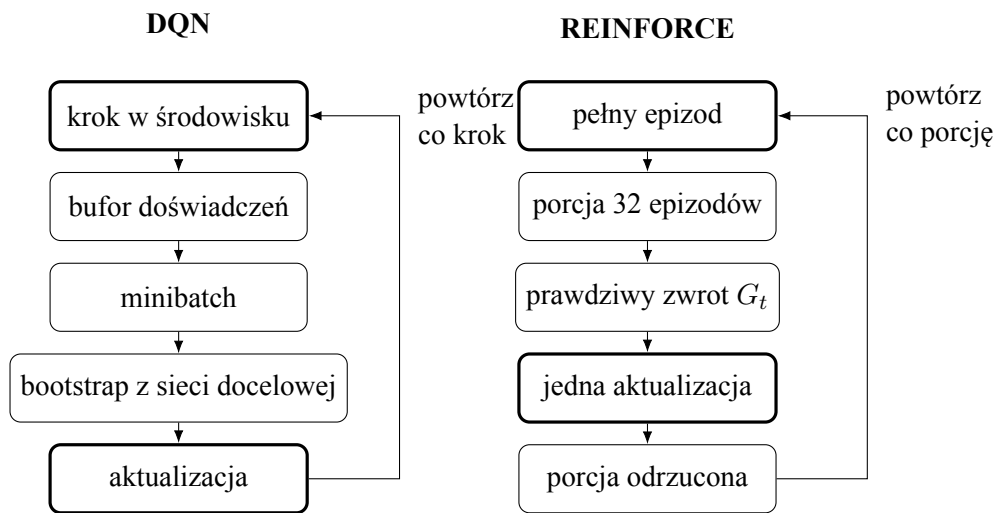
5.5. Porównanie podejść

DQN uczy się przybliżonej wartości Q dla każdej pary stanu i akcji, a decyzję podejmuje pośrednio, wybierając akcję o najwyższej ocenionej wartości. Policy Gradient nie szacuje wartości poszczególnych akcji, tylko uczy się wprost rozkładu prawdopodobieństwa ich wyboru, więc decyzja wynika bezpośrednio z wyjścia sieci polityki.

Różnica ta wpływa też na to, skąd bierze się cel każdej aktualizacji. DQN nie czeka na koniec rozdania, tylko szacuje cel na podstawie wartości ocenionej przez sieć docelową dla następnego stanu — technikę tę określa się mianem *bootstrappingu*. Wartość jednego stanu wyznacza się w niej na podstawie oszacowanej wartości stanu następnego, a nie na podstawie faktycznego wyniku całego epizodu [8]. REINFORCE działa inaczej, czeka, aż rozdanie się zakończy, i aktualizuje politykę na podstawie faktycznie otrzymanego wyniku netto, czyli zwrotu G_t z rysunku 5.7.

Ta różnica wpływa również na sposób wykorzystania zebranych danych. Ocena wartości akcji w DQN nie zależy od tego, jaka polityka aktualnie tę akcję wybiera. Estymator REINFORCE zakłada natomiast, że dany epizod pochodzi z polityki właśnie aktualizowanej, dlatego każda zebrana porcja epizodów jest wykorzystywana tylko do jednej aktualizacji, a następnie odrzucana.

Odmienne wygląda też eksploracja. DQN wymaga osobnego mechanizmu epsilon-greedy, ponieważ sama sieć Q zwraca deterministyczną ocenę wartości. W Policy Gradient eksploracja wynika naturalnie ze stochastycznego charakteru wyuczonej polityki. Te różnice przedstawiono na rysunku 5.8.



Rys. 5.8. DQN aktualizuje sieć na bieżąco, ponownie wykorzystując stare przejścia z bufora. REINFORCE zbiera całą porcję epizodów z aktualnej polityki i wykorzystuje go tylko raz

Zestawienie wszystkich hiperparametrów użytych do treningu obu algorytmów w podstawowym budżecie 50 000 epizodów przedstawiono w tabeli 5.2.

Tabela 5.2. Hiperparametry treningu DQN i REINFORCE

Hiperparametr	DQN	REINFORCE
Optymalizator	Adam	Adam
Learning rate	0,001	0,001
Współczynnik dyskontowania γ	0,99	1
Architektura sieci	256, 256	256, 256
Rozmiar minibatcha	64	32 (epizody)
Bufor doświadczeń	10 000 przejść	–
Moment rozpoczęcia uczenia	po 1000 krokach	po 1. epizodzie
Częstotliwość aktualizacji	co krok środowiska	co 32 epizody
Synchronizacja sieci docelowej	co 1000 kroków	–
Eksploracja	ϵ : 1,0 $\rightarrow$ 0,05 liniowo/10 000 kroków	stochastyczna polityka
Liczba parametrów (RAW / FEATURES)	163 078 / 168 198	163 078 / 168 198

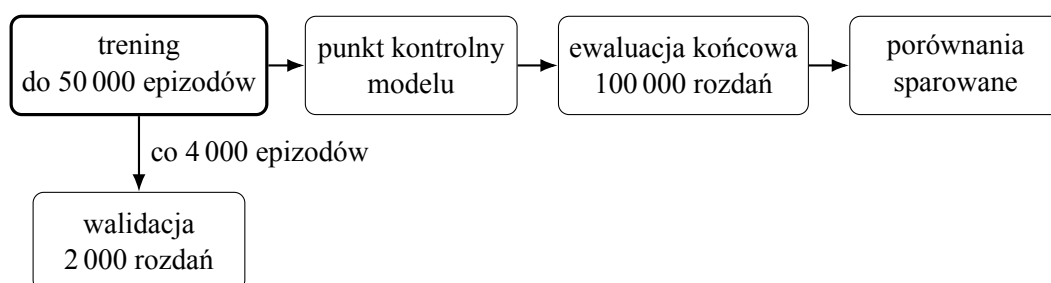
Obie sieci mają identyczną architekturę i liczbę parametrów, ponieważ korzystają z tego samego kształtu wejścia i wyjścia. Gradient clipping nie jest stosowany w żadnym

z algorytmów, a norma gradientu jest jedynie rejestrowana do celów diagnostycznych, bez ograniczania jej podczas treningu.

6. METODYKA EKSPERYMENTÓW

Przeprowadzono eksperymenty, w których porównywany jest wpływ algorytmu uczenia przez wzmacnianie, reprezentacji stanu oraz funkcji nagrody na jakość strategii uzyskiwanej w środowisku Ultimate Texas Hold'em. Wykorzystano dwa algorytmy opisane w rozdziale 5, tzn. DQN oraz REINFORCE. Za podstawową jednostkę treningu przyjęto epizod, czyli pojedyncze rozdanie. Polityką nazywany jest sam wyuczony model decyzyjny, natomiast strategią konkretne zachowanie, które ta polityka realizuje w danym momencie treningu lub ewaluacji.

Właściwości środowiska, sposób generowania rozdań oraz procedura ewaluacji są wspólne dla wszystkich eksperymentów, a poszczególne warianty eksperymentów różnią się wyłącznie badanymi elementami. Cały przebieg jednego eksperymentu, od treningu pojedynczego modelu do porównania wszystkich wariantów, przedstawiono na rysunku 6.1. Trening trwa do 50 000 epizodów i w jego trakcie co 4 000 epizodów wykonywana jest walidacja na 2 000 rozdaniach. Ta część przebiegu ma więc charakter cykliczny. Po zakończeniu treningu wytrenowany model przechodzi jednorazowo do ewaluacji końcowej na 100 000 rozdaniach, a jej wyniki wykorzystywane są następnie w porównaniach sparowanych.



Rys. 6.1. Przebieg jednego eksperymentu

6.1. Macierz eksperymentalna

Rozpatrywane są dwa algorytmy uczenia przez wzmacnianie, dwie reprezentacje stanu oraz dwie funkcje nagrody, co łącznie daje $2 \times 2 \times 2 = 8$ konfiguracji.

Dla każdego algorytmu analizowane są reprezentacje RAW oraz FEATURES, opisane w podrozdziale 5.1.1 i 5.1.2.

W przypadku funkcji nagrody porównywana jest bezpośrednia wartość zysku netto z rozdania oraz jej wersja przeskalowana przez maksymalną wartość stawki.

Wszystkie osiem wariantów przedstawiono w tabeli 6.1.

Tabela 6.1. Konfiguracje wykorzystane w eksperymentach

Algorytm	Reprezentacja	Nagroda
DQN	RAW	zysk netto
DQN	RAW	zysk netto skalowany
DQN	FEATURES	zysk netto
DQN	FEATURES	zysk netto skalowany
REINFORCE	RAW	zysk netto
REINFORCE	RAW	zysk netto skalowany
REINFORCE	FEATURES	zysk netto
REINFORCE	FEATURES	zysk netto skalowany

6.2. Powtarzalność eksperymentów

Ze względu na losowy charakter procesu uczenia pojedynczy przebieg nie jest wystarczający do wiarygodnej oceny konfiguracji. Każdy z ośmiu wariantów trenowany jest dlatego z użyciem pięciu niezależnych ziaren losowości, co daje w sumie $8 \times 5 = 40$ niezależnych procesów treningowych.

Ten sam zestaw pięciu ziaren wykorzystywany jest dla wszystkich wariantów. W ten sposób ograniczony jest wpływ przypadkowych różnic wynikających z losowania rozdań, co również ułatwia porównanie wpływu reprezentacji stanu oraz funkcji nagrody.

6.3. Budżet treningowy

Podstawowy budżet treningowy ustalono na 50 000 epizodów dla każdej konfiguracji. Wartość tę wybrano na podstawie wcześniejszego eksperymentu diagnostycznego, w którym obserwowano zachowanie wszystkich ośmiu wariantów podczas treningu.

W przypadku DQN zwiększanie liczby epizodów nie prowadziło do regularnego wzrostu jakości polityki. Co ciekawe, w części konfiguracji zaobserwowano nawet pogorszenie wyniku po dłuższym treningu.

W przypadku REINFORCE część przebiegów osiągała stabilne, proste polityki, takie jak wybieranie tego samego zakładu dla większości stanów. Dodatkowy przebieg diagnostyczny, rozszerzony do 100 000 epizodów, wykazał, że dalsze zwiększanie budżetu nie prowadziło do trwałej poprawy. Polityka przechodziła pomiędzy różnymi strategiami, a następnie ponownie zbliżała się do wcześniej obserwowanego rozwiązania.

Z tego powodu przyjęto wspólny budżet 50 000 epizodów dla obu algorytmów. Zapewnia to jednakowy budżet interakcji ze środowiskiem, nie oznacza to jednak porównywalnego kosztu obliczeniowego treningu: DQN aktualizuje sieć co krok środowiska po fazie rozgrzewki, podczas gdy REINFORCE aktualizuje politykę raz na 32 epizody, więc w tym samym budżecie epizodów wykonuje znacznie mniej aktualizacji parametrów. Dodatkowo oba algorytmy przyjmują nieco inny współczynnik dyskontowania ($\gamma = 0,99$ dla DQN, $\gamma = 1$ dla REINFORCE, patrz rozdział 5), co warto mieć na uwadze przy bezpośrednim porównaniu ich średniego EV w dalszej części pracy.

6.4. Walidacja okresowa

W trakcie treningu wykonywana jest okresowa ewaluacja aktualnej polityki. Jej celem jest obserwacja procesu uczenia i budowa krzywych przedstawiających zależność jakości strategii od liczby wykonanych epizodów.

Walidacja wykonywana jest co 4 000 epizodów oraz dodatkowo po zakończeniu 50 000 epizodów. Każdy punkt walidacyjny obejmuje 2 000 rozdań wygenerowanych na podstawie tego samego, ustalonego harmonogramu talii.

Dzięki wspólnemu harmonogramowi kolejne wersje polityki oceniane są na identycznym zestawie sytuacji pokerowych. Zmiana wartości oczekiwanej pomiędzy punktami walidacyjnymi wynika więc ze zmiany zachowania agenta, a nie z wykorzystania innego zestawu kart.

Podczas walidacji eksploracja jest wyłączana. Agent DQN wybiera legalną akcję o najwyższej wartości Q , a agent REINFORCE legalną akcję o najwyższym prawdopodobieństwie zgodnie z aktualną polityką. Pozwala to oceniać deterministyczną strategię wynikającą z aktualnego stanu modelu.

Warto podkreślić, że harmonogram używany podczas walidacji jest niezależny od rozdań wykorzystywanych w procesie treningowym.

6.5. Rozszerzona analiza treningu

Po wykonaniu podstawowego treningu przeprowadzono dodatkową analizę wydłużonego treningu. Na podstawie wyników walidacyjnych wybrano jeden wariant DQN oraz jeden wariant REINFORCE, oba z reprezentacją FEATURES, i wytrenowano je ponownie do 100 000 epizodów.

Celem tej rozszerzonej analizy treningu nie było zastąpienie podstawowego budżetu 50 000 epizodów ani wyszukanie najlepszego chwilowego punktu kontrolnego, tylko sprawdzenie, czy dalszy trening prowadzi do trwałej poprawy strategii, jej stabilizacji, czy dalszych oscylacji. Wyniki te są traktowane jako analiza uzupełniająca.

6.6. Ewaluacja końcowa

Po zakończeniu treningu modele są oceniane na oddzielnym harmonogramie rozdań, niewykorzystywanym ani podczas treningu, ani walidacji, ani nawet podczas wyboru budżetu treningowego.

Kończącą ewaluację ustalono na 100 000 identycznych rozdań dla każdej ocenianej strategii. Na tym samym zbiorze oceniane są również RandomAgent oraz RuleBasedAgent. Agenci bazowi nie wymagają procesu treningowego, dlatego liczba 100 000 dotyczy w ich przypadku wyłącznie liczby rozegranych rund ewaluacyjnych.

Dla przypomnienia, za podstawową miarę jakości strategii przyjęto średni zysk netto na jedno rozdanie

$$EV = \frac{1}{N} \sum_{i=1}^N p_i, \quad (6.1)$$

gdzie p_i oznacza zysk netto uzyskany w i -tym rozdaniu, natomiast N oznacza liczbę rozdań ewaluacyjnych.

Oprócz wartości oczekiwanej badane jest również odchylenie standardowe wyników, błąd standardowy średniej, średnia wartość postawionych żetonów, liczności wyników rozdania oraz częstości wyboru poszczególnych akcji.

6.7. Porównania sparowane

Wpływ pojedynczego elementu konfiguracji, czyli reprezentacji stanu, funkcji nagrody albo algorytmu, oceniany jest metodą porównań sparowanych na poziomie ziaren treningowych. Dla dwóch porównywanych konfiguracji A oraz B , wytrenowanych z tym samym zestawem pięciu ziaren, dla każdego ziarna i wyznaczana jest różnica końcowego EV

$$d_i = EV_{A,i} - EV_{B,i}, \quad i = 1, \dots, n, \quad (6.2)$$

gdzie $n = 5$ to liczba sparowanych ziaren. Średnia różnica wynosi

$$\bar{d} = \frac{1}{n} \sum_{i=1}^n d_i, \quad (6.3)$$

a jej zmienność opisywana jest odchyleniem standardowym próby

$$s_d = \sqrt{\frac{\sum_{i=1}^n (d_i - \bar{d})^2}{n - 1}} \quad (6.4)$$

oraz błędem standardowym średniej różnicy $SE_d = s_d/\sqrt{n}$, zgodnie ze wzorami już wprowadzonymi w podrozdziale 4.6. Ponieważ przy $n = 5$ próba jest mała, 95-procentowy przedział ufności dla $\bar{d}$ wyznaczany jest z rozkładu t -Studenta z $n - 1 = 4$ stopniami swobody, a nie z rozkładu normalnego:

$$\bar{d} \pm t_{0,975;4} \cdot SE_d, \quad t_{0,975;4} = 2,776. \quad (6.5)$$

Wartość dodatnia $\bar{d}$ oznacza przewagę konfiguracji A nad B . Metoda ta wykorzystywana jest we wszystkich porównaniach sparowanych w rozdziale 7, dotyczących wpływu reprezentacji stanu, funkcji nagrody oraz algorytmu na końcowe EV, a także porównania budżetu 50 000 i 100 000 epizodów.

Osobnym przypadkiem jest ewaluacja końcowa pojedynczego agenta (RandomAgent, RuleBasedAgent oraz każdego z ośmiu wytrenowanych modeli), która nie jest porównaniem sparowanym, tylko punktowym oszacowaniem EV na

$N = 100\,000$ rozdaniach. Tutaj, ze względu na dużą wartość N , 95-procentowy przedział ufności wyznaczany jest z przybliżenia normalnego, $EV \pm 1,96 \cdot SE$, korzystając z SD i SE zdefiniowanych w podrozdziale 4.6.

7. WYNIKI EKSPERYMENTÓW

W tym rozdziale przedstawiono wyniki eksperymentów. Metodykę eksperymentów opisano w rozdziale 6. Uzyskane wyniki analizowane są pod kątem jakości końcowych strategii, wpływu reprezentacji stanu i funkcji nagrody, przebiegu procesu uczenia oraz zachowania modeli po zwiększeniu budżetu treningowego. Wyniki te przedstawiono także na wykresach w celu lepszej wizualizacji.

Każdą z ośmiu konfiguracji wytrenowano pięciokrotnie z wykorzystaniem tego samego zestawu pięciu ziaren treningowych. Kończącą ocenę każdego modelu przeprowadzono na wspólnym harmonogramie 100 000 rozdań, który nie był wykorzystywany podczas treningu.

7.1. Wyniki końcowej ewaluacji

Najpierw porównano końcowe wyniki wszystkich ośmiu podstawowych konfiguracji po 50 000 epizodów treningowych. Dla każdej konfiguracji obliczono średnią wartość EV z pięciu niezależnych przebiegów treningowych oraz odchylenie standardowe między ziarnami. Wyniki przedstawiono w tabeli 7.1.

Tabela 7.1. Wyniki końcowej ewaluacji modeli po 50 000 epizodów treningowych

Algorytm	Reprezentacja	Nagroda	Średnie EV	SD ziaren
DQN	RAW	zysk netto	-0,6450	0,0184
DQN	RAW	skalowana	-0,5681	0,0401
DQN	FEATURES	zysk netto	-0,4076	0,0080
DQN	FEATURES	skalowana	-0,4165	0,0169
REINFORCE	RAW	zysk netto	-0,3785	0,0074
REINFORCE	RAW	skalowana	-0,3804	0,0014
REINFORCE	FEATURES	zysk netto	-0,3823	0,0014
REINFORCE	FEATURES	skalowana	-0,3211	0,0657

Najwyższe średnie EV spośród wariantów DQN uzyskała konfiguracja FEATURES z funkcją nagrody opartą na zysku netto. Jej średni wynik wyniósł $-0,4076$. Dla REINFORCE najwyższy średni wynik uzyskała konfiguracja FEATURES ze skalowaną funkcją nagrody, dla której średnie EV wyniosło $-0,3211$.

Widoczna jest wyraźna różnica w zmienności tych wyników. Dla wspomnianego wariantu DQN odchylenie standardowe między pięcioma ziarnami wyniosło jedynie 0,0080, natomiast dla konfiguracji REINFORCE FEATURES ze skalowaną nagrodą osiągnęło 0,0657. Oznacza to, że wysoka średnia wartość EV wariantu REINFORCE była związana z dużo większym zróżnicowaniem jakości polityk uzyskanych w poszczególnych przebiegach. Do tego zjawiska powrócono w dalszej części rozdziału.

Na tym samym harmonogramie 100 000 rozdań ponownie oceniono dwóch agentów bazowych, Random i RuleBased. Ich wyniki przedstawiono w tabeli 7.2.

Tabela 7.2. Wyniki końcowej ewaluacji agentów bazowych

Agent	Średnie EV	SD	SE	95% CI
Random	-0,5173	5,2944	0,0167	[-0,5501, -0,4845]
RuleBased	-0,0106	5,2159	0,0165	[-0,0429, 0,0217]

Wartość uzyskana przez agenta regułowego była znacznie bliżej zera niż wyniki wszystkich wytrenowanych modeli uczenia przez wzmacnianie.

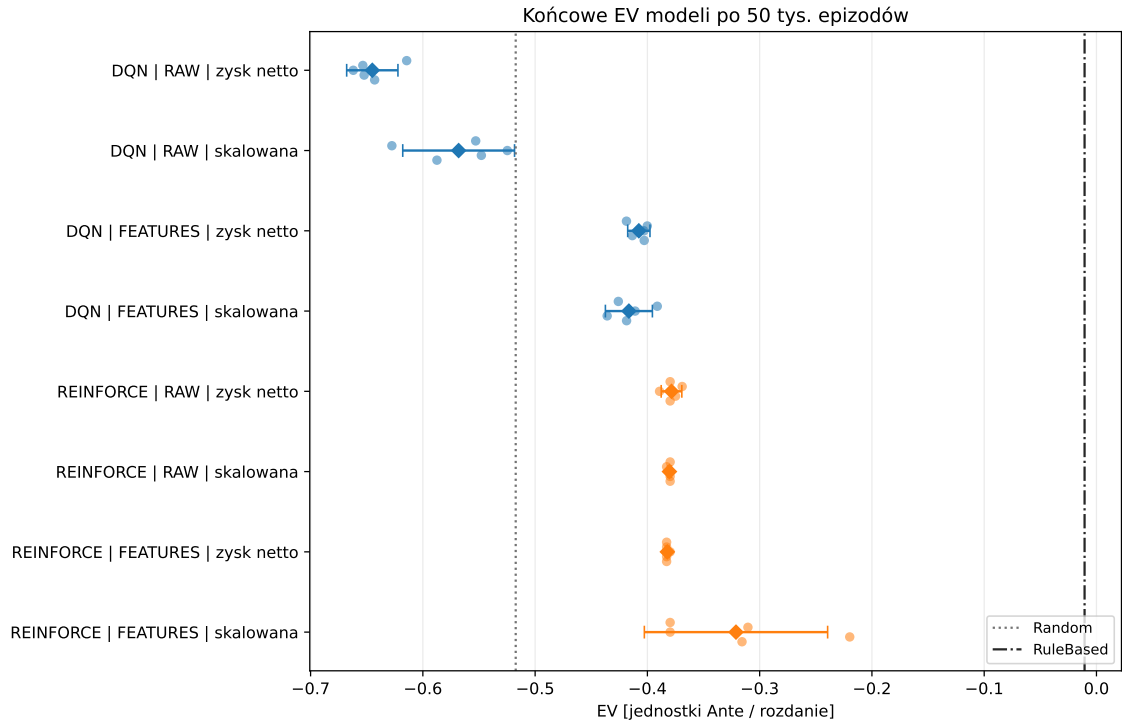
Warto dodać, że wynik agenta regułowego należy odnieść do teoretycznej przewagi kasyna wynikającej z zastosowanej przez niego strategii. Uproszczona strategia Wizard of Odds, na której oparto reguły agenta regułowego, ma teoretyczną przewagę kasyna równą 2,43% wartości zakładu Ante, czyli oczekiwane EV na poziomie około $-0,0243$ [3]. Uzyskane EV agenta regułowego, $-0,0106$, jest więc punktowo wyższe od tej teoretycznej wartości, jednak 95-procentowy przedział ufności, $[-0,0429, 0,0217]$, obejmuje $-0,0243$. Różnicę pomiędzy wynikiem empirycznym a teoretycznym można więc wyjaśnić wariancją próby 100 000 rozdań, a nie błędem implementacji reguł.

Niemniej jednak większość konfiguracji RL osiągnęła wyższe średnie EV niż agent losowy. Wyjątek stanowiły dwa warianty DQN wykorzystujące reprezentację RAW, których średnie EV wyniosły odpowiednio $-0,6450$ i $-0,5681$. Po zastosowaniu reprezentacji FEATURES oba warianty DQN uzyskały już wynik wyższy niż agent losowy. W przypadku REINFORCE wszystkie cztery badane konfiguracje osiągnęły wyższe średnie EV od agenta losowego.

Zestawienie wyników poszczególnych ziaren, ich średnich oraz położenia agentów bazowych przedstawiono na rysunku 7.1. Na osi poziomej znajduje się wartość EV, a na osi pionowej osiem konfiguracji algorytmów. Małe punkty przedstawiają wyniki poszczególnych ziaren, większe znaczniki średnie wartości konfiguracji wraz z 95-procentowymi przedziałami ufności, a linie pionowe oznaczają wyniki agentów Random i RuleBased. Największy rozrzut widoczny jest dla konfiguracji REINFORCE FEATURES ze skalowaną nagrodą. Sama końcowa ewaluacja nie wystarcza jednak do oceny stabilności całego procesu uczenia, dlatego problem ten wraca w dalszej części rozdziału.

Sama informacja o zastosowanym algorytmie nie wystarcza do oceny jakości uzyskanej strategii. W przypadku DQN rezultat zmienia się znacznie wraz ze sposobem reprezentacji stanu. W przypadku REINFORCE największa różnica pomiędzy wariantami widoczna jest dla połączenia reprezentacji FEATURES ze skalowaną funkcją nagrody. Temat ten jest analizowany w dalszej części rozdziału.

Wszystkie wartości EV opierają się na tym samym, współdzielonym harmonogramie 100 000 rozdań, co pozwala prześledzić, jak szybko średni wynik zbiega do swojej ostatecznej wartości. Na rysunku 7.2 przedstawiono ten proces dla agentów Random i RuleBased oraz najlepszych walidacyjnie konfiguracji DQN i REINFORCE. Na osi poziomej, w skali logarytmicznej, znajduje się liczba rozegranych rozdań, a na osi pionowej



Rys. 7.1. Końcowe EV ośmiu konfiguracji po 50 000 epizodów treningowych

średni wynik narastająco. Pod koniec harmonogramu zmiany średniego wyniku narastającego są już niewielkie. Pokazuje to, że przyjęty wcześniej budżet 100 000 rozdań pozwala zaobserwować stabilizację oszacowania EV, a końcowy wynik nie jest oparty wyłącznie na krótkim fragmencie symulacji.

7.2. Wpływ reprezentacji stanu

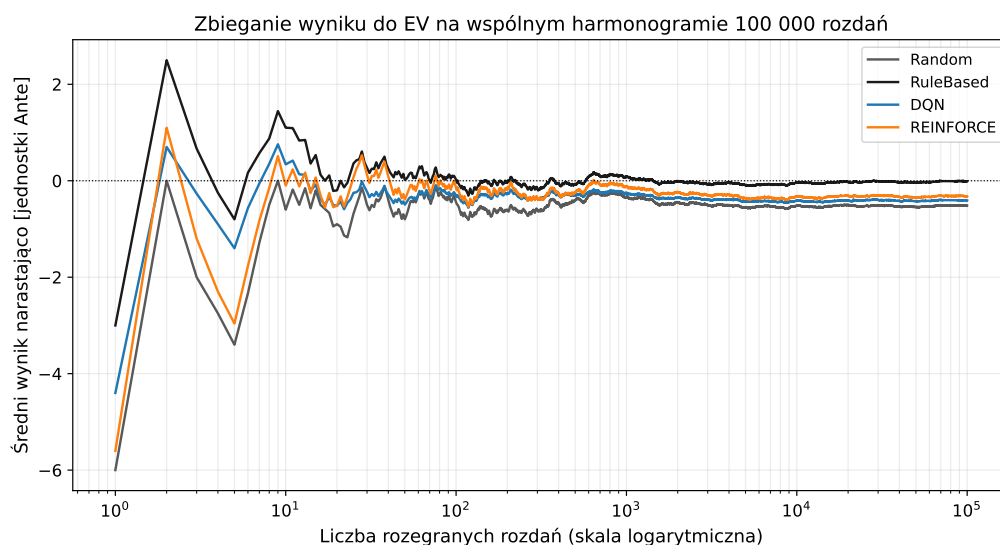
W celu oceny wpływu sposobu reprezentacji stanu porównano warianty RAW i FEATURES przy zachowaniu tego samego algorytmu, funkcji nagrody oraz ziarna treningowego. Dla każdej pary wyznaczono różnicę

$$\Delta EV_{\text{state}} = EV_{\text{FEATURES}} - EV_{\text{RAW}}. \quad (7.1)$$

Zgodnie ze wzorem dodatnia wartość różnicy oznacza zatem przewagę reprezentacji FEATURES. Porównanie wykonano osobno dla obu algorytmów i obu funkcji nagrody. Jednostką porównania jest w tym przypadku ziarno treningowe, dlatego każda wartość średnia oraz odpowiadający jej przedział ufności wynikają z pięciu sparowanych różnic.

Wyniki wszystkich czterech porównań przedstawiono w tabeli 7.3.

W przypadku DQN zastosowanie reprezentacji FEATURES doprowadziło do wyraźnej poprawy końcowego EV dla obu funkcji nagrody, w obu przypadkach cały przedział ufności leży powyżej zera. Wynika z tego, że w badanej konfiguracji DQN rozszerzenie wektora stanu o cechy opisujące sytuację pokerową poprawiało jakość uzyskiwanej strategii.



Rys. 7.2. Zbieganie średniego wyniku do EV na współdzielonym harmonogramie rozdań

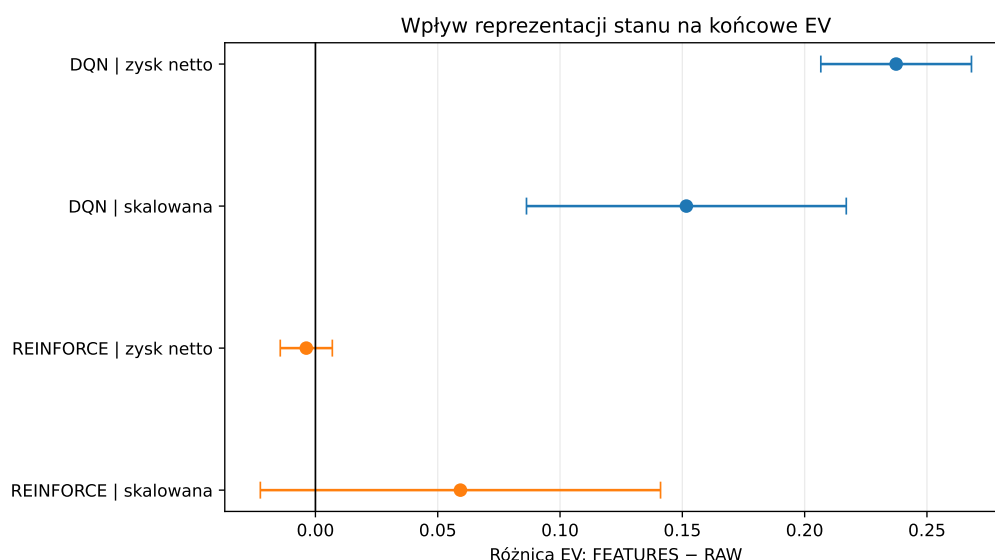
Tabela 7.3. Wpływ reprezentacji stanu na EV

Algorytm	Nagroda	ΔEV_{state}	95% CI
DQN	zysk netto	0,2374	[0,2066, 0,2682]
DQN	skalowana	0,1517	[0,0863, 0,2170]
REINFORCE	zysk netto	-0,0037	[-0,0144, 0,0069]
REINFORCE	skalowana	0,0593	[-0,0225, 0,1411]

Dla REINFORCE wpływ reprezentacji stanu był znacznie mniej jednoznaczny. Przy funkcji nagrody opartej na zysku netto przedział ufności obejmował zero, dlatego na podstawie tych pięciu par nie można wskazać wyraźnej przewagi żadnej z reprezentacji. Przy skalowanej funkcji nagrody punktowe oszacowanie wskazuje na wyższe EV reprezentacji FEATURES, jednak przedział ufności jest znacznie szerszy i również obejmuje zero, więc duże zróżnicowanie wyników między ziarnami nie pozwala uznać tego efektu za równie stabilny jak w przypadku DQN.

Te same porównania przedstawiono także na rysunku 7.3, gdzie na osi poziomej znajduje się różnica $EV_{FEATURES} - EV_{RAW}$, a na osi pionowej cztery kombinacje algorytmu i funkcji nagrody. Punkty przedstawiają średnią różnicę dla pięciu sparowanych ziaren treningowych, a wąsy odpowiadają 95-procentowym przedziałom ufności, wartości dodatnie oznaczają przewagę reprezentacji FEATURES. Widać, że przedziały ufności obu wariantów DQN leżą całkowicie po dodatniej stronie zera, natomiast oba przedziały REINFORCE obejmują zero.

Rezultaty wskazują na odmienny wpływ reprezentacji stanu na oba badane algorytmy. DQN korzystał z informacji zawartych w rozszerzonej reprezentacji FEATURES, natomiast dla REINFORCE nie zaobserwowano równie stabilnego efektu zmiany reprezentacji.



Rys. 7.3. Wpływ reprezentacji stanu na końcowe EV

7.3. Wpływ funkcji nagrody

Drugim elementem konfiguracji, który analizowano, była funkcja nagrody. Porównano bezpośredni zysk netto z jego wersją przeskalowaną przy zachowaniu tego samego algorytmu, reprezentacji stanu oraz ziarna treningowego. Różnicę zdefiniowano jako

$$\Delta EV_{\text{reward}} = EV_{\text{net}} - EV_{\text{scaled}}. \quad (7.2)$$

Ze wzoru wynika, że wartość dodatnia oznacza przewagę bezpośredniego zysku netto, natomiast ujemna przewagę nagrody skalowanej. Wyniki wszystkich czterech porównań przedstawiono w tabeli 7.4.

Tabela 7.4. Wpływ funkcji nagrody na EV

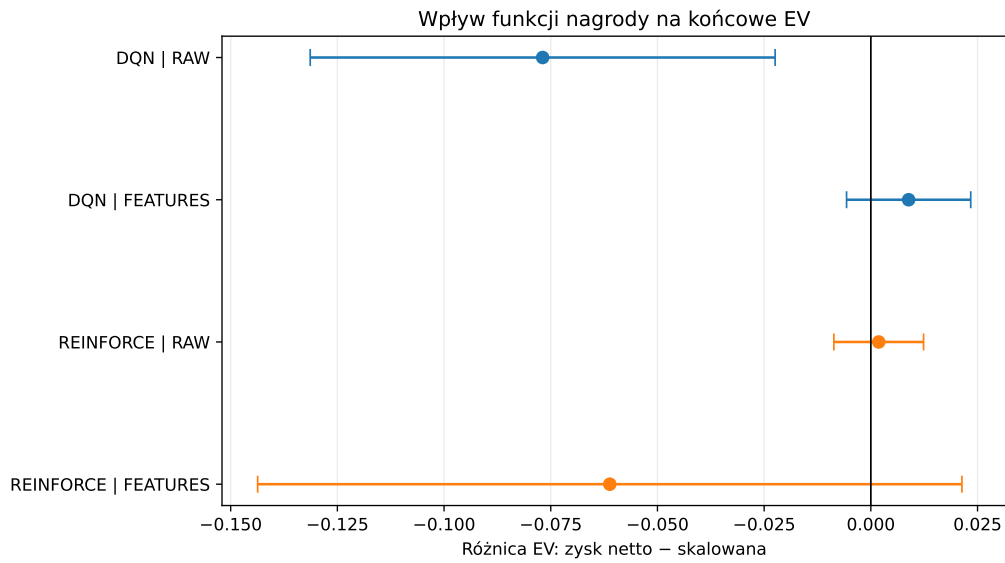
Algorytm	Reprezentacja	$\Delta EV_{\text{reward}}$	95% CI
DQN	RAW	-0,0769	[-0,1314, -0,0224]
DQN	FEATURES	0,0089	[-0,0057, 0,0234]
REINFORCE	RAW	0,0018	[-0,0087, 0,0124]
REINFORCE	FEATURES	-0,0612	[-0,1437, 0,0213]

Dla DQN wykorzystującego reprezentację RAW cały przedział ufności leży poniżej zera. Skalowanie nagrody poprawiło więc w tym wariancie wyniki końcowe w porównaniu z bezpośrednim wykorzystaniem zysku netto. Sytuacja wyglądała inaczej po zastosowaniu FEATURES: różnica była niewielka, a przedział obejmuje zero, dlatego na podstawie tych wyników nie można wskazać wyraźnej przewagi jednej z funkcji nagrody dla DQN korzystającego z tej reprezentacji.

Podobny brak zauważalnej przewagi wystąpił dla REINFORCE z reprezentacją RAW, gdzie przedział ufności również obejmował zero. Największą różnicę punktową dla

REINFORCE zaobserwowano przy reprezentacji FEATURES, gdzie wyższy średni wynik osiągnęła funkcja skalowana. Jednocześnie przedział ufności był tu bardzo szeroki, co jest zgodne z dużym zróżnicowaniem końcowej jakości polityk REINFORCE obserwowanym pomiędzy ziarnami.

Te same porównania przedstawiono także na rysunku 7.4. Na osi poziomej znajduje się różnica $EV_{\text{net}} - EV_{\text{scaled}}$, a na osi pionowej cztery kombinacje algorytmu i reprezentacji stanu. Punkty przedstawiają średnią różnicę dla pięciu sparowanych ziaren treningowych, a wąsy 95-procentowe przedziały ufności. W przeciwieństwie do wykresu wpływu reprezentacji stanu, tutaj tylko jeden z czterech przedziałów, DQN z reprezentacją RAW, leży całkowicie po jednej stronie zera.



Rys. 7.4. Wpływ funkcji nagrody na końcowe EV

W przeciwieństwie do reprezentacji stanu nie obserwuje się więc uniwersalnego efektu skalowania funkcji nagrody. Jej wpływ zależy od połączenia z algorytmem oraz sposobem reprezentacji stanu. Skalowanie poprawiło wyniki DQN korzystającego z reprezentacji RAW, lecz dla DQN z reprezentacją FEATURES różnica była niewielka. W przypadku REINFORCE punktowo najlepszy wynik uzyskała konfiguracja FEATURES ze skalowaną nagrodą, ale wynik ten był jednocześnie silnie zależny od ziarna treningowego.

7.4. Wpływ algorytmu

Problem badawczy tej pracy obejmuje również wybór samego algorytmu uczenia przez wzmacnianie, dlatego oprócz wpływu reprezentacji stanu i funkcji nagrody analizowany jest również główny efekt algorytmu. Porównano REINFORCE z DQN przy zachowaniu tej samej reprezentacji stanu, funkcji nagrody oraz ziarna treningowego. Różnicę zdefiniowano jako

$$\Delta EV_{\text{alg}} = EV_{\text{REINFORCE}} - EV_{\text{DQN}}. \quad (7.3)$$

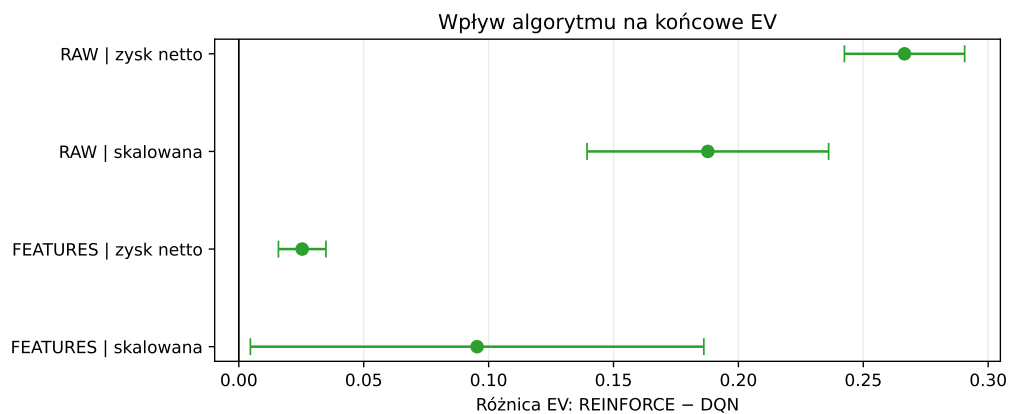
Ze wzoru wynika, że wartość dodatnia oznacza przewagę REINFORCE, a ujemna przewagę DQN. Wyniki wszystkich czterech porównań, z 95-procentowym przedziałem ufności wyznaczonym z rozkładu t -Studenta dla pięciu sparowanych ziaren (podrozdział 6.7), przedstawiono w tabeli 7.5.

Tabela 7.5. Wpływ algorytmu na końcowe EV

Reprezentacja	Nagroda	ΔEV_{alg}	95% CI
RAW	zysk netto	0,2665	[0,2424, 0,2906]
RAW	skalowana	0,1878	[0,1394, 0,2361]
FEATURES	zysk netto	0,0254	[0,0159, 0,0349]
FEATURES	skalowana	0,0954	[0,0046, 0,1862]

We wszystkich czterech przypadkach przedział ufności leży całkowicie po dodatniej stronie zera, co oznacza statystycznie istotną przewagę REINFORCE nad DQN w podstawowym budżecie 50 000 epizodów, niezależnie od zastosowanej reprezentacji stanu i funkcji nagrody. Przewaga ta jest jednak wyraźnie mniejsza dla reprezentacji FEATURES (0,0254 i 0,0954) niż dla RAW (0,2665 i 0,1878), co jest zgodne z wynikiem podrozdziału 7.2: reprezentacja FEATURES poprawiała wyniki DQN w sposób bardziej spójny niż wyniki REINFORCE, więc różnica między algorytmami jest mniej widoczna właśnie tam, gdzie DQN korzysta z dodatkowych cech domenowych.

Te same porównania przedstawiono także na rysunku 7.5.



Rys. 7.5. Wpływ algorytmu na końcowe EV

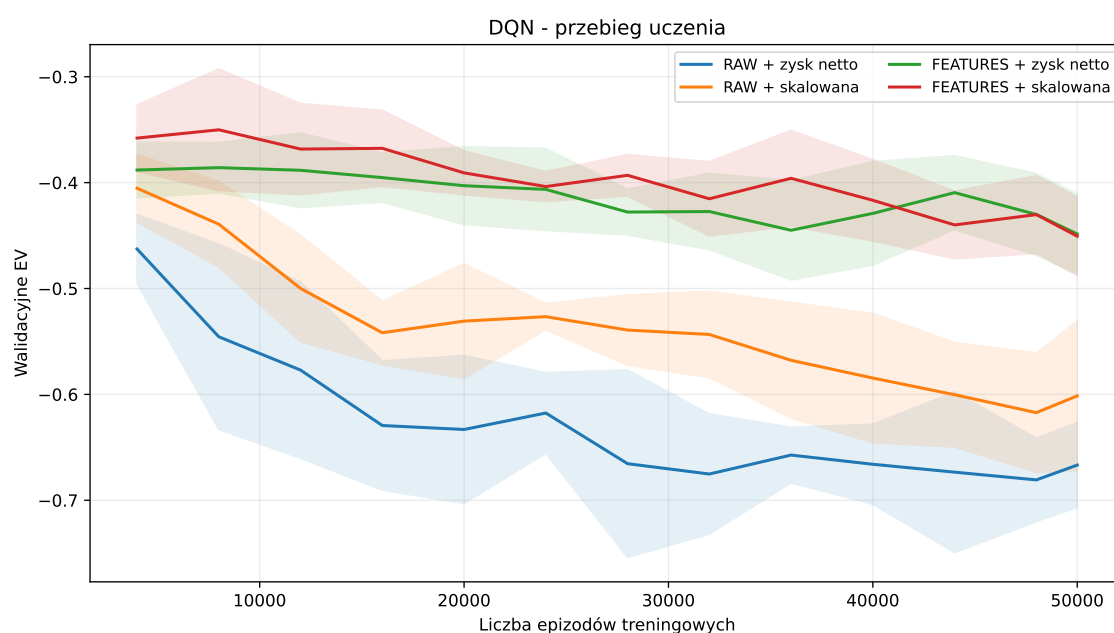
Statystycznie istotna przewaga REINFORCE nie oznacza jednak, że jest to metoda lepsza pod każdym względem. Jak opisano w podrozdziale 7.8, wyższe średnie EV REINFORCE było w wielu przebiegach związane z degeneracją polityki do jednej, niezależnej od stanu akcji, a wybrana walidacyjnie konfiguracja DQN dawała bardziej powtarzalne wyniki między ziarnami. Poza tym oba algorytmy przyjmują nieco inny współczynnik dyskontowania, co opisano jako ograniczenie porównania w rozdziale Podsumowanie i dalszy rozwój pracy.

7.5. Przebieg procesu uczenia

W tej sekcji przedstawiono, jak kształtował się przebieg treningu. W tym celu wykorzystano okresowe pomiary walidacyjne wykonywane zgodnie z procedurą opisaną w rozdziale 6. Każdy punkt krzywej odpowiada ewaluacji aktualnej polityki na 2 000 wspólnych rozdań walidacyjnych.

Wyniki walidacyjne są traktowane jako opis dynamiki procesu uczenia, a nie jako zamiennik końcowej ewaluacji na 100 000 niezależnych rozdań. Mniejsza liczba rozdań w pojedynczym punkcie powoduje większą zmienność oszacowanego EV, jednak wspólny harmonogram walidacyjny umożliwia bezpośrednie porównywanie kolejnych punktów kontrolnych.

Na rysunku 7.6 przedstawiono przebieg treningu czterech konfiguracji DQN. Na osi poziomej znajduje się liczba epizodów treningowych, a na osi pionowej walidacyjne EV. Linie przedstawiają średnie z pięciu ziaren, a pasma jedno odchylenie standardowe między przebiegami treningowymi. Widać na nim, że warianty FEATURES przez cały trening utrzymują się wyraźnie powyżej wariantów RAW, a odstęp między krzywymi nie zmniejsza się znacząco w czasie trwania treningu.



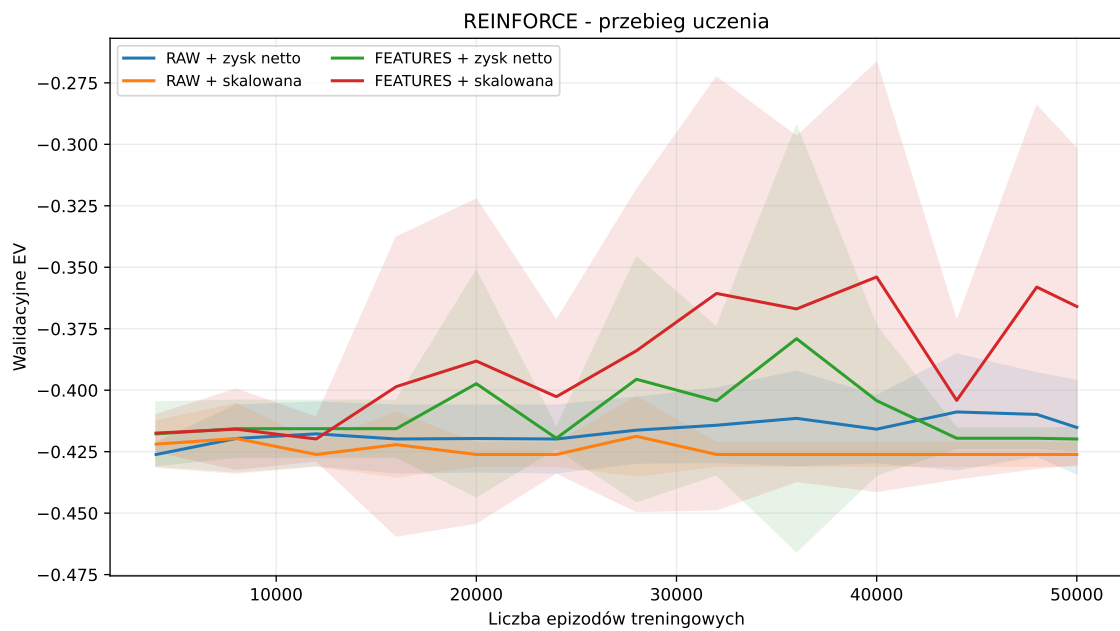
Rys. 7.6. Przebieg uczenia czterech konfiguracji DQN

Już w trakcie treningu widoczna jest różnica pomiędzy reprezentacjami RAW i FEATURES. Oba warianty wykorzystujące FEATURES utrzymują przeciętnie wyższe EV niż odpowiadające im warianty RAW. Jest to zgodne z wynikami końcowej ewaluacji przedstawionymi w sekcji 7.2.

Jednocześnie krzywe nie wykazują monotonicznego wzrostu jakości polityki. Po okresach poprawy występują lokalne spadki wartości EV. Oznacza to, że dalsze wykonywanie aktualizacji sieci nie gwarantuje poprawy strategii na każdym kolejnym punkcie

kontrolnym. Zjawisko to występuje również dla konfiguracji FEATURES.

Analogiczne wyniki dla REINFORCE przedstawiono na rysunku 7.7, z tymi samymi osiami i tym samym sposobem uśredniania. Krzywe REINFORCE są znacznie mniej gładkie niż dla DQN, a pasma odchylenia standardowego są szersze i w wielu punktach przecinają się nawzajem, co wskazuje, że w tym samym momencie treningu różne konfiguracje mogą osiągać porównywalną jakość polityki.



Rys. 7.7. Przebieg uczenia czterech konfiguracji REINFORCE

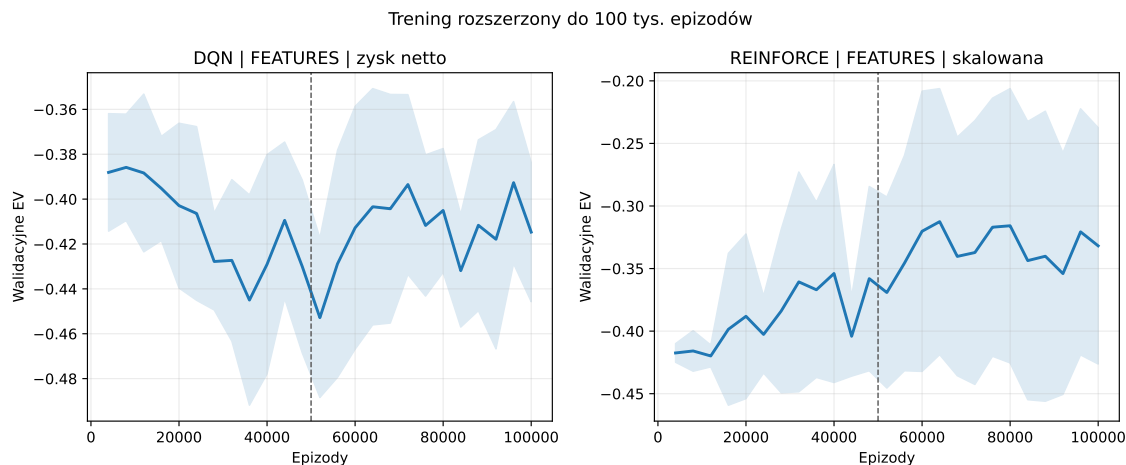
Konfiguracja FEATURES ze skalowaną funkcją nagrody osiągnęła najwyższe średnie EV spośród konfiguracji REINFORCE w końcowej ewaluacji, ale jednocześnie charakteryzowała się największym odchyleniem standardowym między ziarnami. Krzywe walidacyjne pokazują, że zróżnicowanie to powstaje już podczas procesu uczenia i nie jest wyłącznie efektem końcowej ewaluacji.

7.6. Wpływ zwiększenia budżetu treningowego

Po zakończeniu podstawowych eksperymentów rozszerzono trening dwóch konfiguracji wybranych na podstawie wyników walidacyjnych. Dla DQN była to konfiguracja FEATURES z nagrodą opartą na zysku netto, natomiast dla REINFORCE konfiguracja FEATURES ze skalowaną funkcją nagrody. Dla obu konfiguracji uruchomiono te same ziarna z budżetem 100 000 epizodów. Pierwsze 50 000 epizodów odtworzyło się przy tym deterministycznie, co sprawdzono, porównując punkty walidacyjne z podstawowego przebiegu, więc wynik do 100 000 epizodów można traktować jako przedłużenie tej samej trajektorii treningowej.

Przebieg walidacyjnego EV uśredniony po pięciu ziarnach przedstawiono na rysunku 7.8. Na osi poziomej znajduje się liczba epizodów treningowych do 100 000, a na

osi pionowej walidacyjne EV, linia pionowa oznacza podstawowy budżet 50 000 epizodów. Dla DQN wartość końcowa po 100 000 epizodach jest wyższa niż po 50 000, mimo lokalnych wahań w trakcie dalszego treningu. Dla REINFORCE średnia też rośnie, ale trajektorie poszczególnych ziaren różnią się od siebie znacznie bardziej.



Rys. 7.8. Przebieg walidacyjnego EV wybranych konfiguracji DQN i REINFORCE do 100 000 epizodów

Uśredniona krzywa nie pokazuje jednak, czy poprawa dotyczy wszystkich ziaren w podobnym stopniu. Trajektorie poszczególnych przebiegów przedstawiono na rysunku 7.9, z tymi samymi osiami co powyżej, dodatkowo z rozbiciem na pięć kolorów odpowiadających poszczególnym ziarnom oraz naniesioną krzywą średnią. Dla REINFORCE widoczna jest też płaska trajektoria odpowiadająca zdegenerowanym przebiegom. Część linii może się przy tym nakładać, ponieważ identyczna, deterministyczna polityka oceniana na tym samym harmonogramie daje identyczny wynik. Jest to zgodne ze zjawiskiem degeneracji polityki do jednej, niezależnej od stanu akcji, opisanym szerzej w sekcji 7.8.

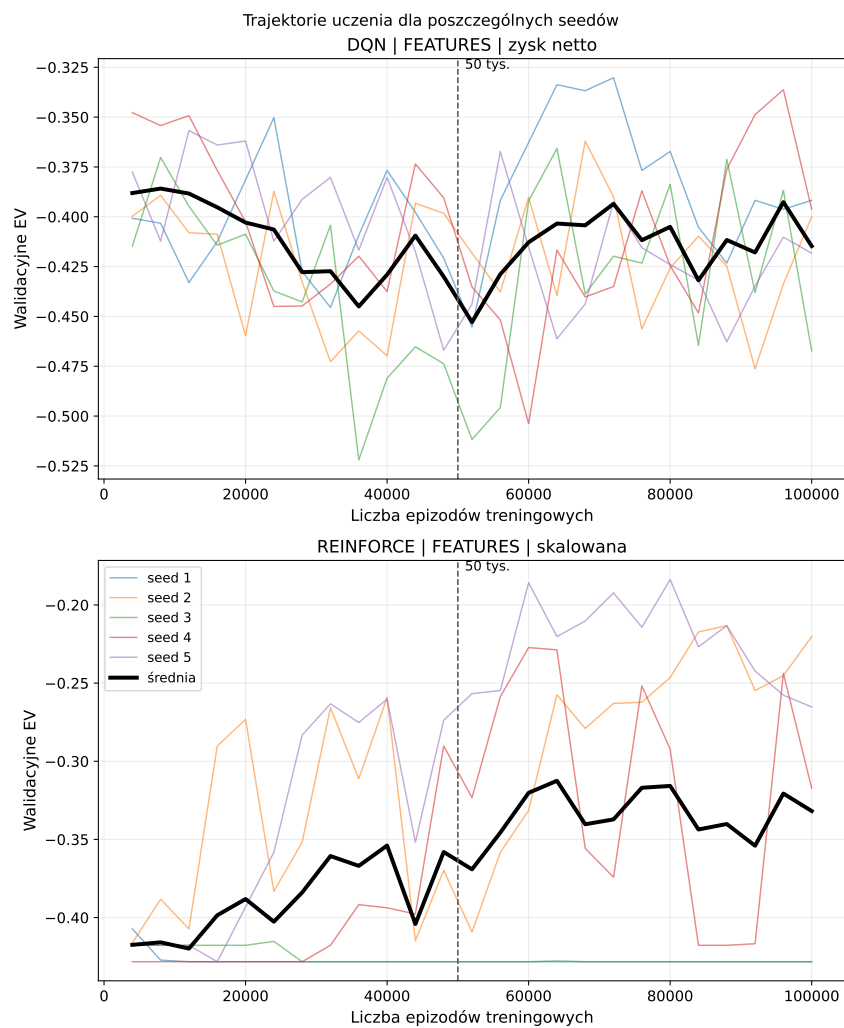
Dla DQN wydłużenie treningu prowadziło do stosunkowo spójnej poprawy. W końcowej ewaluacji na wspólnych 100 000 rozdaniach średnie EV pięciu modeli wzrosło z $-0,4076$ po 50 000 epizodów do $-0,3859$ po 100 000 epizodów. Średnia sparowana poprawa wyniosła

$$\Delta EV_{100k-50k} = 0,0217, \quad (7.4)$$

a 95-procentowy przedział ufności po pięciu ziarnach był równy $[0,0022, 0,0412]$.

Co istotne, poprawę końcowego EV zaobserwowano dla wszystkich pięciu ziaren wybranej konfiguracji DQN. Nie oznacza to monotonicznej poprawy na każdym etapie treningu, co widać na krzywych walidacyjnych, ale rezultat po 100 000 epizodów był dla każdego ziarna lepszy niż wynik odpowiadającego mu modelu po 50 000 epizodów.

Dla REINFORCE średnie EV wzrosło z $-0,3211$ do $-0,2943$. Średnia różnica wyniosła $0,0268$, a więc punktowo była nawet większa niż dla DQN. Jej 95-procentowy



Rys. 7.9. Trajektorie walidacyjnego EV dla pięciu ziaren wybranych konfiguracji DQN i REINFORCE

przedział ufności był jednak równy $[-0,0362, 0,0898]$ i obejmował zero.

Wynik ten był konsekwencją dużego zróżnicowania zmian pomiędzy poszczególnymi ziarnami. Niektóre przebiegi REINFORCE poprawiły się znacznie, podczas gdy inne praktycznie nie zmieniły zachowania lub uzyskały nieznacznie gorszy wynik. Wydłużenie treningu poprawiło więc średni rezultat tej konfiguracji, ale nie usunęło problemu niestabilności procesu uczenia.

Wyniki rozszerzenia budżetu podsumowano w tabeli 7.6.

Tabela 7.6. Wpływ zwiększenia budżetu treningowego z 50 000 do 100 000 epizodów dla wybranej konfiguracji FEATURES każdego algorytmu

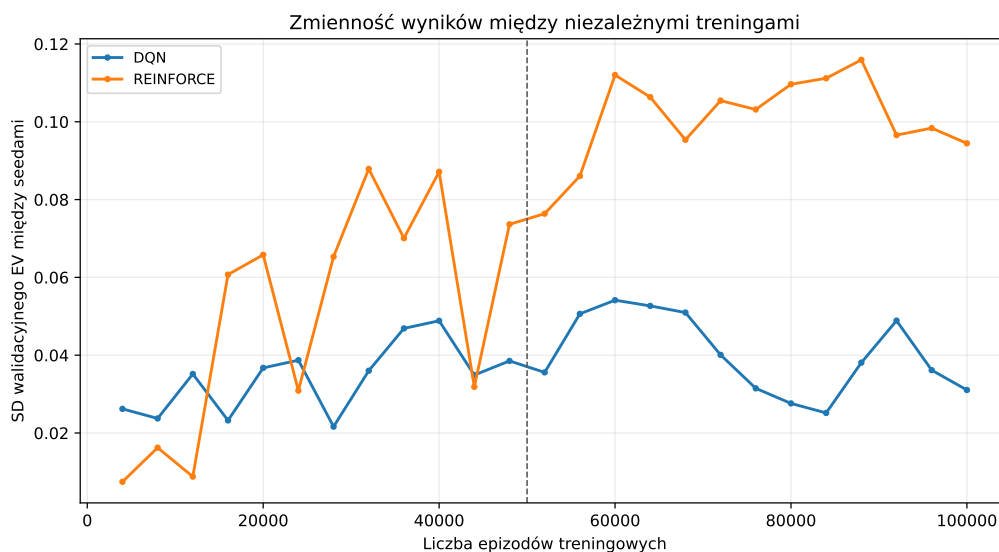
Algorytm	EV 50k	EV 100k	Różnica	95% CI
DQN	-0,4076	-0,3859	0,0217	[0,0022, 0,0412]
REINFORCE	-0,3211	-0,2943	0,0268	[-0,0362, 0,0898]

Rozszerzony eksperyment jest traktowany jako analiza uzupełniająca.

7.7. Stabilność procesu uczenia

W celu przedstawienia powtarzalności procesu treningowego obliczono dla kolejnych punktów kontrolnych odchylenie standardowe walidacyjnego EV pomiędzy pięcioma ziarnami wybranych konfiguracji.

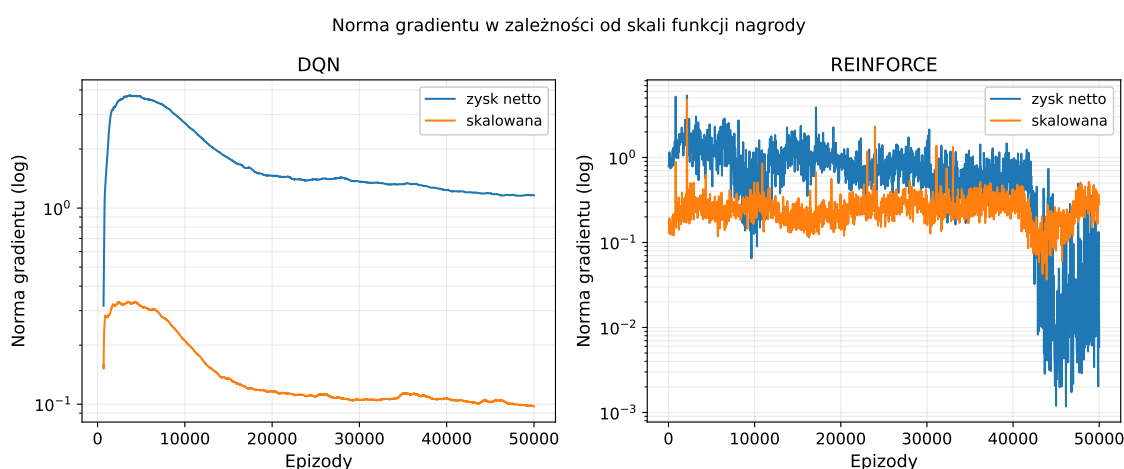
Wyniki przedstawiono na rysunku 7.10. Na osi poziomej znajduje się liczba epizodów treningowych, a na osi pionowej odchylenie standardowe walidacyjnego EV pomiędzy pięcioma ziarnami wybranych konfiguracji DQN i REINFORCE. Krzywa REINFORCE przez większość treningu leży wyraźnie powyżej krzywej DQN i po 50 000 epizodach nadal rośnie, podczas gdy zmienność DQN pozostaje w przybliżeniu na tym samym poziomie.



Rys. 7.10. Zmienność walidacyjnego EV między ziarnami podczas treningu

DQN utrzymywał relatywnie niewielki rozrzut pomiędzy niezależnymi przebiegami. REINFORCE wykazywał natomiast okresy znacznie większej zmienności, co jest zgodne zarówno z krzywymi uczenia, jak i wynikami końcowej ewaluacji. Oznacza to, że w badanym ustawieniu REINFORCE częściej prowadził do jakościowo różnych polityk.

Na rysunku 7.11 porównano przebieg normy gradientu dla obu wariantów funkcji nagrody, dla konfiguracji wykorzystujących reprezentację FEATURES. Na osi poziomej znajduje się liczba epizodów treningowych, a na osi pionowej, w skali logarytmicznej, norma gradientu uśredniona po pięciu ziarnach, dla DQN dodatkowo wygładzona oknem 500 epizodów ze względu na aktualizację co krok środowiska. Norma gradientu dla nagrody skalowanej jest w obu algorytmach systematycznie niższa niż dla zysku netto, choć różnica skali jest znacznie większa dla DQN.



Rys. 7.11. Norma gradientu w zależności od skali funkcji nagrody

Skalowanie nagrody wyraźnie zmieniało skalę gradientów powstających podczas optymalizacji. Nie oznacza to jednak automatycznie poprawy końcowej jakości polityki. Jak pokazały wyniki z sekcji 7.3, mniejsza skala sygnału optymalizacyjnego nie przekładała się na uniwersalną przewagę skalowanej funkcji nagrody.

Analiza gradientów pokazuje, że zmiana funkcji nagrody wpływa na przebieg aktualizacji parametrów, ale sama kontrola skali gradientu nie rozwiązuje problemów związanych ze stabilnością uczenia ani nie gwarantuje uzyskania lepszej strategii.

7.8. Analiza zachowania wyuczonych polityk

W trakcie analizy decyzji podejmowanych przez wybrane konfiguracje DQN i REINFORCE zaobserwowano u części modeli REINFORCE degenerację deterministycznej strategii ewaluacyjnej do jednej akcji przed flopem, zjawisko opisywane w literaturze jako kolaps polityki (ang. *policy collapse*). Polega ono na tym, że wybierana przez agenta akcja argmax przestaje zależeć od obserwowanego stanu i niezależnie od niego jest zawsze taka sama. Test opisany w tej sekcji sprawdza właśnie tę zaobserwowaną akcję argmax, a nie zmierzony rozkład prawdopodobieństwa całej polityki, więc nie dowodzi, że zależność od

stanu zanika w całym rozkładzie, a jedynie w wybieranej ostatecznie decyzji.

Według Volodymyra Mniha i współautorów, dodanie regularyzacji entropii polityki do funkcji celu poprawia eksplorację poprzez ograniczanie przedwczesnej zbieżności do suboptymalnych, deterministycznych polityk [33]. Jak opisano w podrozdziale 5.4, zastosowana w tej pracy konfiguracja REINFORCE nie wykorzystuje ani regularyzacji entropii, ani punktu odniesienia (ang. *baseline*). W szczególności brak regularyzacji entropii oznacza, że metoda nie ma mechanizmu ograniczającego przedwczesną zbieżność do polityki deterministycznej.

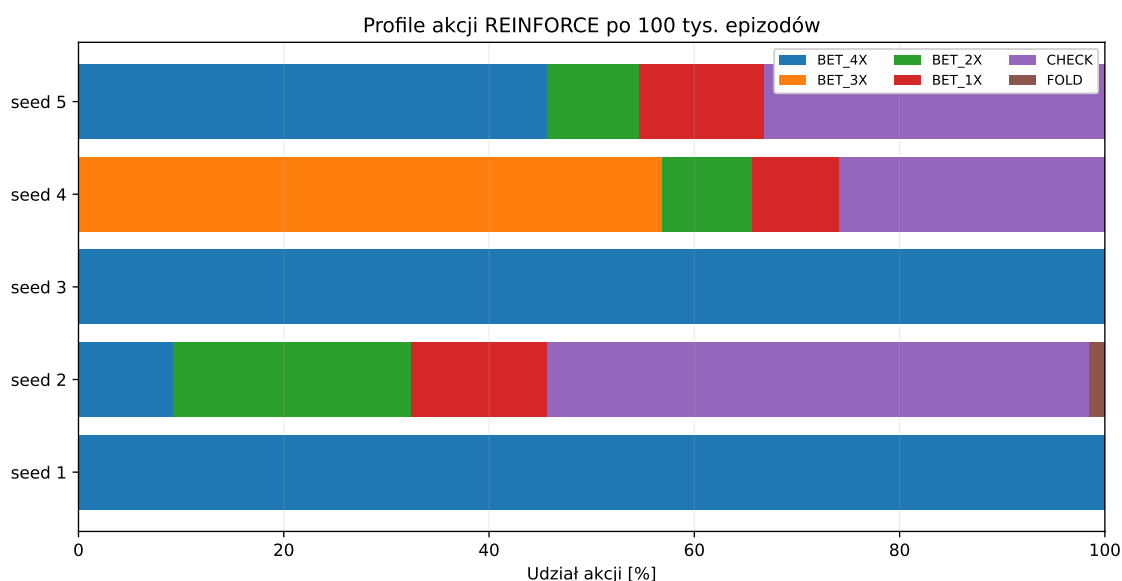
Aby ocenić skalę tego zjawiska, dla każdej z ośmiu podstawowych konfiguracji sprawdzono, dla ilu z pięciu ziaren wybierana akcja przed flopem była identyczna we wszystkich 100 000 rozdaniach końcowej ewaluacji, niezależnie od obserwowanego stanu. W praktyce mierzona jest więc degeneracja decyzji przed flopem. Jeżeli zdegenerowaną akcją jest BET_4X, rozdanie kończy się natychmiast rozstrzygnięciem, więc pozostała część polityki, dotycząca flopu i rivera, nie jest w takim rozdaniu w ogóle wykorzystywana, a zaobserwowana degeneracja decyzji preflop jest równoważna degeneracji całej ścieżki decyzyjnej. Dla żadnej z czterech konfiguracji DQN nie zaobserwowano takiej degeneracji. Dla REINFORCE zjawisko to okazało się natomiast dominujące, co przedstawiono w tabeli 7.7.

Tabela 7.7. Liczba ziaren REINFORCE z polityką zdegenerowaną do jednej akcji przed flopem, spośród pięciu, po 50 000 epizodów

Reprezentacja	Nagroda	Zdegenerowane ziarna
RAW	zysk netto	3 / 5
RAW	skalowana	5 / 5
FEATURES	zysk netto	5 / 5
FEATURES	skalowana	2 / 5

W piętnastu spośród dwudziestu przebiegów REINFORCE zaobserwowano degenerację decyzji przed flopem do jednej akcji. Nie jest to więc wyjątek dotyczący jednej konfiguracji, lecz częsty wynik w całym zbiorze przebiegów REINFORCE. Co istotne, konfiguracja REINFORCE FEATURES ze skalowaną funkcją nagrody, która osiągnęła najwyższe średnie EV spośród wariantów REINFORCE, jest jednocześnie tą, w której degeneracja wystąpiła najrzadziej. Dalsza analiza zachowania prowadzona jest właśnie na tej konfiguracji, ponieważ zawiera ona jednocześnie modele zdegenerowane i zróżnicowane, co pozwala porównać obie grupy w ramach tego samego zestawu ziaren treningowych.

Na rysunku 7.12 przedstawiono rozkład akcji pięciu modeli tej konfiguracji po rozszerzeniu treningu do 100 000 epizodów. Na osi poziomej znajduje się udział poszczególnych akcji w końcowej ewaluacji. Na osi pionowej przedstawiono pięć ziaren. Każdy pasek przedstawia pełny rozkład akcji jednego modelu. Trzy z pięciu pasków składają się z wielu kolorów odpowiadających różnym akcjom, natomiast dwa są jednokolorowe na całej długości.



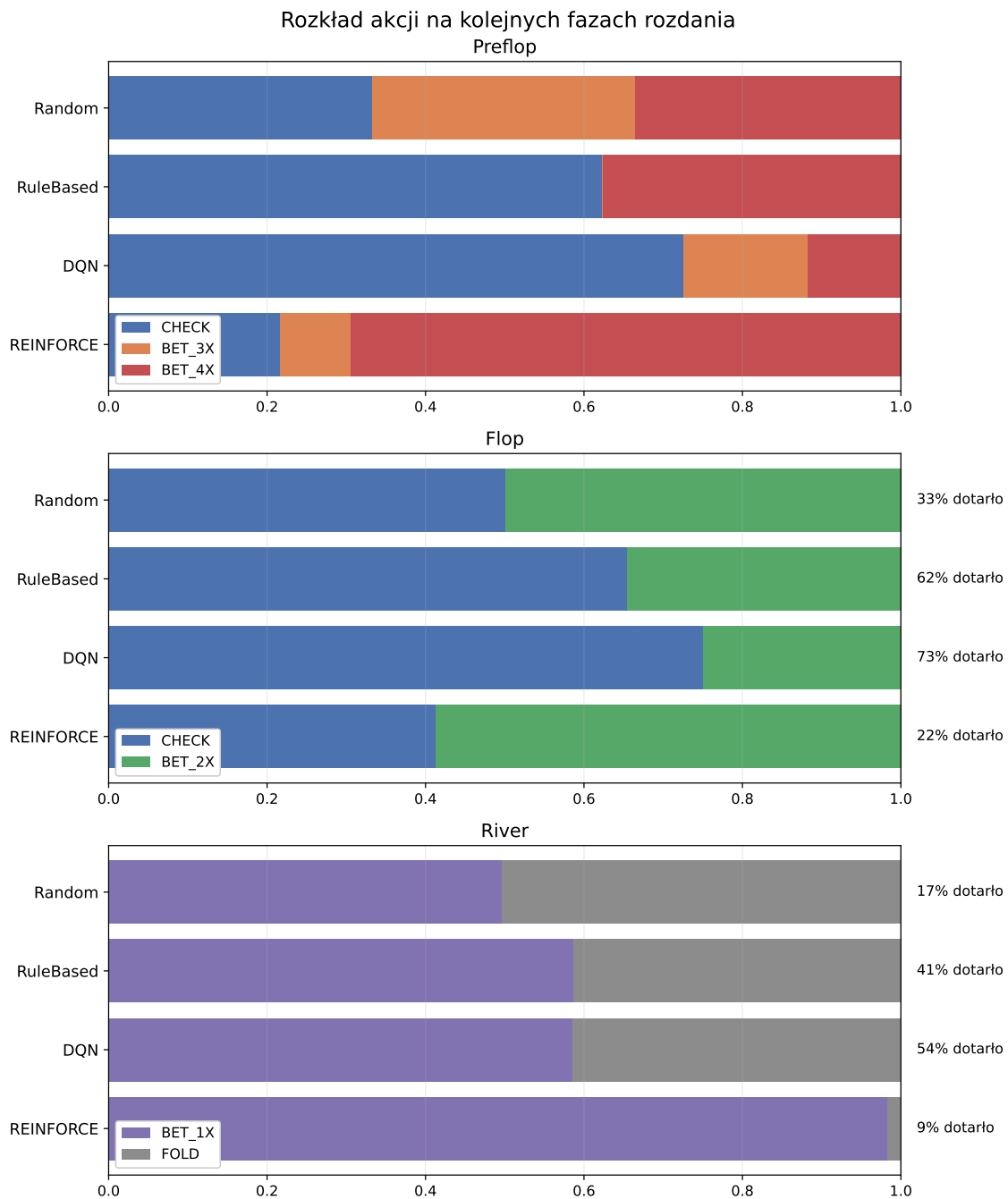
Rys. 7.12. Profile akcji pięciu modeli REINFORCE FEATURES ze skalowaną funkcją nagrody

Profile poszczególnych ziaren różnią się wyraźnie. Dwa z pięciu modeli wybierały akcję BET_4X we wszystkich 100 000 rozdaniach końcowej ewaluacji. Oznacza to, że polityka tych modeli uległa degeneracji do bardzo prostej strategii, w której decyzja przed flopem przestała zależeć od obserwowanego stanu gry.

Pozostałe przebiegi prowadziły do bardziej zróżnicowanych polityk, które podejmowały również decyzje zależne od kolejnych faz rozdania. Wyjaśnia to część wysokiej zmienności wyników REINFORCE pomiędzy ziarnami. Ta sama konfiguracja treningowa może prowadzić zarówno do polityki wykorzystującej informacje ze stanu, jak i do rozwiązania zdominowanego przez jedną akcję.

Zwiększenie budżetu treningowego do 100 000 epizodów nie usunęło jednak tego zjawiska. Dwa ziarna pozostające w prostej polityce po 50 000 epizodów nie zmieniły jej również podczas dalszego treningu. Oznacza to, że sam wzrost liczby epizodów nie wystarczył do wyprowadzenia tych przebiegów z osiągniętego rozwiązania.

Zbadano też zachowania agentów poprzez porównanie rozkładu akcji na kolejnych fazach rozdania. Na rysunku 7.13 przedstawiono ten rozkład dla agentów Random i RuleBased oraz najlepszych walidacyjnie konfiguracji DQN i REINFORCE. Trzy panele odpowiadają preflopowi, flopowi i riverowi, oś pozioma w każdym z nich przedstawia udział poszczególnych akcji wśród rozdań, które faktycznie dotarły do danej fazy, a przy panelach flopu i riveru dodatkowo podano, jaki odsetek wszystkich rozdań do nich dotarł. Dla DQN i REINFORCE rozdania pochodzą ze wszystkich pięciu ziaren danej konfiguracji, a nie z jednego wybranego przebiegu. REINFORCE dociera do riveru w zaledwie 9% rozdań, podczas gdy dla DQN odsetek ten wynosi 54%. Jedną z głównych przyczyn tej różnicy jest częste wykonywanie zakładu już przed flopem, po którym dalsze decyzje gracza nie są wymagane.



Rys. 7.13. Rozkład akcji na kolejnych fazach rozdania

Przeanalizowano również zachowanie agentów poprzez porównanie decyzji przed flopem dla poszczególnych układów początkowych. Na rysunku 7.14 przedstawiono dominujące decyzje agenta regułowego oraz wybranych walidacyjnie konfiguracji DQN i REINFORCE. Trzy panele odpowiadają trzem strategiom, a układ trzynastu na trzynastu pól w każdym z nich odpowiada wszystkim kombinacjom dwóch kart początkowych, uporządkowanym według rangi. Pola na przekątnej odpowiadają parom, pola powyżej przekątnej układom w tym samym kolorze (ang. *suited*), a pola poniżej przekątnej układom w różnych kolorach (ang. *offsuit*). Kolor pola oznacza dominującą akcję wybraną dla danego układu. Dla DQN i REINFORCE decyzje pochodzą ze wszystkich pięciu ziaren danej konfiguracji, a nie z jednego wybranego przebiegu. Agent regułowy tworzy wyraźny, przekątny podział pól, natomiast u konfiguracji RL taki podział jest znacznie słabiej widoczny albo nie występuje wcale.

Należy zaznaczyć, że agent regułowy posiada wyraźnie zdefiniowany obszar układów początkowych, dla których wykonuje zakład BET_4X, zgodny z regułami strategii opisanymi w rozdziale 4.4. Nauczone polityki tworzą inne granice decyzyjne, wynikające wyłącznie z procesu optymalizacji oczekiwanej nagrody.

Porównanie pokazuje, że modele RL nie odtworzyły wprost strategii agenta regułowego. Jednocześnie ich zachowanie nie jest równoważne strategii losowej. Szczególnie dla konfiguracji DQN widoczne są powtarzalne zależności pomiędzy układem początkowym i wybieraną akcją. Oznacza to, że w trakcie treningu modele nauczyły się wykorzystywać część informacji zawartej w obserwowanym stanie, mimo że końcowa jakość strategii pozostawała niższa od jakości agenta regułowego.

7.9. Dyskusja wyników

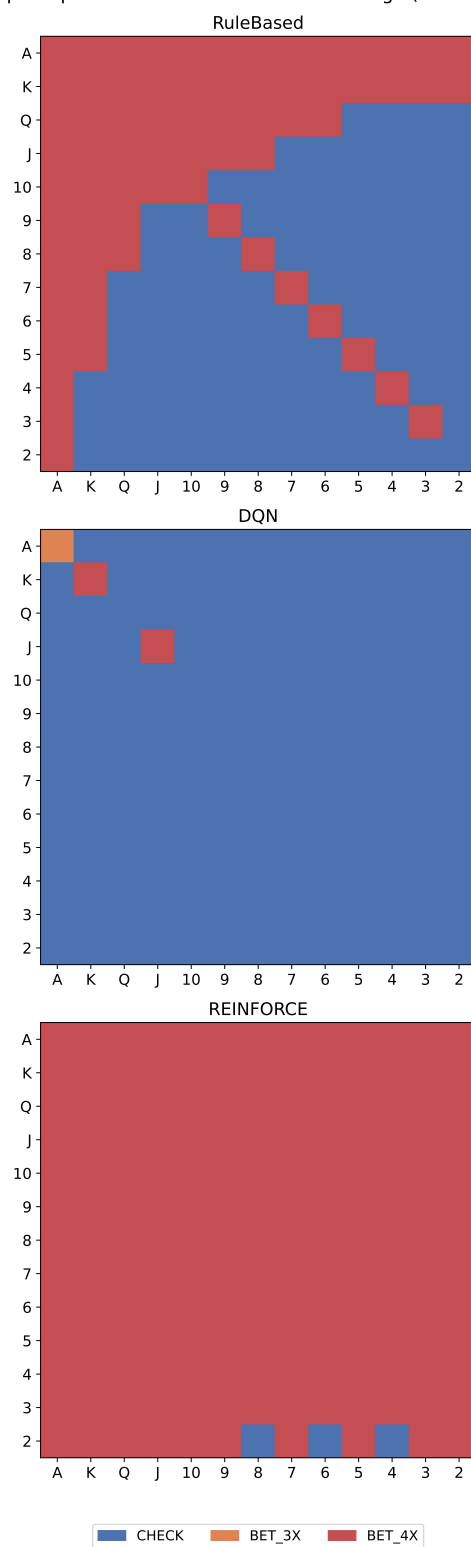
Dotychczasowe eksperymenty pokazują, że na jakość strategii w Ultimate Texas Hold'em wpływa nie tylko wybór algorytmu uczenia, ale również sposób przedstawienia stanu i właściwości procesu optymalizacji.

Najbardziej jednoznaczny efekt zaobserwowano dla reprezentacji stanu w algorytmie DQN. Rozszerzenie RAW o cechy pokerowe w reprezentacji FEATURES poprawiło wynik dla obu funkcji nagrody i efekt ten wystąpił konsekwentnie dla pięciu sparowanych ziaren treningowych. Wynik ten wskazuje, że w badanym budżecie treningowym jawne dostarczenie informacji o strukturze sytuacji pokerowej ułatwia DQN wykorzystanie dostępnej obserwacji.

Dla REINFORCE analogicznego, stabilnego efektu reprezentacji stanu nie zaobserwowano. Najwyższe średnie EV uzyskało połączenie FEATURES ze skalowaną nagrodą, jednak było ono jednocześnie konfiguracją o największej zmienności pomiędzy ziarnami. Wynik średni nie odzwierciedla więc w tym przypadku typowego rezultatu pojedynczego treningu równie dobrze jak dla DQN.

Wpływ skalowania nagrody również nie był jednoznaczny. Dla DQN korzystają-

Decyzja preflop w zależności od układu startowego (dominująca akcja)



Rys. 7.14. Dominujące decyzje przed flopem w zależności od układu dwóch kart początkowych

cego z reprezentacji RAW nagroda skalowana prowadziła do wyższego EV, natomiast po zastosowaniu FEATURES różnica pomiędzy funkcjami nagrody była niewielka. Diagnostyka norm gradientu pokazała, że zmiana skali nagrody wyraźnie wpływała na obserwowaną wielkość sygnału optymalizacyjnego, ale samo ograniczenie tej skali nie gwarantuje poprawy końcowej polityki.

Porównując oba algorytmy przy podstawowym budżecie 50 000 epizodów, REINFORCE osiągnął wyższe średnie EV od DQN dla każdego odpowiadającego sobie połączenia reprezentacji stanu i funkcji nagrody. Nie oznacza to jednak jednoznacznej przewagi REINFORCE. Wyniki tego algorytmu były bardziej zależne od ziarna treningowego, a większość przebiegów, piętnaście z dwudziestu, prowadziła do degeneracji polityki do jednej akcji niezależnej od stanu gry.

DQN osiągał niższe średnie EV. Dla konfiguracji wybranej do rozszerzonego treningu jego wyniki były jednak bardziej powtarzalne między ziarnami niż dla odpowiadającej jej konfiguracji REINFORCE. Dla części pozostałych wariantów REINFORCE niski rozrzut końcowego EV wynikał natomiast z tego, że wiele ziaren zbiegało do podobnej, zdegenerowanej polityki, a nie z rzeczywistej stabilności procesu uczenia. Również eksperyment z większym budżetem treningowym wskazał na różnicę pomiędzy algorytmami. Wszystkie pięć ziaren wybranej do rozszerzonego treningu konfiguracji DQN poprawiło wynik po zwiększeniu budżetu do 100 000 epizodów, natomiast dla odpowiadającej konfiguracji REINFORCE poprawa była silnie zależna od konkretnego ziarna.

Żadna z badanych konfiguracji RL nie osiągnęła jakości strategii RuleBased. Agent regułowy uzyskał w końcowej ewaluacji EV równe $-0,0106$, podczas gdy najlepsze średnie wyniki podstawowych konfiguracji RL pozostawały wyraźnie ujemne. Pokazuje to, że przy zastosowanych architekturach i budżecie treningowym prosta strategia wykorzystująca wiedzę dziedzinową pozostaje znacznie skuteczniejsza.

Niemniej jednak nie oznacza to, że modele RL nie wykorzystwały żadnej użytecznej informacji o strukturze problemu. Większość konfiguracji osiągnęła wynik wyższy od agenta losowego, a analiza decyzji pokazała zależności pomiędzy obserwowanym stanem i podejmowanymi akcjami. Wyniki sugerują więc, że badane metody były zdolne do uczenia się strategii, lecz jakość tej strategii pozostawała ograniczona.

REINFORCE pozostał metodą bez funkcji bazowej, sieci wartości i dodatkowej regularyzacji entropii. Reprezentacja FEATURES zawiera natomiast ręcznie zaprojektowane cechy pokerowe, dlatego jej przewaga nad RAW pokazuje wartość tej konkretnej informacji dziedzinowej, a nie uniwersalną przewagę każdej reprezentacji o większej liczbie cech.

Z tego względu wyników nie traktuje się jako rozstrzygnięcia o ogólnej wyższości REINFORCE nad DQN, ani odwrotnie. Pokazują one natomiast wyraźnie, że w badanym środowisku DQN i REINFORCE reagują inaczej na reprezentację stanu, skalę nagrody i zwiększenie budżetu treningowego. Różnica pomiędzy średnią jakością polityki a stabil-

nością jej uzyskiwania stanowi jeden z ważniejszych rezultatów przeprowadzonych eksperymentów.

PODSUMOWANIE I DALSZY ROZWÓJ PRACY

W tej pracy sprawdzono, jak wybrane metody uczenia przez wzmacnianie sprawdzają się w podejmowaniu decyzji w Ultimate Texas Hold'em oraz jak na jakość uzyskiwanej strategii wpływa sposób reprezentacji stanu i konstrukcja funkcji nagrody.

W tym celu przygotowano kompletne środowisko symulacyjne gry, mechanizm prowadzenia treningu i ewaluacji, agentów bazowych oraz agentów wykorzystujących DQN i REINFORCE. Następnie przeprowadzono eksperyment obejmujący osiem podstawowych konfiguracji, z których każdą wytrenowano dla pięciu ziaren. Kończącą ocenę modeli wykonano na wspólnym harmonogramie 100 000 rozdań.

Realizacja celu pracy

Pierwszym celem pracy było przygotowanie środowiska pozwalającego na prowadzenie powtarzalnych eksperymentów z wykorzystaniem Ultimate Texas Hold'em. W ramach realizacji tego celu zaimplementowano przebieg pełnego rozdania, ocenę układów pokerowych, zasady wykonywania zakładów oraz ich końcowe rozliczenie.

Na bazie środowiska przygotowano wspólną infrastrukturę dla agentów. Pozwoliła ona uruchamiać te same rozdania dla różnych strategii, zapisywać wyniki i wykonywane akcje oraz porównywać agentów na wspólnych harmonogramach talii. Jako punkty odniesienia wykorzystano agenta losowego i agenta regułowego.

Kolejnym etapem była implementacja dwóch metod uczenia przez wzmacnianie. DQN reprezentuje podejście oparte na aproksymacji funkcji wartości akcji, natomiast REINFORCE bezpośrednio optymalizuje politykę. Dla obu metod wykorzystano te same dwa warianty reprezentacji stanu oraz te same dwa warianty funkcji nagrody, dzięki czemu można było porównywać wpływ poszczególnych elementów konfiguracji.

Za istotną część praktycznego celu pracy uznano również przygotowanie powtarzalnego procesu eksperymentalnego. Oddzielono dane wykorzystywane podczas treningu, okresowej walidacji oraz końcowej ewaluacji. Wszystkie modele w końcowym porównaniu oceniono na tym samym harmonogramie rozdań. Pozwoliło to ograniczyć wpływ przypadkowej kolejności kart na porównania pomiędzy strategiami.

W rezultacie zaprojektowano kompletny proces obejmujący implementację środowiska, trening agentów, zapis modeli, okresową walidację, końcową ewaluację oraz analizę wyników.

Wnioski z przeprowadzonych eksperymentów

Najbardziej jednoznaczny wynik uzyskano dla wpływu reprezentacji stanu na algorytm DQN. W obu wariantach funkcji nagrody reprezentacja FEATURES prowadziła do wyższego końcowego EV niż RAW. Oznacza to, że przy zastosowanym budżecie trenin-

gowym jawne przekazanie agentowi dodatkowych cech opisujących sytuację pokerową ułatwiało DQN uczenie lepszej strategii.

Dla REINFORCE nie zaobserwowano równie wyraźnego efektu reprezentacji stanu. Przy zysku netto wyniki RAW i FEATURES były zbliżone, natomiast dla nagrody skalowanej konfiguracja FEATURES osiągnęła wyższy wynik punktowy, ale jednocześnie charakteryzowała się dużą zmiennością pomiędzy ziarnami. Wpływ reprezentacji na REINFORCE okazał się więc znacznie mniej powtarzalny niż dla DQN.

Przeanalizowano wpływ funkcji nagrody. Skalowanie wyniku netto poprawiło rezultat DQN korzystającego z reprezentacji RAW, jednak przy reprezentacji FEATURES różnica pomiędzy funkcjami nagrody była niewielka. Dla REINFORCE również nie uzyskano wyników pozwalających stwierdzić, że jedna funkcja nagrody wykazuje większą skuteczność od drugiej. Skalowanie zmieniało przebieg optymalizacji i skalę gradientów, ale nie dawało uniwersalnej poprawy jakości końcowej polityki.

Porównanie samych algorytmów również nie daje jednej odpowiedzi na pytanie o lepszą metodę. Przy podstawowym budżecie 50 000 epizodów REINFORCE osiągał wyższe średnie EV niż DQN dla odpowiadających sobie konfiguracji. Jednocześnie jego trening był znacznie bardziej zależny od ziarna i w wielu przebiegach prowadził do przedwczesnej zbieżności decyzji do bardzo prostej polityki.

DQN osiągał niższe średnie wyniki. Wybrana walidacyjnie konfiguracja DQN dawała jednak bardziej powtarzalne wyniki między ziarnami niż odpowiadająca jej konfiguracja REINFORCE. Było to widoczne również po zwiększeniu budżetu treningowego do 100 000 epizodów, dla wybranej konfiguracji DQN wszystkie pięć ziaren poprawiło końcowy wynik, podczas gdy dla odpowiadającej konfiguracji REINFORCE zmiana była znacznie mniej regularna.

Większość konfiguracji uczenia przez wzmacnianie osiągnęła wynik lepszy od agenta losowego, co sugeruje, że wyuczone zachowanie było skuteczniejsze od losowego wyboru legalnej akcji. Dopiero po przeanalizowaniu samych decyzji potwierdzono, że część wytrenowanych modeli faktycznie uzależniała wybieraną akcję od obserwowanego stanu gry. Żadna z badanych konfiguracji nie zbliżyła się jednak do wyniku agenta Rule-Based, którego EV w końcowej ewaluacji wyniosło $-0,0106$.

Odpowiadając na postawiony we wstępie problem badawczy, wybór algorytmu, sposobu reprezentacji stanu oraz funkcji nagrody faktycznie wpływał zarówno na przebieg uczenia, jak i jakość końcowej strategii. Wpływ tych elementów nie był jednak jednakowy dla obu algorytmów. Najbardziej powtarzalnym efektem była poprawa DQN po zastosowaniu reprezentacji FEATURES, natomiast największym problemem REINFORCE okazała się stabilność procesu uczenia.

Uzyskane wyniki pokazują również, że ocena metody wyłącznie na podstawie średniego EV może być niewystarczająca. REINFORCE uzyskiwał wyższe wyniki średnie, ale jednocześnie często zbiegał do bardzo prostych, deterministycznych polityk. Takiego zja-

wiska nie zaobserwowano dla DQN, którego wybrana walidacyjnie konfiguracja dawała też bardziej powtarzalne wyniki między ziarnami. W przypadku zastosowania uczenia przez wzmacnianie w problemie o dużej losowości ważna jest więc nie tylko jakość najlepszej uzyskanej polityki, ale również to, jak często podobną politykę można otrzymać po ponownym przeprowadzeniu treningu.

Ograniczenia pracy

Zakres tej pracy ograniczono pod kilkoma względami, które zostaną teraz krótko podsumowane.

Pierwszym ograniczeniem jest liczba niezależnych przebiegów treningowych. Każdą konfigurację wytrenowano dla pięciu ziaren, co pozwoliło porównywać treningi parami, ale przy tak małej próbie przedziały ufności dla efektów zależnych od ziarna pozostają dość szerokie.

Dla obu algorytmów przyjęto jedną architekturę sieci. Celem eksperymentu było porównanie algorytmów, reprezentacji stanu i funkcji nagrody w tych samych warunkach, a nie wyszukanie najlepszej możliwej konfiguracji dla każdego z nich. Nie oznacza to więc, że DQN albo REINFORCE nie mogłyby osiągnąć lepszego wyniku po innym doborze konfiguracji.

Kolejnym ograniczeniem jest różnica współczynnika dyskontowania między algorytmami. Dla DQN przyjęto $\gamma = 0,99$, natomiast dla REINFORCE $\gamma = 1$. Przy nagrodzie wyłącznie terminalnej różnica ta jest niewielka, ponieważ DQN nieznacznie dyskontuje wynik rozdania w zależności od liczby wcześniejszych kroków oczekiwania, podczas gdy REINFORCE optymalizuje bezpośrednio rzeczywisty wynik finansowy. Oba algorytmy optymalizują więc nieco odmienny cel.

Dotyczy to zwłaszcza REINFORCE. Zastosowano jego podstawową wersję, bez sieci wartości i bez regularyzacji entropii. W szczególności brak regularyzacji entropii oznacza, że metoda nie miała mechanizmu ograniczającego przedwczesną zbieżność do jednej, deterministycznej akcji, co w przeprowadzonych eksperymentach często prowadziło do degeneracji decyzji przed flopem. Ten wynik opisuje więc zachowanie konkretnej, wykorzystanej implementacji, a nie ogólną cechę metod policy gradient.

Ograniczono się również do dwóch wariantów funkcji nagrody, opartych na tym samym wyniku finansowym rozdania i różniących się tylko skalą. Pozwoliło to sprawdzić wpływ skali sygnału nagrody bez zmiany celu agenta, ale nie obejmuje innych sposobów kształtowania nagrody.

Reprezentacja FEATURES zawiera ręcznie dobrane cechy pokerowe. Jej przewaga nad RAW pokazuje więc wartość tej konkretnej wiedzy dziedzinowej, a nie to, że każda bardziej rozbudowana reprezentacja stanu byłaby lepsza.

Reprezentacja RAW koduje karty własne gracza i karty wspólne w kolejności, w jakiej trafiają do obserwacji, a nie w kolejności kanonicznej, na przykład według rangi. Ta

sama para kart własnych może więc zostać zakodowana na dwa różne sposoby w zależności od tego, która karta została rozdana jako pierwsza. Modele musiały nauczyć się tej symetrii samodzielnie, z danych treningowych, a nie miały jej zagwarantowanej przez konstrukcję reprezentacji.

Ograniczenia samego środowiska, w tym brak zakładów pobocznych Trips i Progressive, opisano już w podrozdziale 3.11. Oznacza to, że badany problem decyzyjny obejmuje wyłącznie zakład Play wobec krupiera działającego według stałych reguł, bez adaptacji do zmieniającego się przeciwnika.

Ostatnim ograniczeniem jest zakres eksperymentu z budżetem 100 000 epizodów. Do rozszerzonego treningu wybrano po jednej konfiguracji dla każdego algorytmu na podstawie wyników walidacyjnych, więc część ta jest traktowana jako analiza uzupełniająca, a nie bezpośrednie porównanie wszystkich ośmiu podstawowych konfiguracji.

Możliwe kierunki dalszego rozwoju

Najbardziej oczywistym kierunkiem dalszych prac jest rozwinięcie metody policy gradient. Skoro REINFORCE tak często degenerował się do prostej polityki, naturalnym kierunkiem rozwoju jest dodanie regularyzacji entropii oraz punktu odniesienia (ang. *baseline*) ograniczającego wariancję gradientu, a dalej sprawdzenie metod actor-critic albo PPO, które łączą optymalizację polityki z estymacją wartości stanu i dodatkowymi mechanizmami stabilizującymi trening [34].

Dla DQN warto byłoby szerzej przetestować hiperparametry oraz dłuższy trening. W rozszerzonym eksperymencie wszystkie pięć ziaren poprawiło wynik między 50 000 a 100 000 epizodów, więc interesujące byłoby sprawdzenie, czy ten trend utrzymuje się przy jeszcze większym budżecie.

Kolejnym obszarem badań jest sama reprezentacja obserwacji. W tej pracy porównano surowe kodowanie kart z wersją rozszerzoną o ręcznie zaprojektowane cechy pokerowe. Alternatywą byłaby chociażby reprezentacja uczona przez samą sieć.

Warto byłoby także zwiększyć liczbę ziaren treningowych. Pozwoliłoby to dokładniej opisać rozkład wyników każdej konfiguracji i lepiej ocenić, jak często REINFORCE faktycznie prowadzi do zdegenerowanej polityki.

Środowisko można też rozszerzyć o dodatkowe warianty gry i zakłady poboczne. Nie zmieniałoby to głównego problemu decyzyjnego opisanego w tej pracy, ale pozwoliłoby wykorzystać przygotowany silnik do szerszych eksperymentów.

Wnioski końcowe

Żadna z ośmiu badanych konfiguracji nie okazała się najlepsza pod każdym względem. REINFORCE osiągał wyższe średnie EV, ale był podatny na degenerację polityki. DQN uzyskiwał niższe wyniki, natomiast reprezentacja FEATURES dawała mu wyraźną i powtarzalną poprawę względem RAW, a wydłużenie treningu do 100 000 epizodów

poprawiło wynik wszystkich pięciu ziaren wybranej do tego eksperymentu konfiguracji DQN.

Żadna z metod uczenia przez wzmacnianie nie osiągnęła wyników uzyskanych przez agenta regułowego. Zaobserwowano jednak konkretne zależności między reprezentacją stanu, skalą nagrody, algorytmem i stabilnością treningu. Najwyższe EV nie było więc jedynym wynikiem tej pracy, równie ważne okazało się to, jak poszczególne elementy konfiguracji wpływają na sam proces uczenia i na zachowanie wytrenowanej polityki.

Przygotowano środowisko i cały proces eksperymentalny w taki sposób, że może on być punktem wyjścia do dalszych badań nad uczeniem przez wzmacnianie w zagadnieniu Ultimate Texas Hold'em.

BIBLIOGRAFIA

- [1] M. Bowling, N. Burch, M. Johanson i O. Tammelin, „Heads-up Limit Hold'em Poker is Solved,” *Science*, t. 347, 2015, Dostęp: 2026-09-01. DOI: 10.1126/science.1259433
- [2] Tulalip Resort Casino, *Ultimate Texas Hold'em Game Rules*, https://www.everythingtulalip.com/wp-content/uploads/2024/04/ONEclub_0920_GameRules_RackCard_UltimateTexasHoldem-WEB.pdf, Dostęp: 2026-09-01.
- [3] Wizard of Odds, *Ultimate Texas Hold'em*, <https://wizardofodds.com/games/ultimate-texas-hold-em/>, Dostęp: 2026-09-01.
- [4] Nevada Gaming Control Board, *Ultimate Texas Hold'em - Rules of Play*, https://www.gaming.nv.gov/siteassets/content/divisions/enforcement/rules-of-play/Ultimate_Texas_Hold_em.pdf, Dostęp: 2026-09-01.
- [5] E. W. Packel, *The Mathematics of Games and Gambling*, 2nd. Mathematical Association of America, 2006, t. 28, Dostęp: 2026-09-01.
- [6] J. B. Maverick, *Understanding the House Edge: How Casinos Ensure Profit*, <https://www.investopedia.com/articles/personal-finance/110415/why-does-house-always-win-look-casino-profitability.asp>, Dostęp: 2026-09-01.
- [7] Wizard of Odds, *What is the House Edge?* <https://wizardofodds.com/gambling/house-edge/>, Dostęp: 2026-09-01.
- [8] R. S. Sutton i A. G. Barto, *Reinforcement Learning: An Introduction*, 2 wyd. Cambridge, Massachusetts: The MIT Press, 2018, Dostęp: 2026-09-01.
- [9] S. Russell i P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th. Pearson, 2021, Dostęp: 2026-09-01.
- [10] D. C. Montgomery i G. C. Runger, *Applied Statistics and Probability for Engineers*, 7th. Wiley, 2018, Dostęp: 2026-09-01.
- [11] M. Lewczuk, „Jak działa reinforcement learning. Wprowadzenie dla początkujących,” 2024, Dostęp: 2026-09-01.
- [12] J. Bień, „Uczenie maszynowe: przygotowanie danych – kodowanie zmiennych kategoryalnych,” 2022, Dostęp: 2026-09-01.
- [13] Google for Developers, „Słowniczek systemów uczących się,” 2026, Dostęp: 2026-09-01.
- [14] Wydział Matematyki, Informatyki i Mechaniki, Uniwersytet Warszawski, „Sztuczna inteligencja/SI Moduł 13 - Uczenie się ze wzmocnieniem,” 2023, Dostęp: 2026-09-01.
- [15] V. Mnih i in., „Human-level control through deep reinforcement learning,” *Nature*, t. 518, 2015, Dostęp: 2026-09-01. DOI: 10.1038/nature14236
- [16] A. Y. Ng, D. Harada i S. Russell, „Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping,” w: *Proceedings of the Sixteenth International Conference on Machine Learning*, Dostęp: 2026-09-01, 1999.

- [17] T. Miller, *Reward Shaping*, w: *Mastering Reinforcement Learning*, <https://gibberblot.github.io/rl-notes/single-agent/reward-shaping.html>, Dostęp: 2026-09-01.
- [18] C. J. C. H. Watkins i P. Dayan, „Q-learning,” *Machine Learning*, t. 8, 1992, Dostęp: 2026-09-01. DOI: 10.1007/BF00992698
- [19] GeeksforGeeks, „Q-Learning in Reinforcement Learning,” 2026, Dostęp: 2026-09-01.
- [20] V. Nair i G. E. Hinton, „Rectified Linear Units Improve Restricted Boltzmann Machines,” Dostęp: 2026-09-01, 2010.
- [21] GeeksforGeeks, „Epsilon-Greedy Algorithm in Reinforcement Learning,” 2023, Dostęp: 2026-09-01.
- [22] R. S, „Introduction to Experience Replay for Off-Policy Deep Reinforcement Learning,” 2022, Dostęp: 2026-09-01.
- [23] L.-J. Lin, „Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching,” *Machine Learning*, t. 8, nr 3-4, 1992, Dostęp: 2026-09-01. DOI: 10.1007/BF00992699
- [24] K. Pykes, „Understanding the Bellman Equation in Reinforcement Learning,” 2024, Dostęp: 2026-09-01.
- [25] Zilliz, „How does the discount factor (gamma) affect RL training?,” 2025, Dostęp: 2026-09-01.
- [26] Europejski Instytut Certyfikacji IT, „Jakie jest znaczenie współczynnika dyskonta (gamma) w kontekście uczenia się przez wzmacnianie i jaki ma on wpływ na szkolenie i wydajność agenta DRL?,” 2024, Dostęp: 2026-09-01.
- [27] R. Alake, „Loss Functions in Machine Learning Explained,” 2026, Dostęp: 2026-09-01.
- [28] P. J. Huber, „Robust Estimation of a Location Parameter,” *The Annals of Mathematical Statistics*, t. 35, nr 1, 1964, Dostęp: 2026-09-01. DOI: 10.1214/aoms/1177703732
- [29] GeeksforGeeks, „Huber Loss Function in Machine Learning,” 2025, Dostęp: 2026-09-01.
- [30] D. P. Kingma i J. Ba, *Adam: A Method for Stochastic Optimization*, <https://arxiv.org/abs/1412.6980>, Dostęp: 2026-09-01, 2015. DOI: 10.48550/arXiv.1412.6980
- [31] R. J. Williams, „Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning,” *Machine Learning*, t. 8, 1992, Dostęp: 2026-09-01. DOI: 10.1007/BF00992696
- [32] I. Goodfellow, Y. Bengio i A. Courville, *Deep Learning*. Cambridge, Massachusetts: MIT Press, 2016, Dostęp: 2026-09-01.
- [33] V. Mnih i in., „Asynchronous Methods for Deep Reinforcement Learning,” Dostęp: 2026-09-01, t. 48, 2016. DOI: 10.48550/arXiv.1602.01783
- [34] J. Schulman, F. Wolski, P. Dhariwal, A. Radford i O. Klimov, *Proximal Policy Optimization Algorithms*, <https://arxiv.org/abs/1707.06347>, Dostęp: 2026-09-01, 2017. DOI: 10.48550/arXiv.1707.06347

Spis rysunków

3.1	Architektura i przepływ informacji w środowisku UTH	21
3.2	Maszyna stanów pojedynczego rozdania UTH	29
5.1	Kodowanie one-hot pojedynczej karty	46
5.2	FEATURES jako RAW rozszerzone o 20 cech domenowych	46
5.3	Liniowy spadek wartości epsilon w trakcie treningu	49
5.4	Sieć Q i sieć docelowa	50
5.5	Przepływ gradientu podczas aktualizacji sieci Q	52
5.6	Wybór akcji w Policy Gradient	52
5.7	Zwrot G_t w epizodzie REINFORCE	53
5.8	DQN aktualizuje sieć na bieżąco, ponownie wykorzystując stare przejścia z bufora. REINFORCE zbiera całą porcję epizodów z aktualnej polityki i wykorzystuje go tylko raz	55
6.1	Przebieg jednego eksperymentu	57
7.1	Końcowe EV ośmiu konfiguracji po 50 000 epizodów treningowych	65
7.2	Zbieganie średniego wyniku do EV na współdzielonym harmonogramie rozdań	66
7.3	Wpływ reprezentacji stanu na końcowe EV	67
7.4	Wpływ funkcji nagrody na końcowe EV	68
7.5	Wpływ algorytmu na końcowe EV	69
7.6	Przebieg uczenia czterech konfiguracji DQN	70
7.7	Przebieg uczenia czterech konfiguracji REINFORCE	71
7.8	Przebieg walidacyjnego EV wybranych konfiguracji DQN i REINFORCE do 100 000 epizodów	72
7.9	Trajektorie walidacyjnego EV dla pięciu ziaren wybranych konfiguracji DQN i REINFORCE	73
7.10	Zmienność walidacyjnego EV między ziarnami podczas treningu	74
7.11	Norma gradientu w zależności od skali funkcji nagrody	75
7.12	Profile akcji pięciu modeli REINFORCE FEATURES ze skalowaną funkcją nagrody	77
7.13	Rozkład akcji na kolejnych fazach rozdania	78

7.14 Dominujące decyzje przed flopem w zależności od układu dwóch kart początkowych	80
---	----

Spis tabel

2.1	Ranking układów pokerowych od najsilniejszego do najsłabszego	12
2.2	Dostępne decyzje gracza w Ultimate Texas Hold'em	13
2.3	Tabela wypłat zakładu Blind przyjęta w modelu środowiska	14
2.4	Przykładowe wartości przewagi kasyna dla wybranych gier kasynowych .	14
3.1	Podział odpowiedzialności pomiędzy modułami środowiska UTH	21
3.2	Liczba kart przechowywanych w stanie w poszczególnych fazach	26
3.3	Rozliczenie zakładów po porównaniu układów	31
3.4	Tabela wypłat zakładu Blind	31
3.5	Publiczny interfejs klasy UTHGame	34
4.1	Układy początkowe uprawniające do zakładu Play 4× przed flopem	40
4.2	Wyniki agentów bazowych dla 10 000 rozdań	43
4.3	Rozkład wyników agentów bazowych dla 10 000 rozdań	43
5.1	20 cech domenowych reprezentacji FEATURES	47
5.2	Hiperparametry treningu DQN i REINFORCE	55
6.1	Konfiguracje wykorzystane w eksperymentach	58
7.1	Wyniki końcowej ewaluacji modeli po 50 000 epizodów treningowych . .	63
7.2	Wyniki końcowej ewaluacji agentów bazowych	64
7.3	Wpływ reprezentacji stanu na EV	66
7.4	Wpływ funkcji nagrody na EV	67
7.5	Wpływ algorytmu na końcowe EV	69
7.6	Wpływ zwiększenia budżetu treningowego z 50 000 do 100 000 epizodów dla wybranej konfiguracji FEATURES każdego algorytmu	74
7.7	Liczba ziaren REINFORCE z polityką zdegenerowaną do jednej akcji przed flopem, spośród pięciu, po 50 000 epizodów	76